운영체제 활용

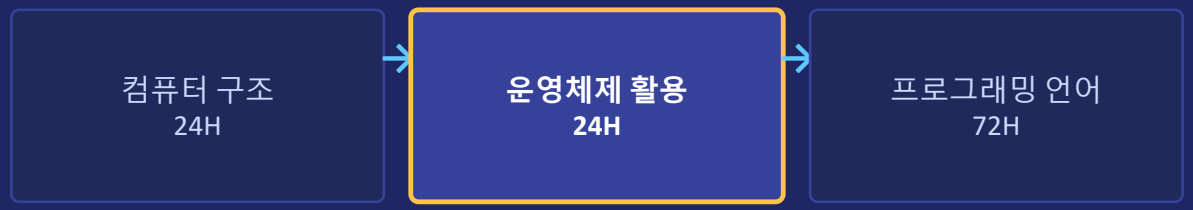
HYUNDAI AI Insight Campus · 온 디바이스 AI

개념 → 인터랙티브 시뮬레이션 → 디지털 워크시트 → 장애 분석



과정 위치와 24시간 학습 범위

임베디드 컴퓨팅 Part 1의 연결 구조



🧠 왜 여기서 OS인가?

하드웨어가 어떻게 구성되는지 배운 뒤, 여러 프로그램이 그 하드웨어를 어떻게 안전하게 공유하는지 이해하는 단계입니다.

🔗 뒤 과정과 연결

Linux & Shell, Embedded I/O, AI Runtime, ROS2에서 발생하는 문제를 OS 관점으로 해석할 수 있게 합니다.

▶️ 이론만 하지 않기

모든 핵심 개념은 인터랙티브 시뮬레이션에서 상태 변화와 자원 흐름으로 확인합니다.

✓ 평가 방식

매 챕터 디지털 워크시트에서 예측 → 채점 → 검산 → 설명의 사이클로 실력을 확인합니다.

핵심 관점: “운영체제는 하드웨어를 숨기는 층”을 넘어 “한정된 자원을 여러 작업에 배분하는 정책 시스템”이다.

24시간 후 수강생이 할 수 있어야 하는 것

💬 설명

Process·Thread·Memory·File·Device·Network의 핵심 동작을 자신의 말로 설명

👁️ 관찰

top/free/df/journalctl 등으로 실행 중 시스템의 상태를 읽기

🔮 예측

스케줄링·Paging·Buffer·TCP 같은 흐름에서 다음 음 상태를 예측

🔍 진단

CPU·RAM·Disk·Device·Service 지표를 조합해 Root Root Cause 후보 좁히기

⚡ 연결

온디바이스 AI Pipeline에서 OS 자원 병목과 Thermal 문제를 해석

✅ 검증

수정 후 동일 조건에서 성능·로그·장치 상태를 재확인

수업 운영 방식: 예측 → 시뮬레이션 → 워크시트

1. 개념 설명 → 2. 먼저 예측 → **3. HTML 시뮬레이터** → 4. 결과 관찰 → 5. 워크시트 채점 → 6. 한 문장 설명

✍️ 예측 먼저

시뮬레이터를 먼저 보면 "봤기 때문에 맞힌 것"인지 "이해해서 예측한 것"인지 구분하기 어렵습니다. 그래서 항상 먼저 적습니다.

▶️ 움직임으로 검산

프로세스 이동, Page In, Packet 재전송처럼 정적인 그림으로 이해하기 어려운 개념은 HTML에서 상태 변화로 확인합니다.

✓ 오답은 학습 증거

워크시트 채점 시 틀린 칸 옆에 정답이 표시되고 예측은 잠깁니다. 틀린 답을 지우지 않아 생각의 변화를 남깁니다.

배포 파일 권장 구성 PPTX + HTML 시뮬레이터 + HTML 디지털 워크시트를 같은 폴더에서 운영

추가: 개념 → 워크플로우 → 애니메이션

3일 × 8시간 전체 로드맵

DAY 1

기본 구조와 실행

C01 운영체제 개요
C02 Linux 운영환경
C03 프로세스와 스레드
C04 CPU 스케줄링

8 HOURS

DAY 2

메모리·파일·장치

C05 메모리 관리
C06 파일시스템
C07 장치와 I/O
C08 시스템 콜과 실행

8 HOURS

DAY 3

서비스·네트워크·AI·진단

C09 부팅과 서비스
C10 네트워크
C11 온디바이스 AI 자원
C12 종합 장애분석

8 HOURS

COURSE SCHEDULE

실제 운영 일정 · 3일 24시간

DAY 1	2026.08.28 (금)	운영체제 개요 · Linux 환경 · Process/Thread · Scheduling	8H
DAY 2	2026.08.31 (월)	Memory · Filesystem · Device I/O · System Call	8H
DAY 3	2026.09.01 (화)	Boot/Service · Network · On-device AI · Troubleshooting	8H

주강사홍길동 · 보조강사성춘향

운영체제 활용 24H는 일정표상 3일 × 8시간으로 편성

COURSE MATERIALS

교재 · 시뮬레이터 · 워크시트 구성

📘 참고 교재

「엣지 AI 임베디드 시스템으로 현실 문제를 해결하는」
운영체제 개념을 온디바이스/임베디드 흐름과 연결할 때 참고

▶️ HTML 시뮬레이터

os_24h_interactive_animations_reduced_advanced.html
12개 챕터 핵심 개념을 직접 조작하고 상태 변화를 관찰

✓ 디지털 워크시트

os_24h_digital_worksheet.html
예측 → 채점 → 잠금 → 검산 → 완료 화면의 학습 기록

⌨️ Linux 실습 환경

Shell 명령어 자체를 깊게 가르치기보다 OS 상태를 관찰하는 도구로 사용

🗣️ PPT 발표자 노트

각 슬라이드별 강사 스크립트, 질문, 시연 순서, 주의 포인트를 포함

📌 학습 증거

워크시트 완료 화면, 실습 기록, 장애분석 근거를 수강생 산출물로 활용

PPT에서 실습 수행 구간을 찾는 표식

▶ ANIMATION 수행

이 표식이 나오면 PPT 설명을 멈추고
PRO HTML 시뮬레이터에서 직접 조작합니다.

✍ WORKSHEET 수행

이 표식이 나오면 디지털 워크시트에서
예측 → 채점 → 검산 → 서술을 완료합니다.

강의 중 표식 사용 규칙

설명 슬라이드에서는 PPT만 사용 → ANIMATION 표식에서 브라우저 전환 → 다시 PPT 복귀 → WORKSHEET 표식에서 기록/채점

`ANIMATION: os_24h_interactive_animations_PRO.html` | `WORKSHEET: os_24h_digital_worksheet.html`

DAY 1

기본 구조와 실행

프로그램이 “실행된다”는 말 속의 OS 동작을 추적합니다.

C01 운영체제 개요
C02 Linux 운영환경
C03 프로세스와 스레드
C04 CPU 스케줄링



Chapter 01. 운영체제 개요

OS 구조 · User/Kernel Space · 자원 중재 · RTOS

학습 목표

1. 운영체제를 하드웨어와 애플리케이션 사이의 자원 관리자 관점으로 설명한다.
2. User Space와 Kernel Space 분리의 이유를 설명한다.
3. 온디바이스 AI 실행 흐름에서 OS의 역할을 찾는다.

시작 질문

User Space와 Kernel Space를 왜 나누는가?



운영체제 개요 · 그림 전체 보기



그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) 운영체제의 위치
2) 보호 경계
3) 자원 관리자
4) 범용 OS vs RTOS

운영체제 개요 · 단계별 그림 해설 ①



1단계 해설

① 운영체제의 위치

Application과 Hardware 사이에서 CPU·Memory·File·Device를 추상화하고 중재

② 보호 경계

User Space에서 보호된 자원 접근 시 System Call을 통해 Kernel로 진입

설명 포인트

- 운영체제의 위치가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 보호 경계가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 "무엇이 어디를 지나가는가"를 말하게 합니다.

운영체제 개요 · 단계별 그림 해설 ②



2단계 해설

③ 자원 관리자

여러 프로세스가 한정된 CPU/RAM/장치를 공유하도록 정책과 순서를 결정

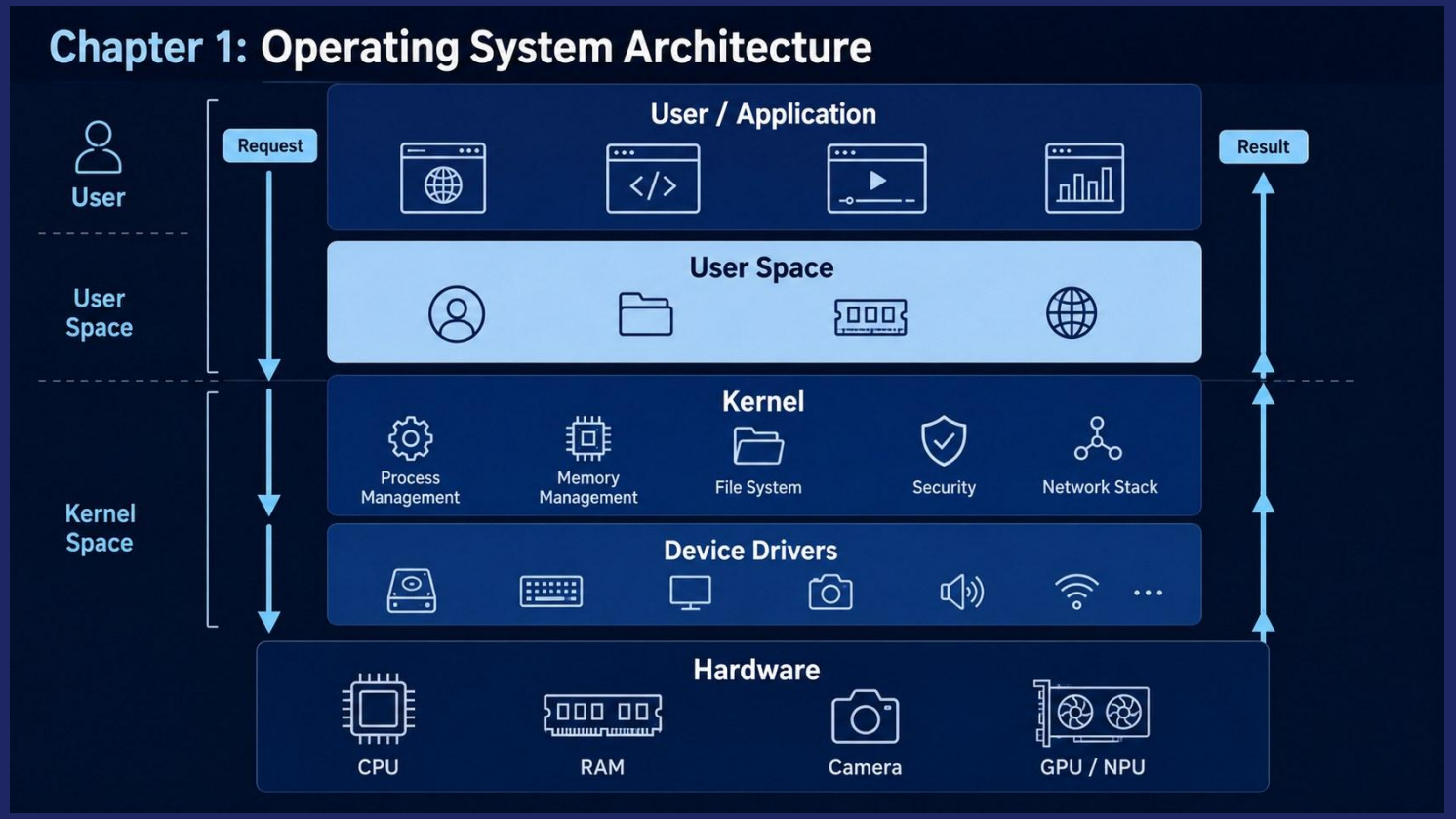
④ 범용 OS vs RTOS

평균 처리량·편의성 중심과 예측 가능한 응답시간 중심의 차이

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- 자원 관리자와 범용 OS vs RTOS는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

운영체제 개요 · 복잡한 그림 세부 해설 1



확대해서 볼 부분

1. 운영체제의 위치

Application과 Hardware 사이에서 CPU·Memory·File·Device를 추상화하고 중재합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

운영체제 개요 · 복잡한 그림 세부 해설 2



확대해서 볼 부분

2. 보호 경계

User Space는 제한된 권한으로 실행되고 보호 자원 접근은 System Call을 통해 Kernel로 넘어갑니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 "무엇이 입력이고 무엇이 상태/자원인가?"를 수강생이 직접 가리키게 합니다.

운영체제 개요 · 복잡한 그림 세부 해설 3



확대해서 볼 부분

3. 자원 관리자
여러 프로세스가 CPU/RAM/장치를 공유하도록 순서와 정책을 결정합니다.

4. 범용 OS vs RTOS
평균 처리량과 편의성 중심의 범용 OS와 예측 가능한 Deadline 중심의 RTOS를 구분합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 "무엇이 입력이고 무엇이 상태/자원인가?"를 수강생이 직접 가리키게 합니다.

운영체제 개요 · 동작 워크플로우

개념을 "정의"가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.



이 워크플로우에서 반드시 관찰할 3가지

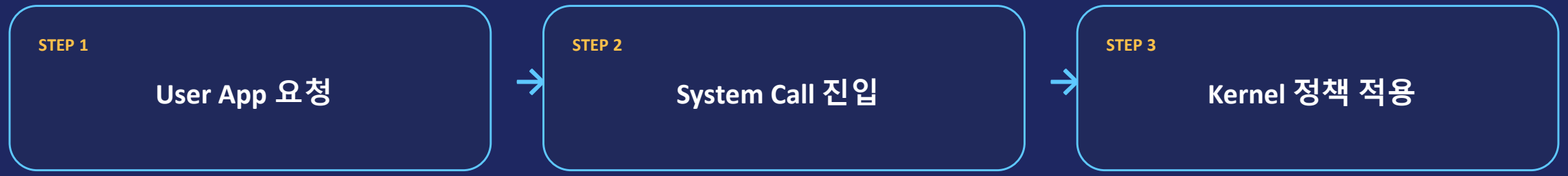
- User↔Kernel 경계를 지나는 순간
- 요청 종류에 따라 CPU/RAM/I/O 비중이 달라짐
- 범용 OS와 RTOS는 응답시간 정책이 다름

강사 멈춤 질문

어디서 User Mode가 Kernel Mode로 바뀌는가?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

운영체제 개요 · 워크플로우 전반부 단계 해설



1	User App 요청	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	System Call 진입	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	Kernel 정책 적용	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

운영체제 개요 · 워크플로우 후반부 단계 해설



4	**Driver 호출**	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	**Hardware 실행**	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	**결과 반환**	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

문제 상황: AI 앱이 카메라와 CPU를 동시에 많이 사용한다면?

SCENARIO

AI 앱이 카메라와 CPU를 동시에 많이 사용한다면?

증상만 보고 바로 조치하지 말고
OS 상태 변화를 먼저 추적한다.

1 **STEP 1**

앱은 카메라 레지스터를 직접 만지지 않고 Driver를 통해 접근한다.

2 **STEP 2**

CPU를 독점하면 Scheduler가 다른 프로세스의 실행 기회를 조정한다.

3 **STEP 3**

메모리·파일·네트워크 접근도 각각 Kernel 서비스를 통해 보호된다.

4 **STEP 4**

결국 OS는 “실행 환경”이 아니라 “자원 충돌을 관리하는 정책 계층”이다.

▶ ANIMATION 준비 구간

운영체제 개요 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
정상 흐름	파일 읽기 실행	요청 토큰이 App→Kernel→Driver→Storage로 이동	Device I/O 상승, CPU는 중간 수준	어디서 User Mode가 Kernel Mode로 바뀌는가?
자원 경쟁	AI 추론 실행	CPU/RAM 게이지가 동시에 증가하고 OS가 자원 중재	CPU·RAM 사용률, 응답 지연	OS가 없다면 두 앱의 충돌을 누가 조정할까?
실시간성 비교	RTOS 지터 조절	범용 OS 응답 막대는 흔들리고 RTOS는 일정하게 유지	지터, Deadline miss	평균 속도와 예측 가능한 응답 중 무엇이 더 중요한가?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 01 · 운영체제 개요

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=1`

수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

▶ 지금 ANIMATION 수행

인터랙티브 시뮬레이션 · OS 요청·자원 중재 시뮬레이터

HTML SIMULATOR

OS 요청·자원 중재 시뮬레이터



1. 시뮬레이터에서 “파일 읽기”를 선택하고 요청 흐름을 관찰
2. “카메라 프레임”으로 바꾸고 Device I/O 사용량 변화를 비교
3. “AI 추론”을 선택해 CPU/RAM 부하가 함께 증가하는지 확인
4. RTOS 비교 화면에서 지터를 올려 Deadline 안정성 차이를 토론

시연 전에 반드시 질문

**“버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
.”**

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 01 · 운영체제 개요

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. 파일 읽기와 AI 추론의 자원 게이지 비교
2. RTOS Deadline 시나리오에서 지터 관찰
3. User↔Kernel 경계를 한 문장으로 설명

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 01 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. 워크시트 C01의 예측 문제를 먼저 작성
2. 채점 후 오답을 수정하지 말고 시뮬레이터로 원인을 확인
3. 마지막 서술: “OS는 자원 관리자다”를 한 문장으로 작성

체크 질문

- User Space와 Kernel Space를 왜 나누는가?
- Driver는 어느 계층에서 어떤 문제를 해결하는가?
- 온디바이스 AI에서 OS가 없다면 어떤 문제가 생길까?

파일: `os_24h_digital_worksheet.html`

Chapter 02. Linux 운영환경 이해

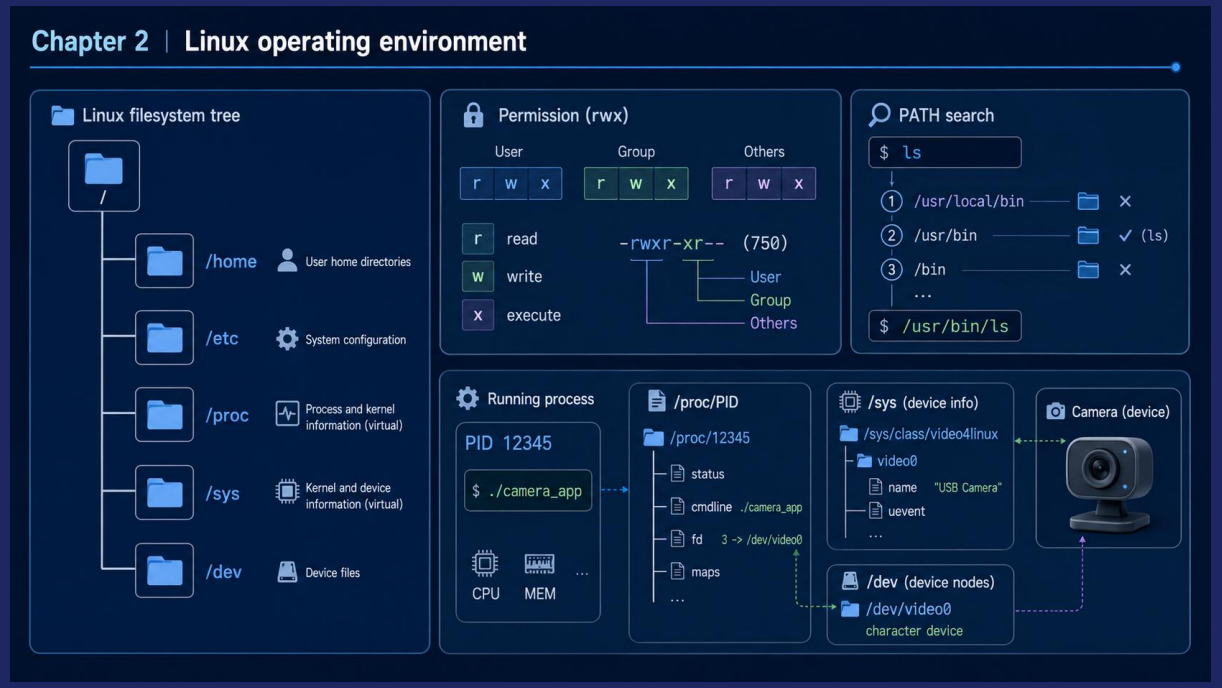
디렉터리 · 권한 · PATH · /proc · /sys · /dev

학습 목표

1. Linux 디렉터리 구조에서 시스템 정보의 위치를 찾는다.
2. rwx 권한과 PATH 검색 흐름을 설명한다.
3. /proc·/sys·/dev를 이용해 프로세스와 장치 상태를 탐색한다.

시작 질문

755 에서 7,5,5 는 각각 무엇을 의미하는가?



Linux 운영환경 이해 · 그림 전체 보기

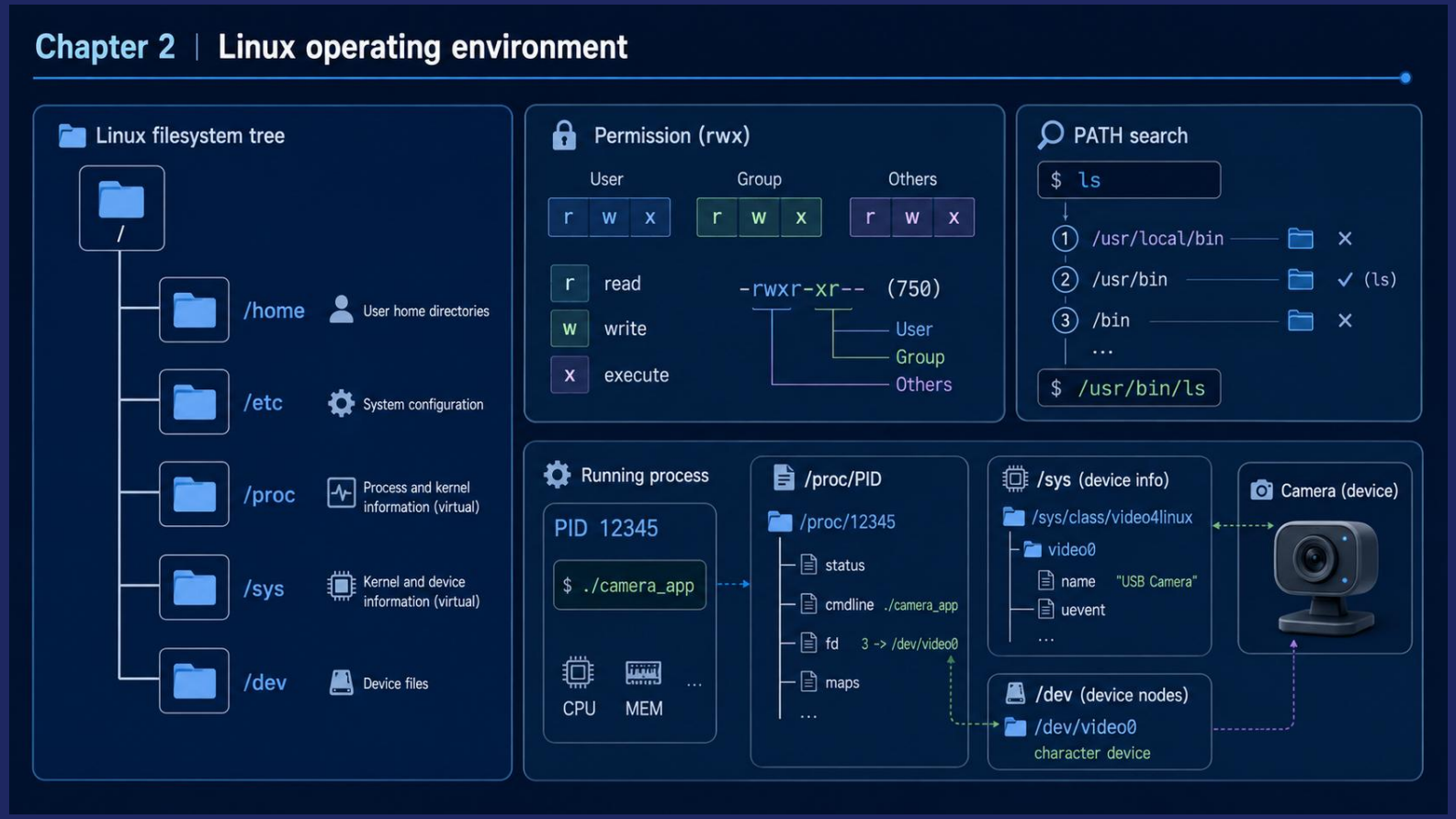


그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

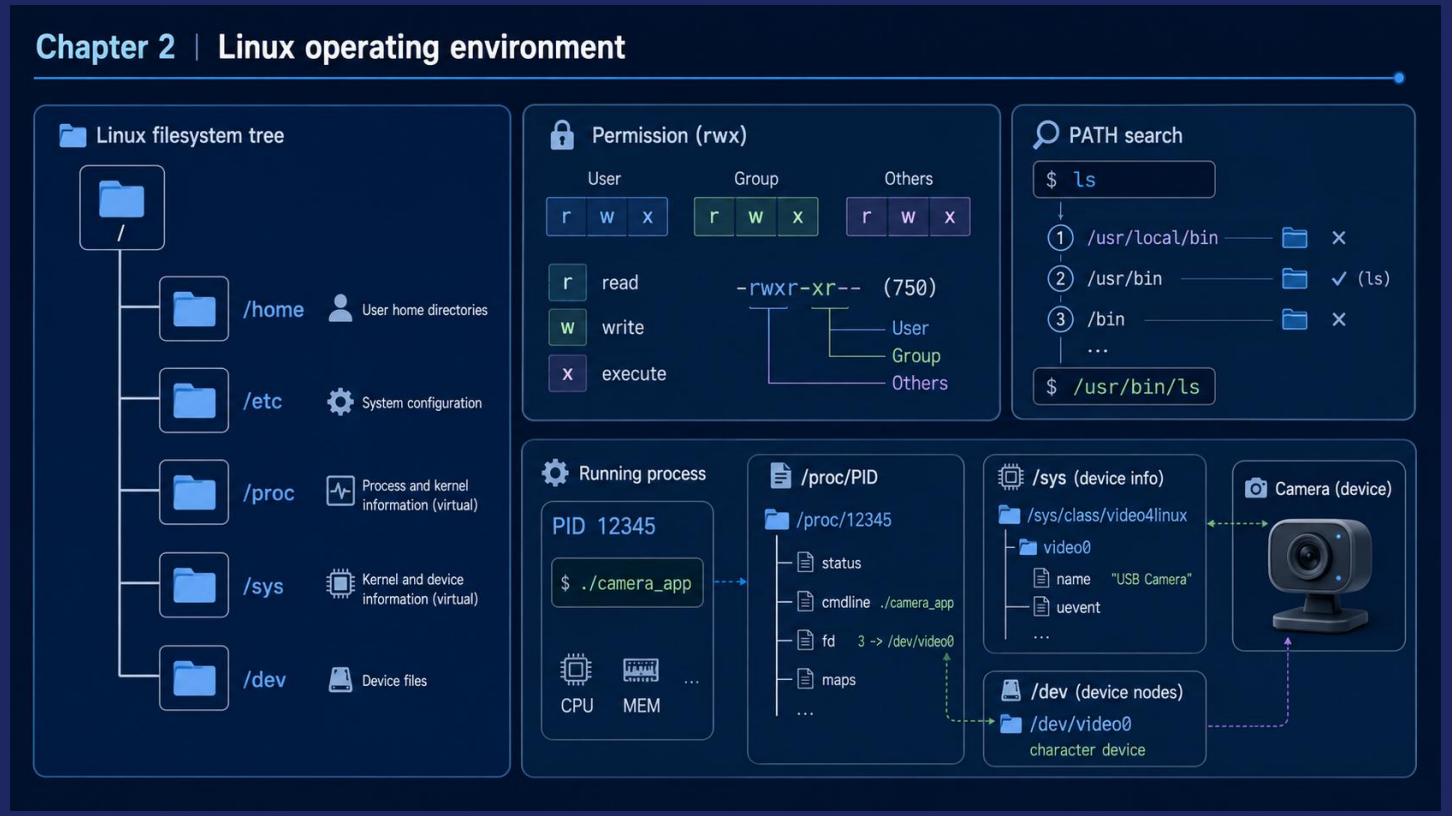
설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) 디렉터리 의미
2) 권한 모델
3) PATH
4) 가상 파일시스템

Linux 운영환경 이해 · 단계별 그림 해설 ①



1단계 해설

① 디렉터리 의미

/home 사용자 데이터, /etc 설정, /proc 프로세스/커널 정보, /sys 장치/커널 객체, /dev 장치 노드

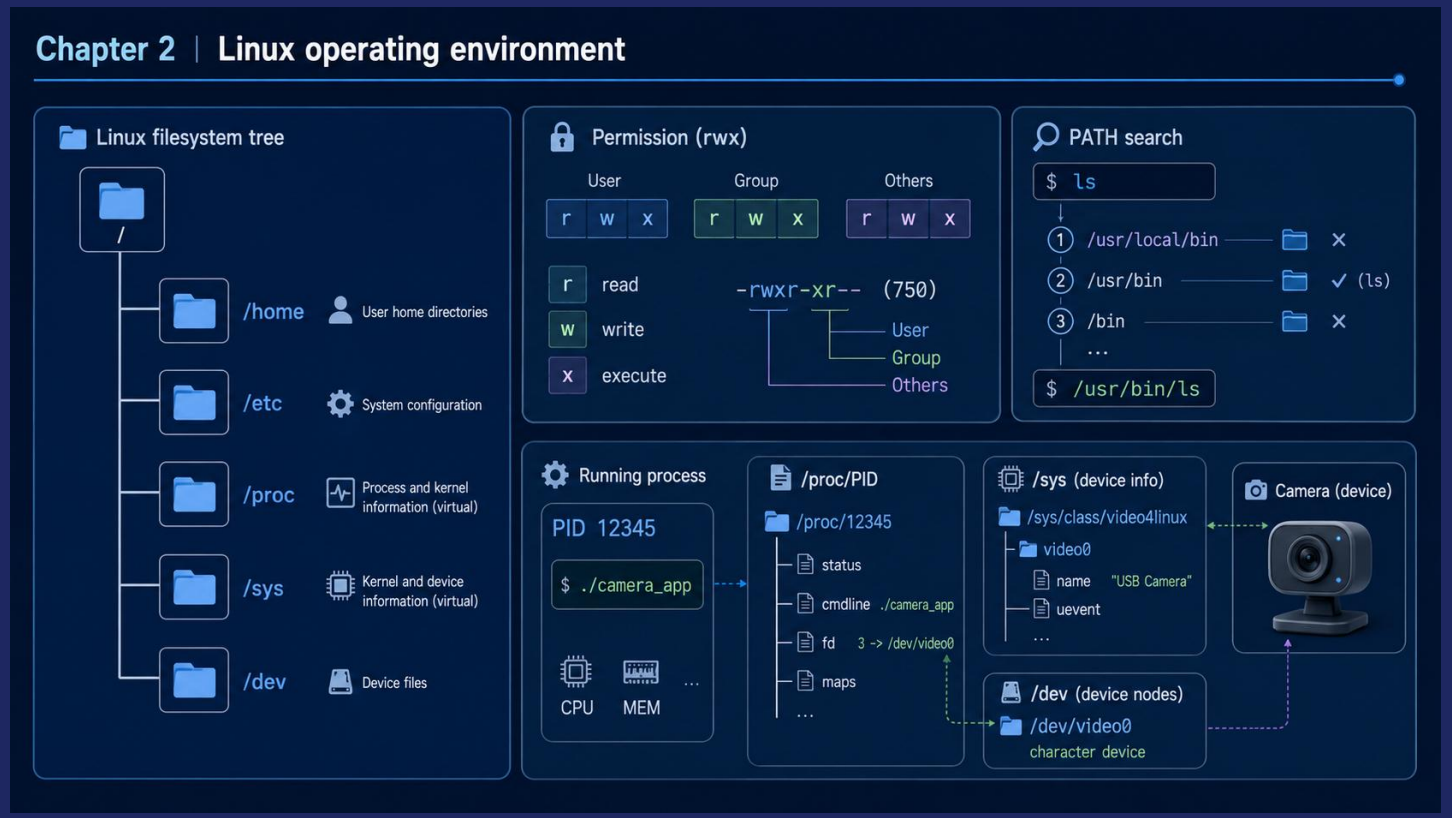
② 권한 모델

Owner/Group/Others에 대해 read/write/execute를 독립적으로 부여

설명 포인트

- 디렉터리 의미가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 권한 모델가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

Linux 운영환경 이해 · 단계별 그림 해설 ②



2단계 해설

③ PATH

명령 이름만 입력했을 때 실행 파일을 찾는 검색 경로 목록

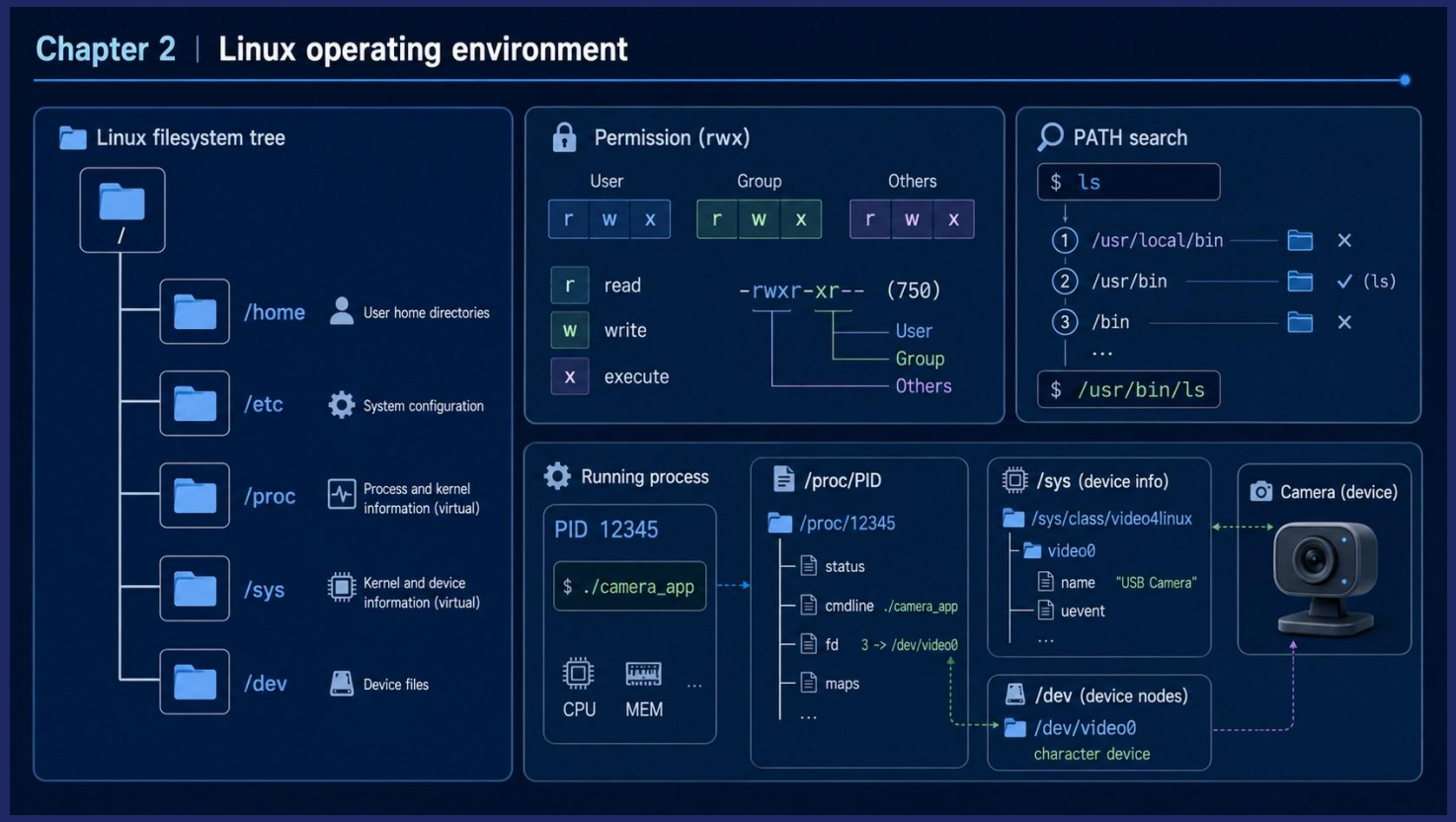
④ 가상 파일시스템

/proc와 /sys는 디스크 파일이 아니라 Kernel 상태를 파일 형태로 노출

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- PATH와 가상 파일시스템는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

Linux 운영환경 · 복잡한 그림 세부 해설 1



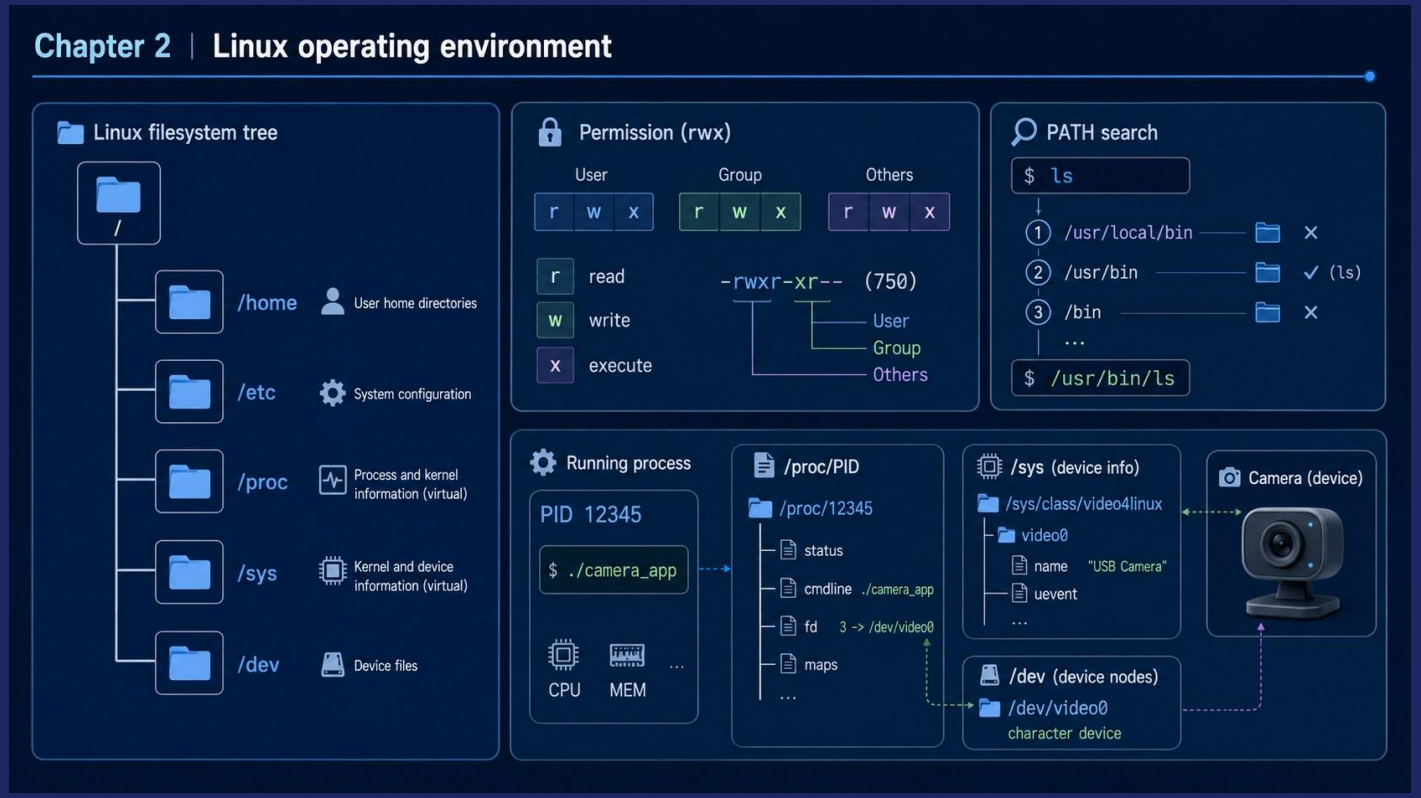
확대해서 볼 부분

1. 디렉터리 구조

/home·/etc·/proc·/sys·/dev의 역할을 위치가 아니라 정보 종류로 구분합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

Linux 운영환경 · 복잡한 그림 세부 해설 2



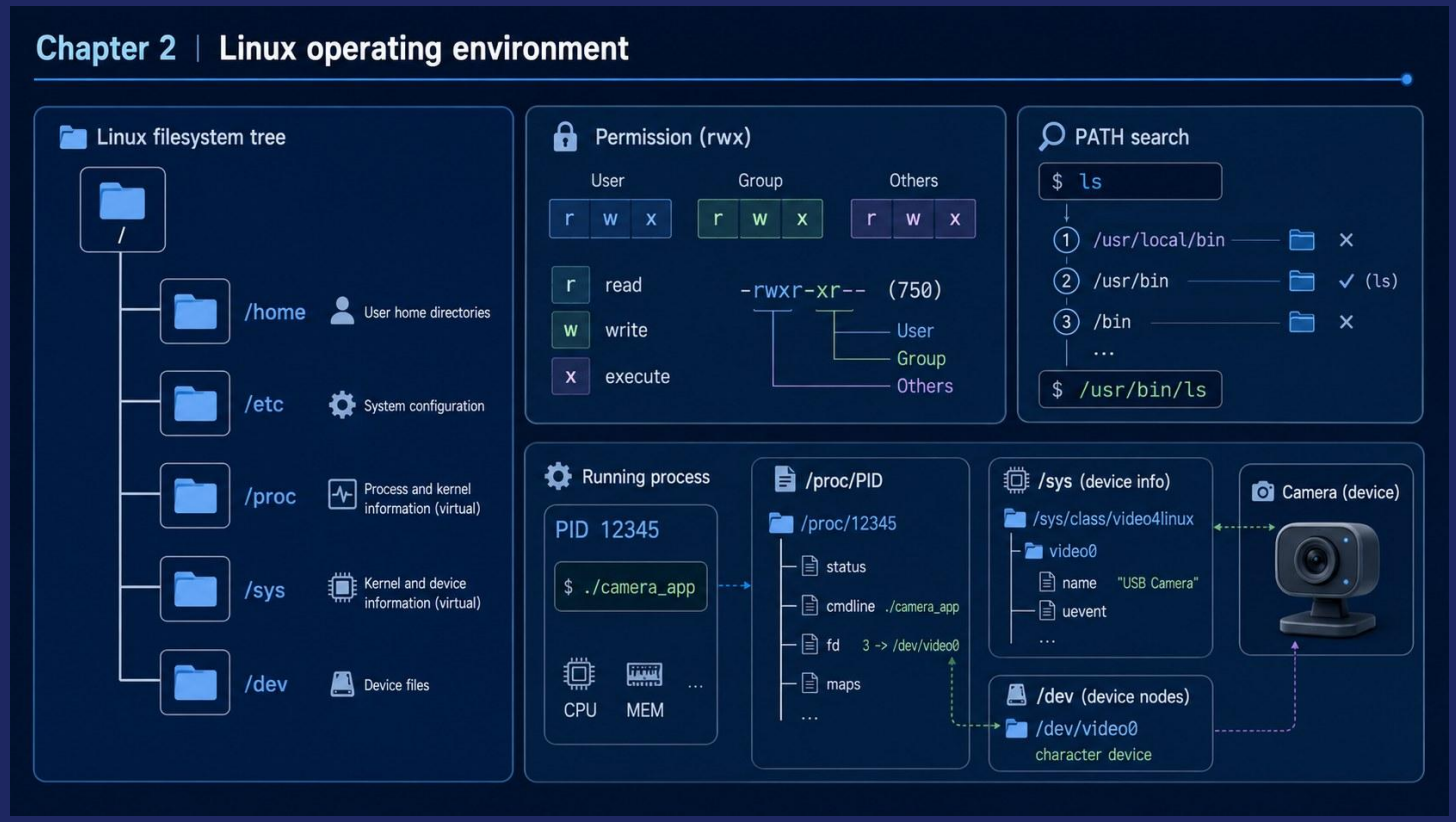
확대해서 볼 부분

2. 권한 rwx

Owner/Group/Others의 read·write·execute를 숫자 권한과 연결합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

Linux 운영환경 · 복잡한 그림 세부 해설 3



확대해서 볼 부분

3. PATH 검색

명령 이름을 입력하면 PATH 목록을 순차 검색해 실행 파일을 찾습니다.

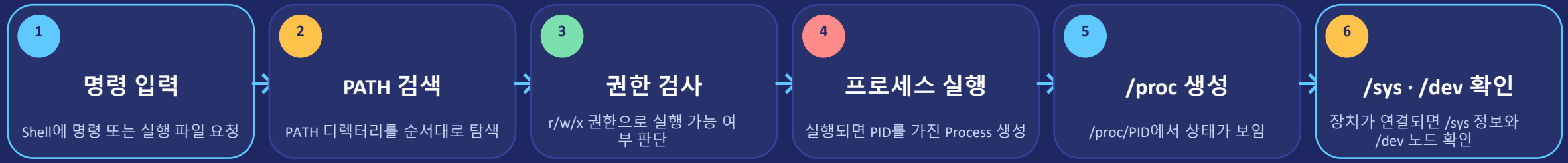
4. /proc·/sys·/dev

프로세스·커널·장치 상태가 파일/노드 형태로 노출됩니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

Linux 운영환경 이해 · 동작 워크플로우

개념을 “정의”가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.



이 워크플로우에서 반드시 관찰할 3가지

- 경로·권한·프로세스·장치가 서로 다른 문제임
- /proc와 /sys는 Kernel 상태를 파일처럼 보여줌
- /dev 노드의 존재는 장치 접근 가능성의 중요한 단서

강사 멈춤 질문

명령 이름만 쳤는데 실행되는 이유는?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

Linux 운영환경 · 워크플로우 전반부 단계 해설



1	**명령 입력**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**PATH 검색**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**권한 확인**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

Linux 운영환경 · 워크플로우 후반부 단계 해설



4	**프로세스 생성**	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	**/proc 생성**	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	**/sys·/dev 장치 확인**	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

문제 상황: camera_app은 있는데 실행도, 카메라 접근도 안 된다

SCENARIO

camera_app은 있는데 실행도, 카메라 접근도 안 된다

증상만 보고 바로 조치하지 말고 OS 상태 변화를 먼저 추적한다.

1 STEP 1

먼저 현재 위치와 파일 존재 여부를 확인한다.

2 STEP 2

실행 권한(x)이 없으면 Permission denied가 발생한다.

3 STEP 3

프로세스가 실행되면 /proc/PID가 생기고 fd에서 장치 연결을 볼 수 있다.

4 STEP 4

카메라가 연결되면 /sys와 /dev에 관련 노드가 나타나는지 확인한다.

▶ ANIMATION 준비 구간

Linux 운영환경 이해 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
경로 탐색	PATH 검색 실행	후보 디렉터리가 차례로 강조되고 실행 파일에서 멈춤	검색 순서, 발견 위치	명령 이름만 쳤는데 실행되는 이유는?
권한 오류	x 권한 OFF 후 접근	파일까지 도달하지만 실행 단계에서 차단	권한 숫자, Permission denied	chmod 777이 왜 좋은 기본 해결책이 아닌가?
장치 이벤트	Camera 연결/해제	/sys와 /dev 노드가 생성·제거됨	/sys/class, /dev/video0	프로세스가 살아 있는데 카메라가 안 될 때 무엇을 먼저 볼까?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 02 · Linux 운영환경

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=2`

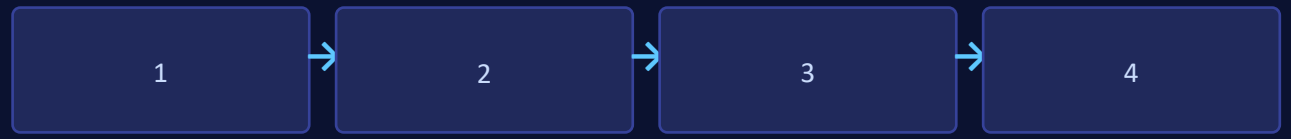
수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

▶ 지금 ANIMATION 수행

인터랙티브 시뮬레이션 · Linux 경로·권한 + /proc·/sys 관찰

HTML SIMULATOR

Linux 경로·권한 + /proc·/sys 관찰



1. 디렉터리 탐색 화면에서 / → /home → /proc → /sys → /dev 이동
2. r/w/x 조합을 바꾸고 접근 성공/실패 비교
3. Process 생성 버튼으로 /proc/PID 생성 확인
4. Camera 연결/해제로 /sys와 /dev 노드 변화 확인

시연 전에 반드시 질문

"버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
."

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 02 · Linux 운영환경

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. PATH 미등록과 실행권한 없음 시나리오 비교
2. Camera Hot Plug로 /sys·/dev 변화 관찰
3. 755 권한을 말로 해석

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 02 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. 755·644·700 권한을 말로 해석
2. PATH가 명령을 찾는 순서를 그림으로 설명
3. /proc·/sys·/dev 중 어떤 정보를 어디서 볼지 사례별 선택

체크 질문

- 755에서 7,5,5는 각각 무엇을 의미하는가?
- /proc와 /sys는 왜 “가상” 파일시스템인가?
- /dev/video0가 없다는 것은 어떤 단서를 주는가?

파일: `os_24h_digital_worksheet.html`

Chapter 03. 프로세스와 스레드

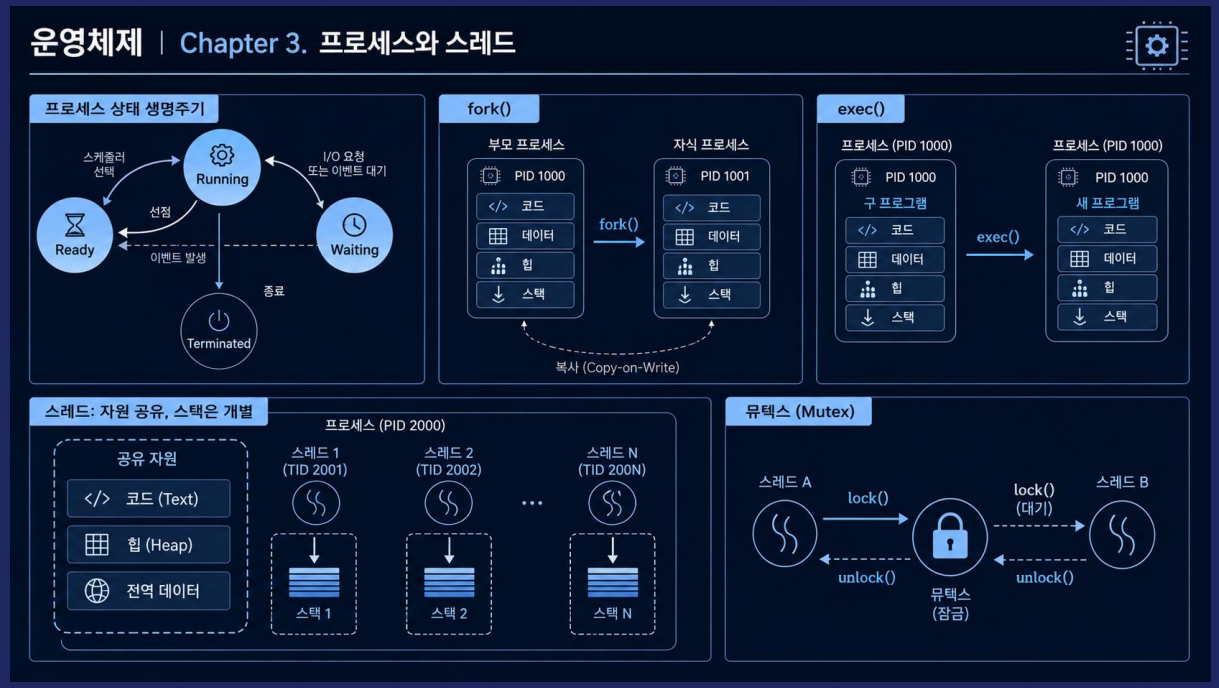
상태 전이 · fork/exec · Thread 공유 · Race/Mutex

학습 목표

1. 프로세스의 Ready/Running/Waiting 상태 전이를 설명한다.
2. fork와 exec의 차이를 설명한다.
3. Thread 공유 자원과 Race Condition을 이해하고 Mutex 필요성을 판단한다.

시작 질문

Waiting과 Ready의 차이는 무엇인가?



프로세스와 스레드 · 그림 전체 보기

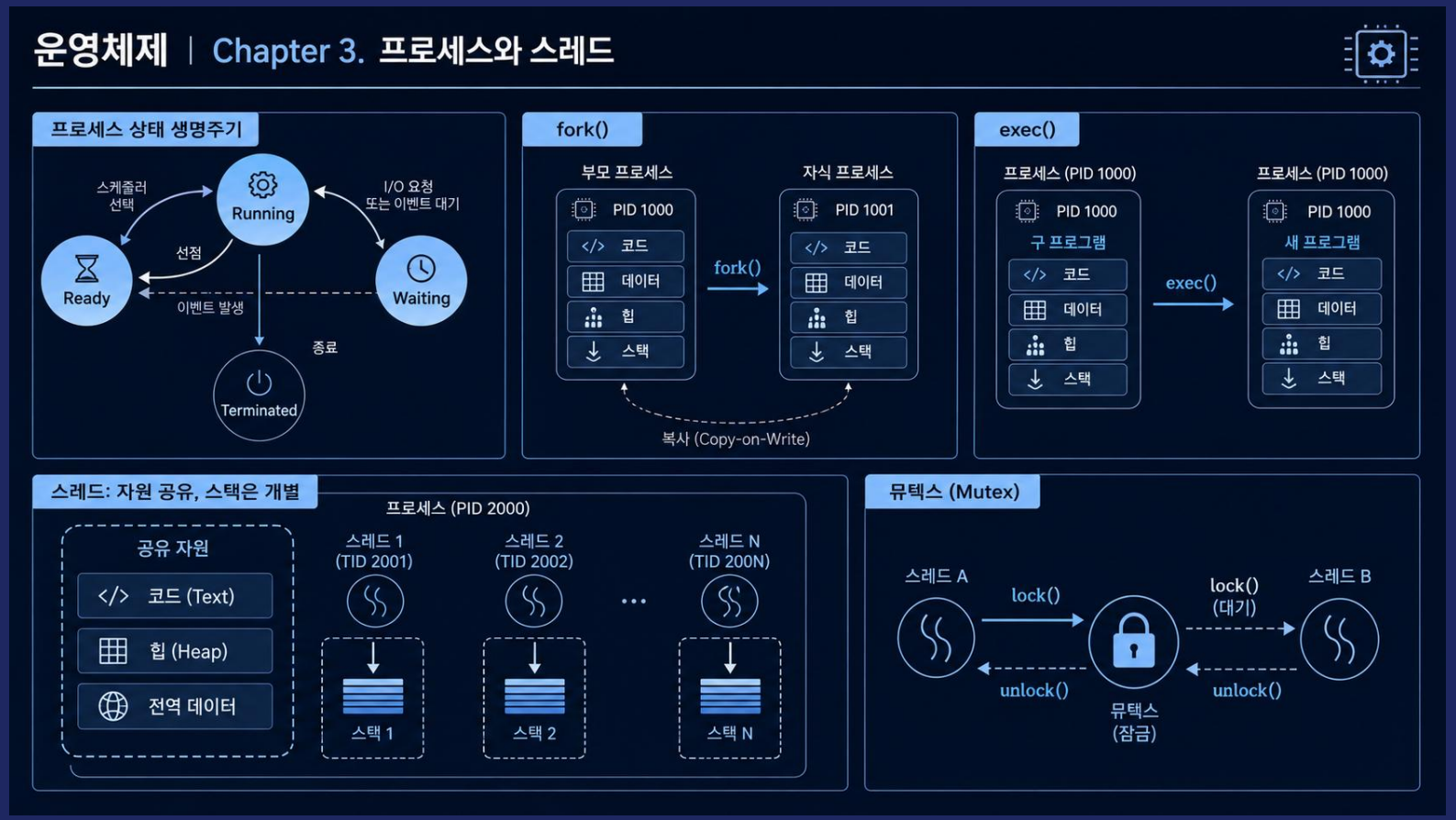


그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

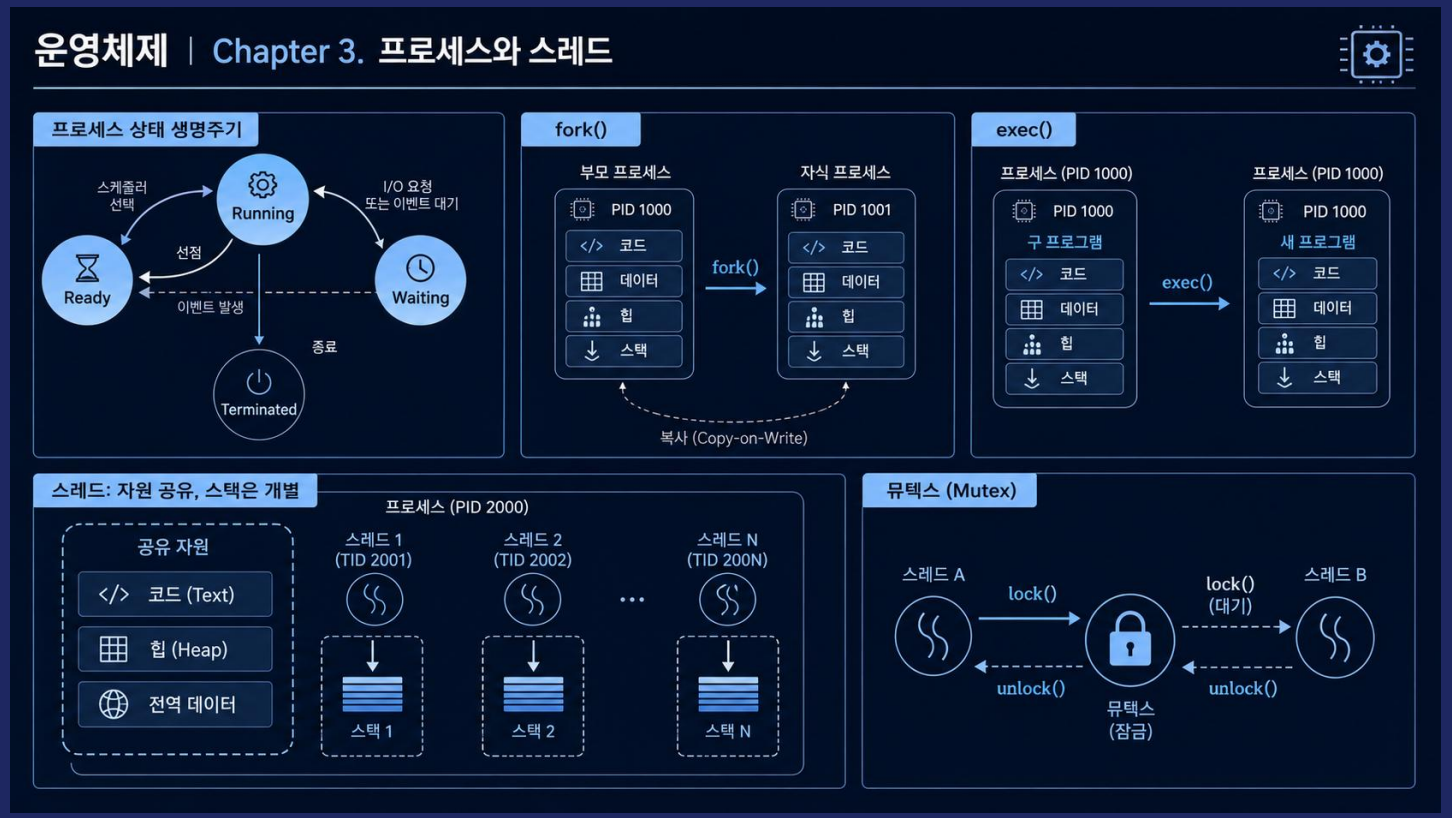
설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) 프로세스
2) 상태 전이
3) fork / exec
4) Thread

프로세스와 스레드 · 단계별 그림 해설 ①



1단계 해설

① 프로세스

실행 중인 프로그램의 인스턴스이며 PID와 독립 주소 공간을 가짐

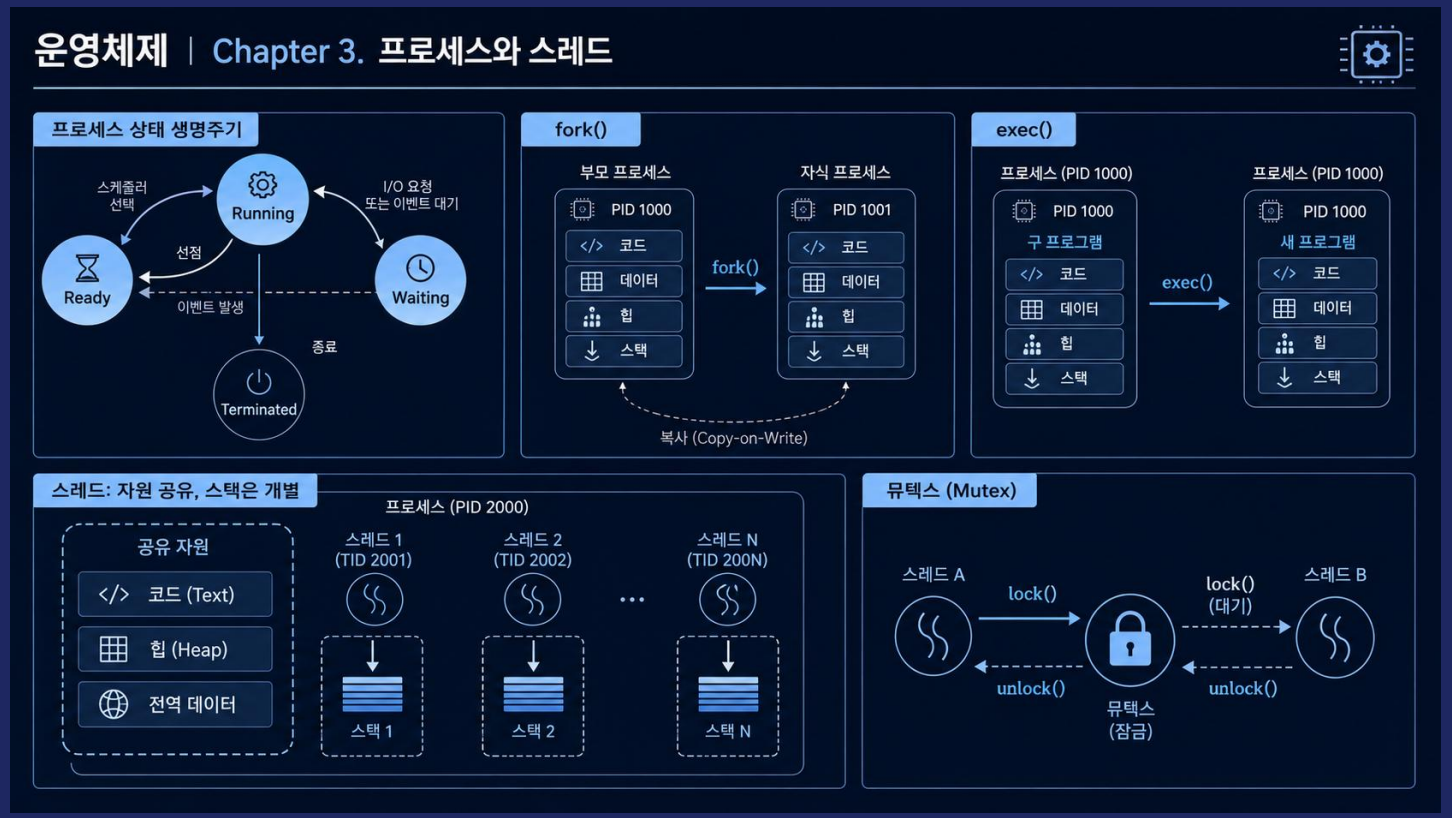
② 상태 전이

Ready는 CPU 대기, Running은 실행, Waiting은 I/O·이벤트 대기

설명 포인트

- 프로세스가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 상태 전이가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

프로세스와 스레드 · 단계별 그림 해설 ②



2단계 해설

③ fork / exec

fork는 새 프로세스 생성, exec는 같은 PID에서 실행 이미지를 교체

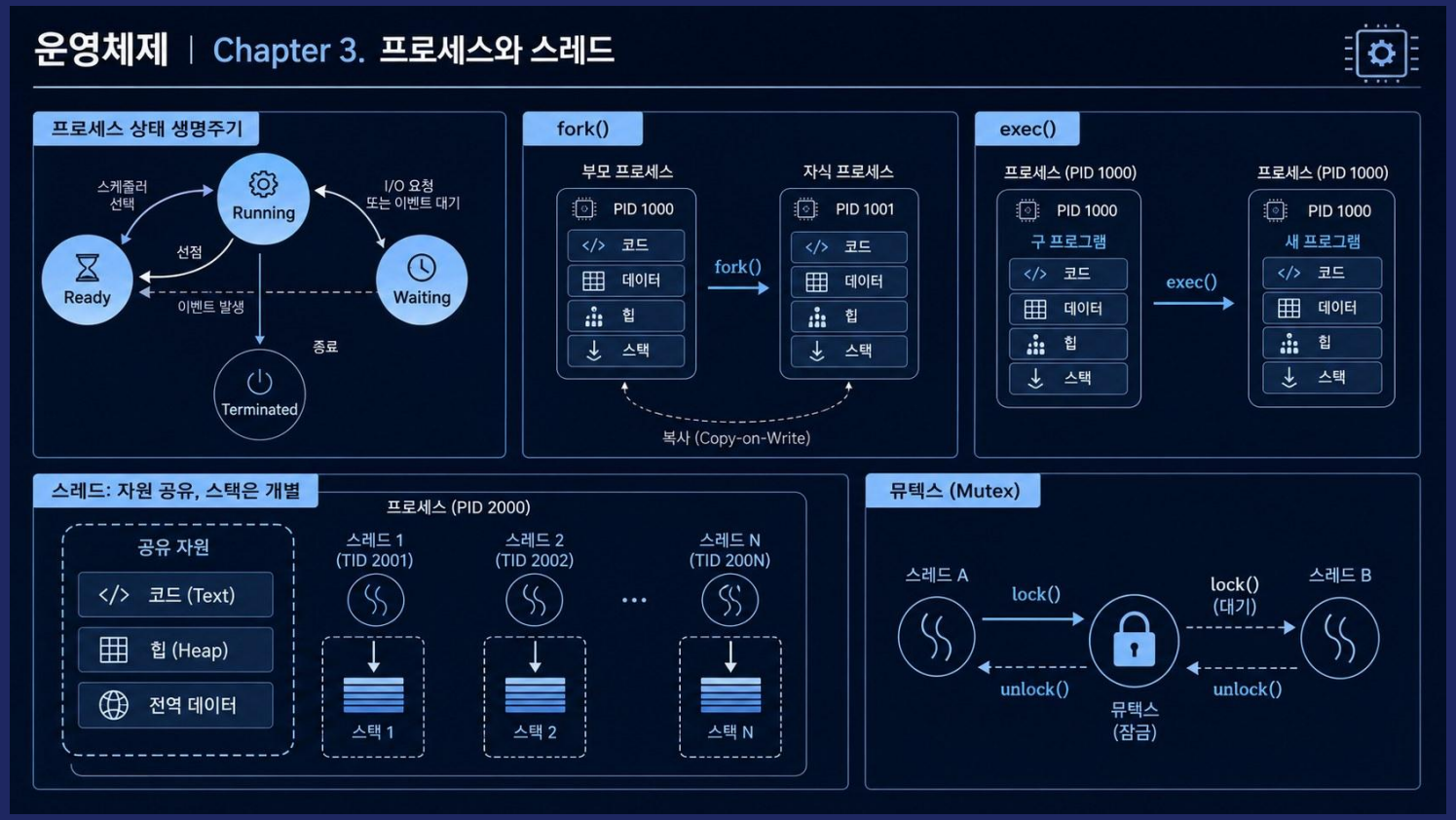
④ Thread

Code/Data/Heap은 공유하고 각 Thread가 Stack을 별도로 가짐

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- fork / exec와 Thread는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

프로세스와 스레드 · 복잡한 그림 세부 해설 1



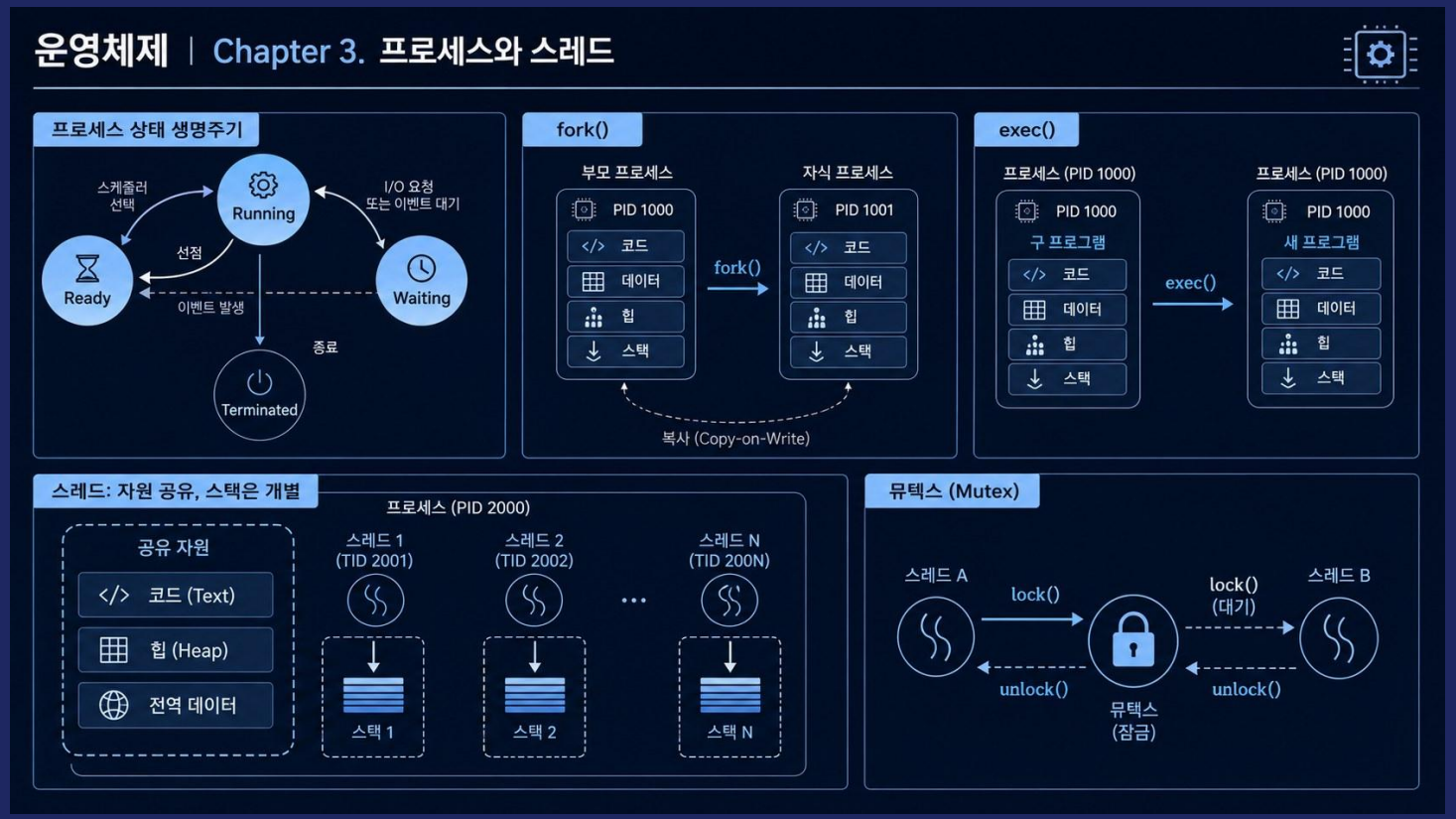
확대해서 볼 부분

1. 프로세스 상태

New·Ready·Running·Waiting·Terminated가 사건에 따라 이동합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

프로세스와 스레드 · 복잡한 그림 세부 해설 2



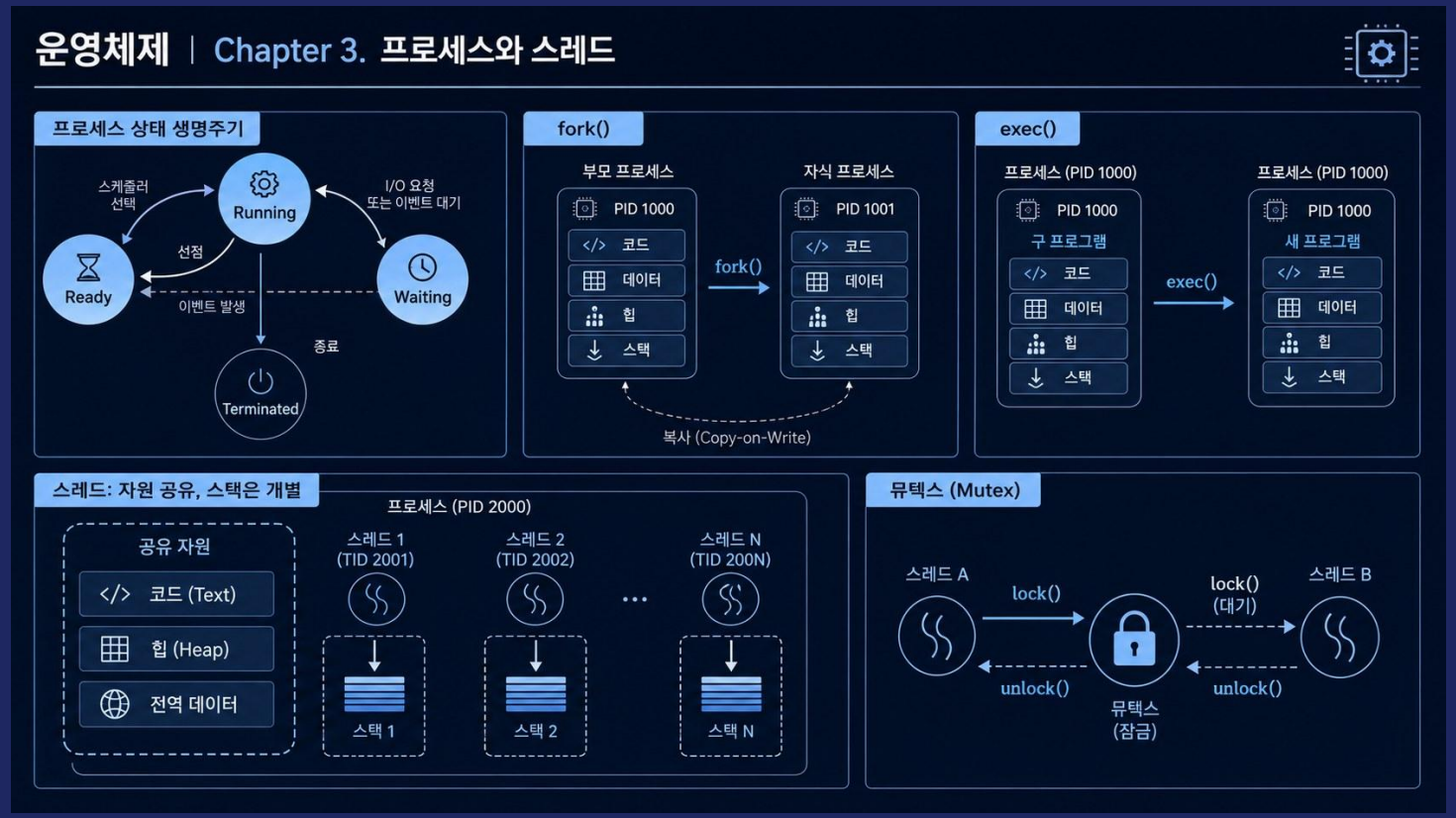
확대해서 볼 부분

2. fork / exec

fork는 프로세스를 만들고 exec는 현재 실행 이미지를 교체합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

프로세스와 스레드 · 복잡한 그림 세부 해설 3



확대해서 볼 부분

3. Thread 공유

Code/Data/Heap은 공유하고 Thread별 Stack은 분리됩니다.

4. Race / Mutex

공유 데이터의 비원자적 연산은 Race를 만들며 Mutex로 임계 영역을 보호합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

프로세스와 스레드 · 동작 워크플로우

개념을 “정의”가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.

1. **New** — 프로그램 실행으로 Process 생성
2. **Ready** — CPU를 기다리는 Ready Queue 진입
3. **Running** — Scheduler가 선택하면 Running
4. **Waiting** — I/O 요청 시 Waiting으로 이동
5. **Ready 복귀** — 이벤트 완료 후 다시 Ready
6. **Terminate / fork / exec** — 종료하거나 fork/exec로 실행 구조 변화

이 워크플로우에서 반드시 관찰할 3가지

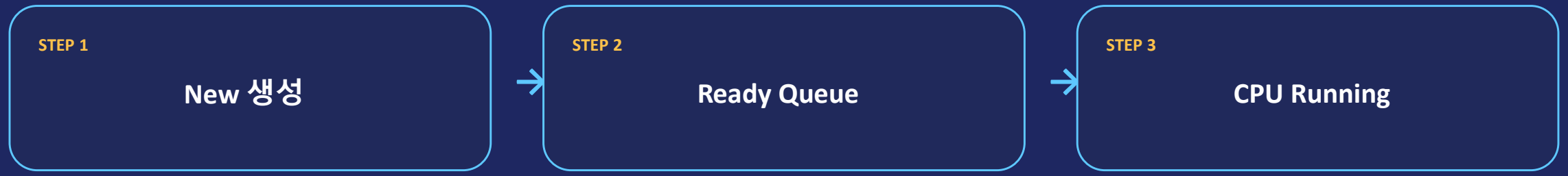
- Ready와 Waiting은 “기다린다”는 점은 같지만 이유가 다름
- fork는 Process를 만들고 exec는 실행 이미지를 교체
- Thread는 공유 자원과 개별 Stack을 동시에 가짐

강사 멈춤 질문

Waiting과 Ready를 구분하는 핵심 조건은?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

프로세스와 스레드 · 워크플로우 전반부 단계 해설



1	New 생성	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	Ready Queue	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	CPU Running	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

프로세스와 스레드 · 워크플로우 후반부 단계 해설



4	I/O Waiting	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	Ready 복귀	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	종료/회수	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

문제 상황: 두 스레드가 counter++ 했는데 결과가 기대값보다 작다

SCENARIO

두 스레드가 counter++ 했는데 결과가 기대값보다 작다

증상만 보고 바로 조치하지 말고 OS 상태 변화를 먼저 추적한다.

1 **STEP 1**

counter++는 읽기-증가-쓰기의 여러 단계로 나뉜다.

2 **STEP 2**

두 Thread가 같은 값을 읽으면 한 번의 증가가 사라질 수 있다.

3 **STEP 3**

Mutex는 임계 구역에 한 Thread만 들어가도록 직렬화한다.

4 **STEP 4**

동기화는 정확성을 주지만 대기와 오버헤드를 만들 수 있다.

▶ ANIMATION 준비 구간

프로세스와 스레드 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
상태 전이	상태 진행 버튼	Process 카드가 Ready→Running→Waiting→Ready로 이동	현재 상태, Queue 위치	Waiting과 Ready를 구분하는 핵심 조건은?
fork / exec	fork 후 exec 실행	자식 Process가 생기고 exec에서 실행 이미지가 교체	PID 수, 실행 이미지	fork와 exec를 같이 쓰는 이유는?
Race / Mutex	Mutex OFF/ON 비교	두 Thread가 동시에 접근하거나 한쪽이 대기	기대 counter vs 실제 counter	정확성을 얻는 대신 Mutex가 만드는 비용은?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 03 · 프로세스와 스레드

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=3`

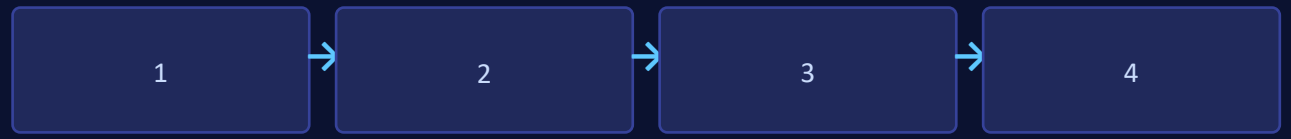
수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

▶ 지금 ANIMATION 수행

인터랙티브 시뮬레이션 · 프로세스 생명주기 + Thread Race 시뮬레이터

HTML SIMULATOR

프로세스 생명주기 + Thread Race 시뮬레이터



1. 상태 진행 버튼으로 Ready→Running→Waiting→Ready 흐름 확인
2. fork()로 자식 생성 후 exec()로 이미지 교체
3. Mutex OFF로 반복 실행하여 실제값과 기대값 비교
4. Mutex ON으로 다시 실행하여 정확성 회복 확인

시연 전에 반드시 질문

"버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
."

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 03 · 프로세스와 스레드

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. I/O 요청 상태 전이 추적
2. fork→exec 흐름 실행
3. Race와 Mutex 결과 Counter 비교

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 03 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. 상태 전이를 사건과 연결해 표 완성
2. Process vs Thread를 메모리 관점에서 비교
3. Race Condition 사례를 설명하고 Mutex 적용 위치 표시

체크 질문

- Waiting과 Ready의 차이는 무엇인가?
- fork 후 부모와 자식은 무엇이 같은가/다른가?
- Mutex를 아무 곳에나 크게 잡으면 어떤 문제가 생길까?

파일: `os_24h_digital_worksheet.html`

Chapter 04. CPU와 스케줄링

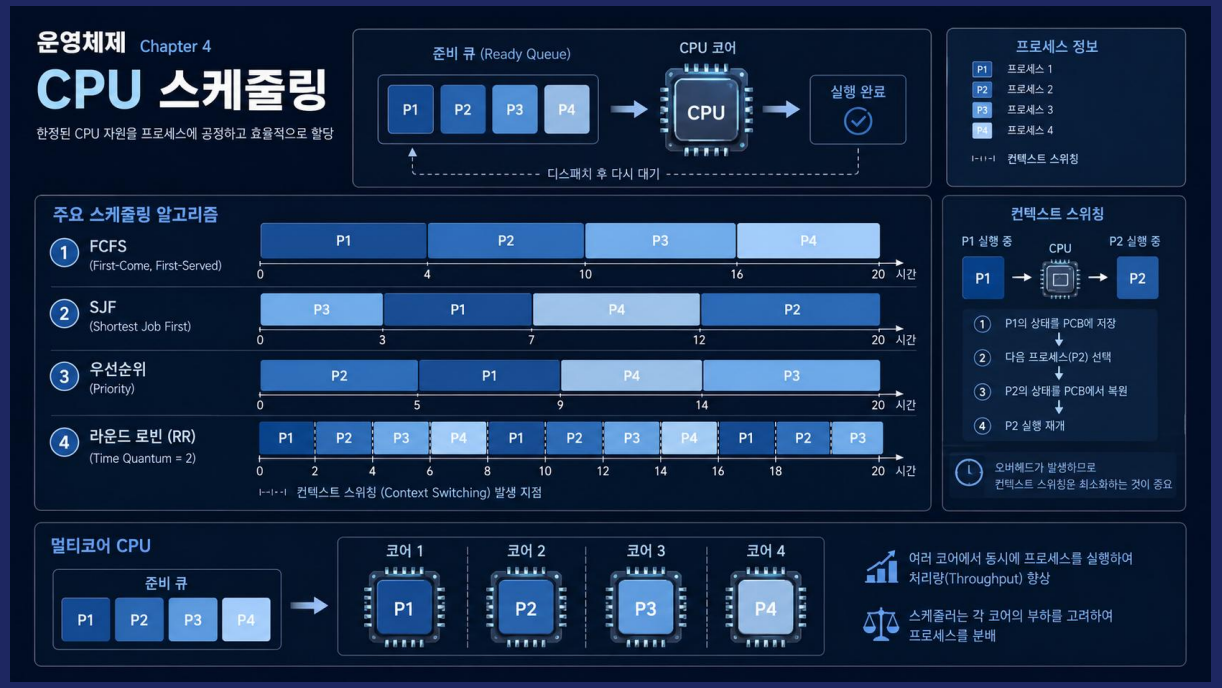
FCFS · SJF · Priority · Round Robin · Context Switch · Multi-Core

학습 목표

1. 대표 스케줄링 알고리즘의 선택 기준과 결과를 비교한다.
2. Time Quantum과 Context Switch의 관계를 설명한다.
3. 멀티코어와 CPU Affinity가 실행에 미치는 영향을 설명한다.

시작 질문

SJF가 평균 대기시간에 유리한 이유는?



CPU와 스케줄링 · 그림 전체 보기

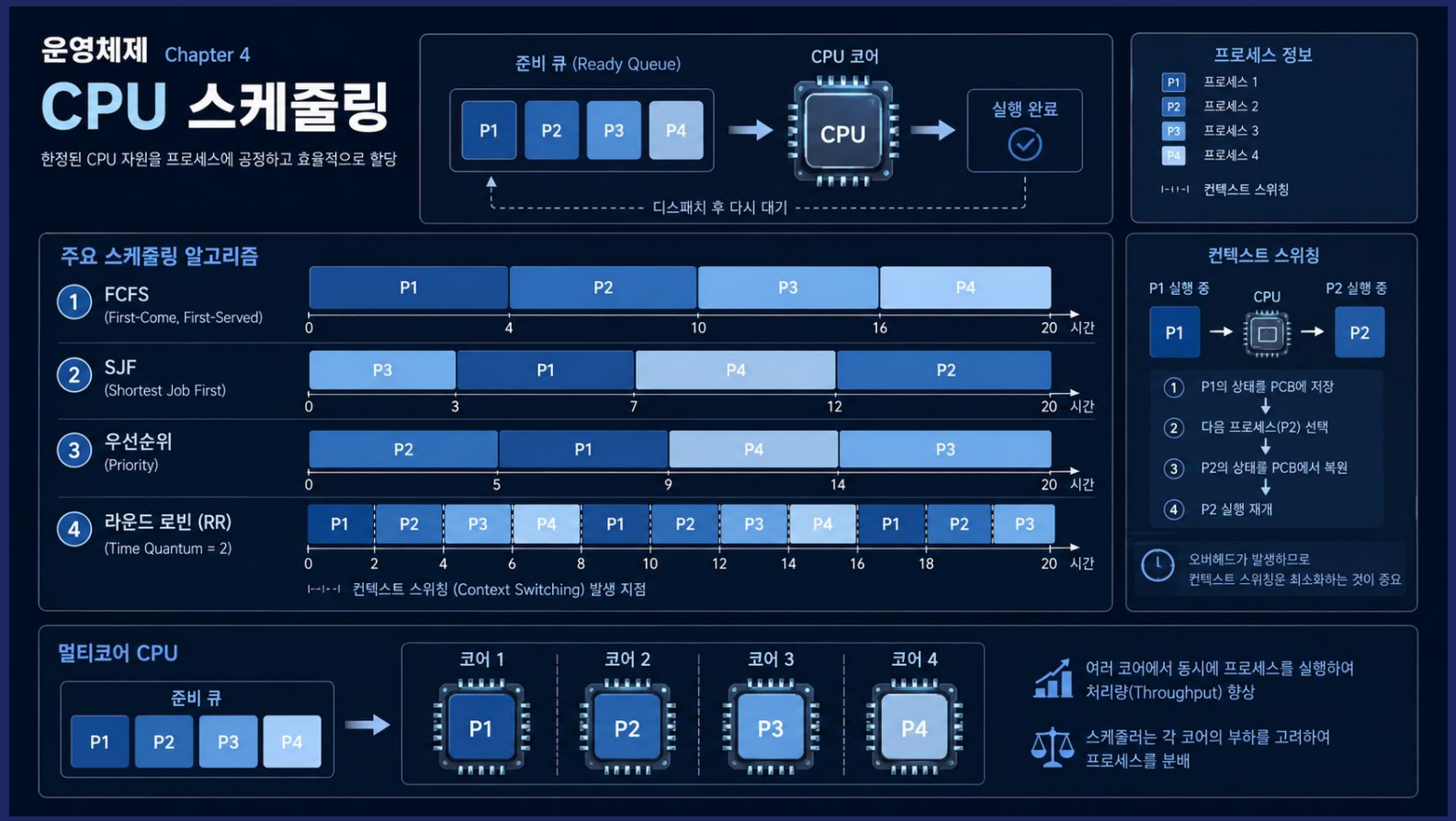


그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

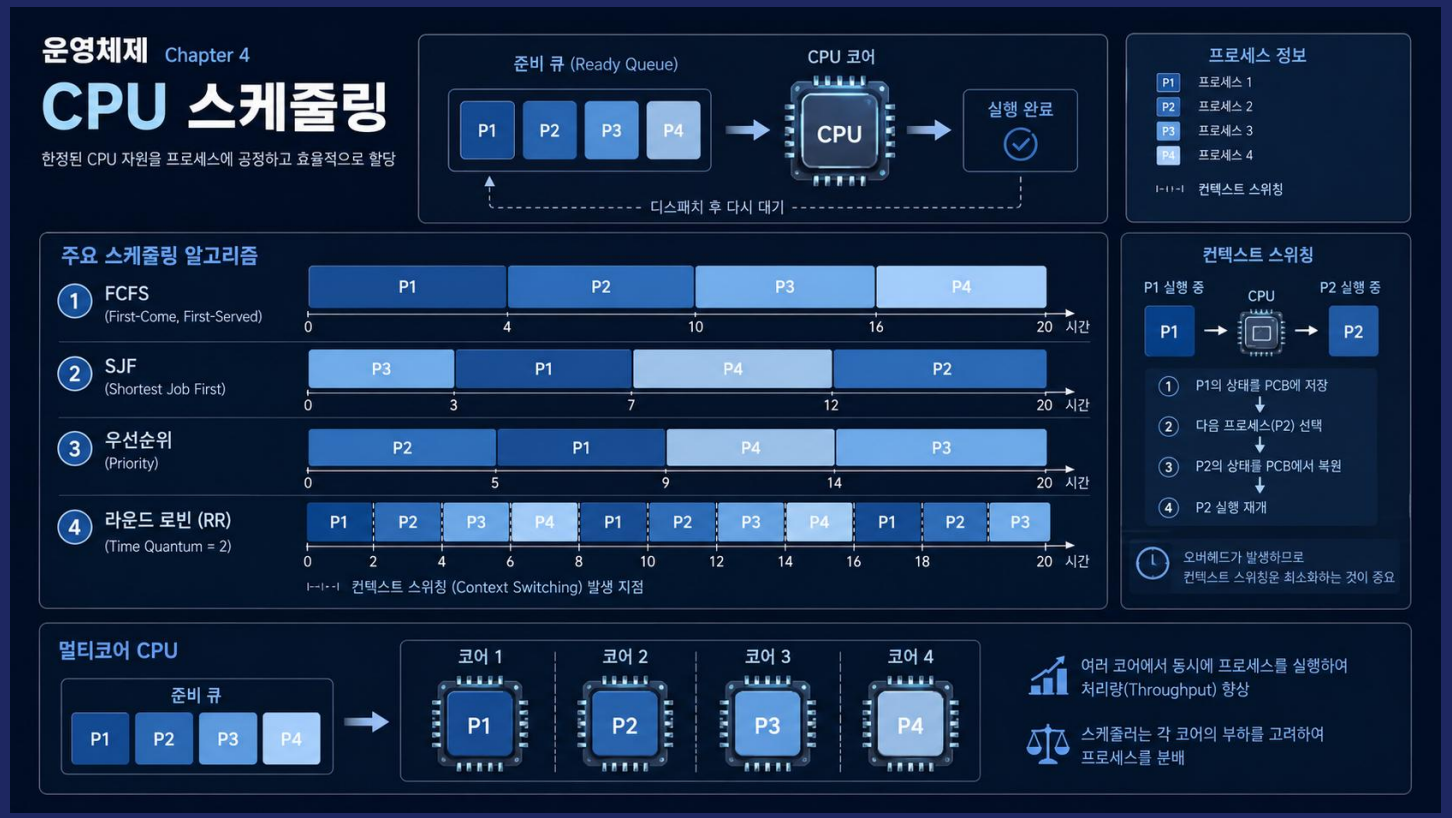
설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) FCFS
2) SJF
3) Priority
4) Round Robin

CPU와 스케줄링 · 단계별 그림 해설 ①



1단계 해설

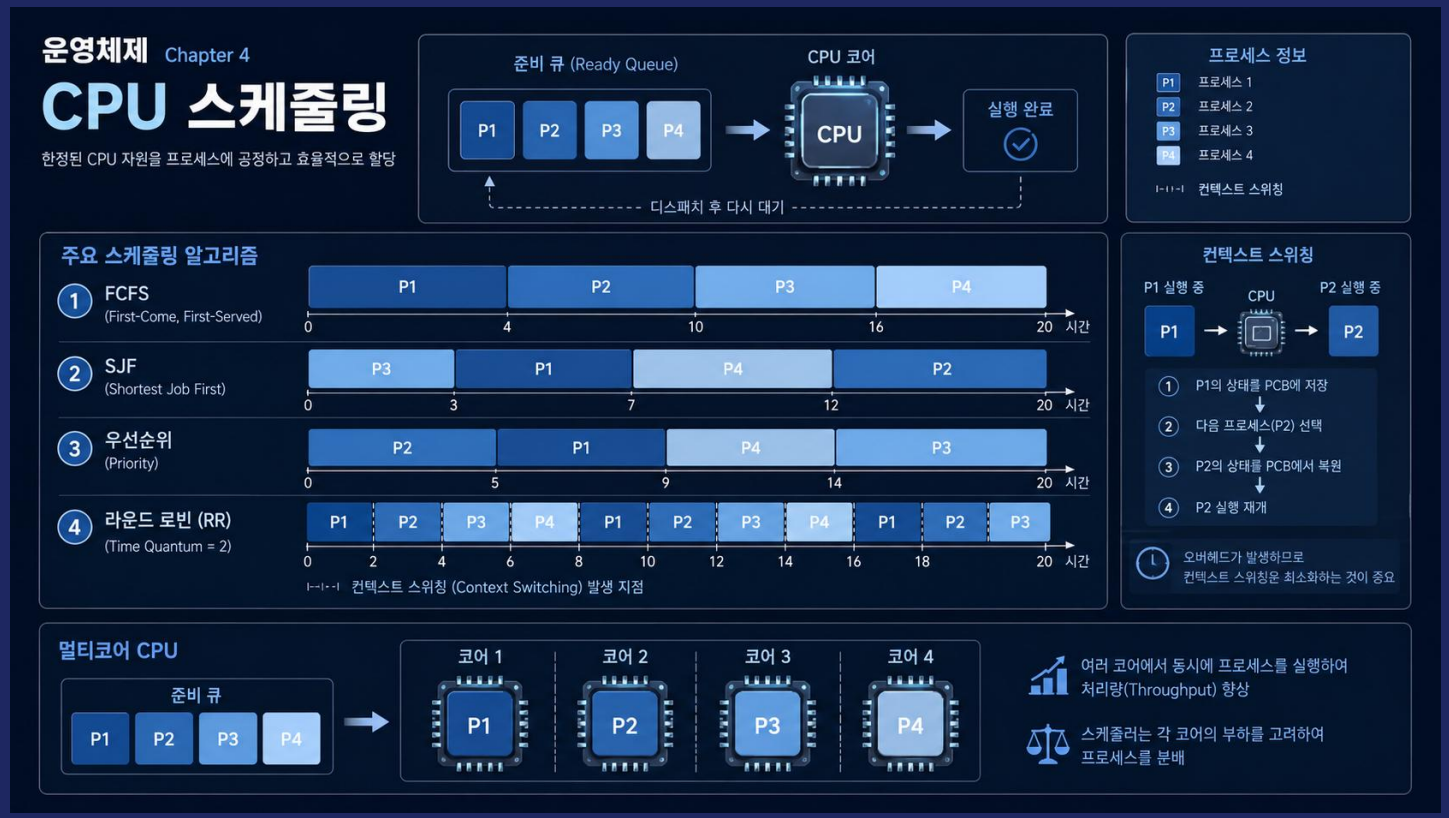
① FCFS
도착 순서대로 처리: 단순하지만 긴 작업이 앞에 있으면 Convoy Effect

② SJF
짧은 작업 우선: 평균 대기시간을 줄일 수 있지만 실행시간 예측이 필요

설명 포인트

- FCFS가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 SJF가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

CPU와 스케줄링 · 단계별 그림 해설 ②



2단계 해설

③ Priority

중요도 기반 실행: 낮은 우선순위의 Starvation 가능

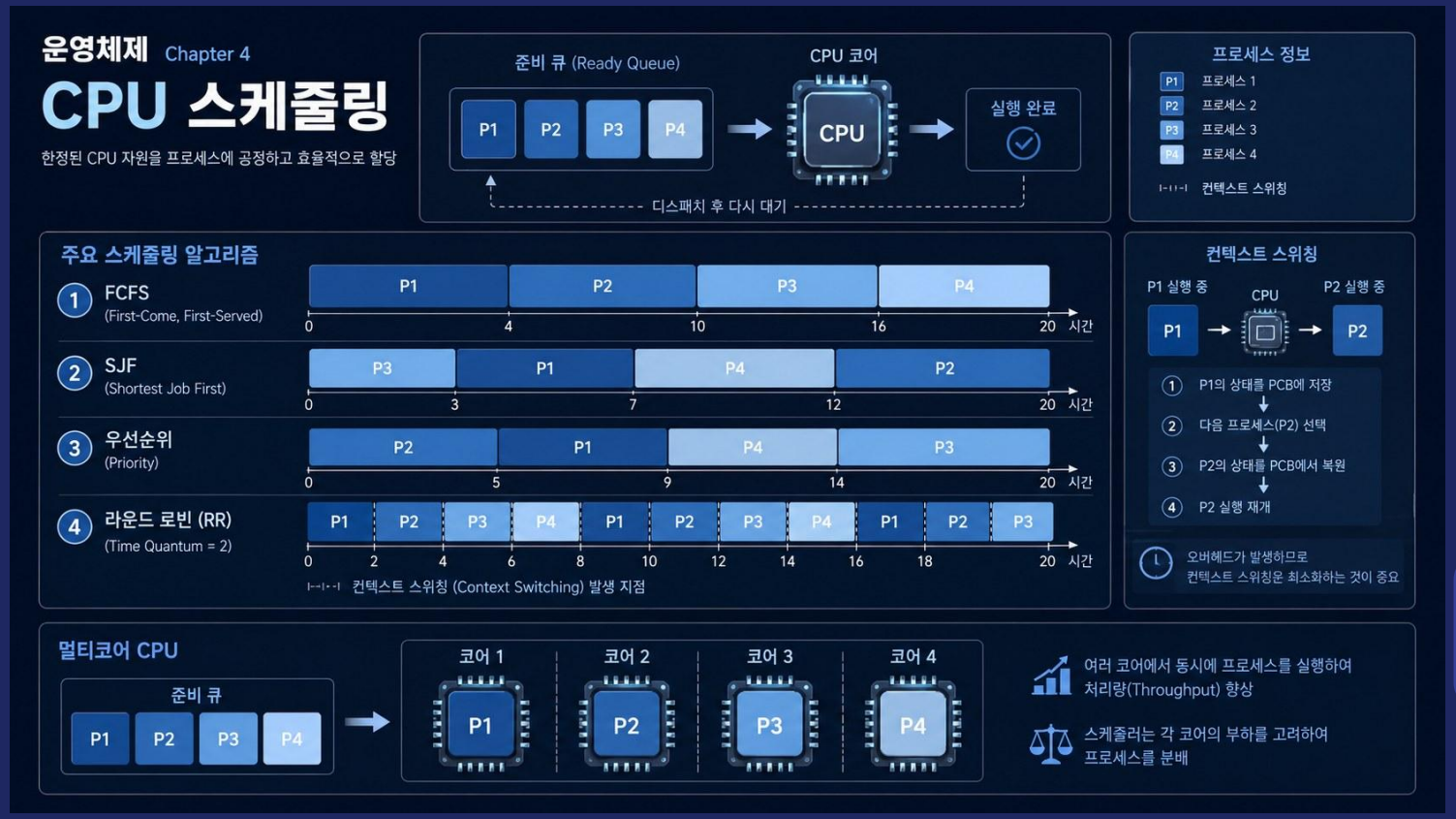
④ Round Robin

Quantum 단위 선점: 반응성이 좋지만 Quantum이 작으면 전환 오버헤드 증가

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- Priority와 Round Robin는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

CPU 스케줄링 · 복잡한 그림 세부 해설 1

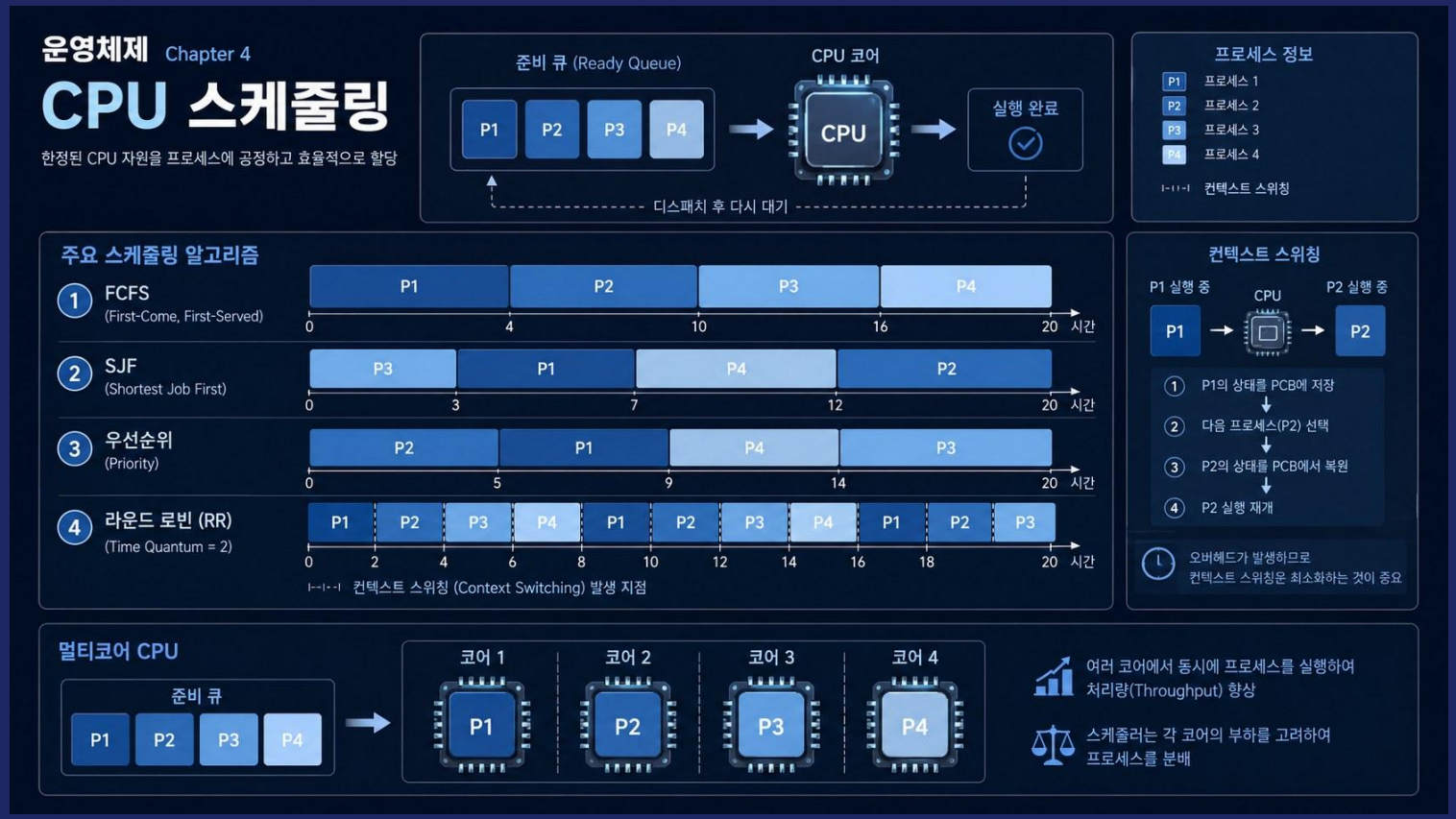


확대해서 볼 부분

1. Ready Queue
실행 가능한 프로세스가 CPU 할당을 기다리는 곳입니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

CPU 스케줄링 · 복잡한 그림 세부 해설 2



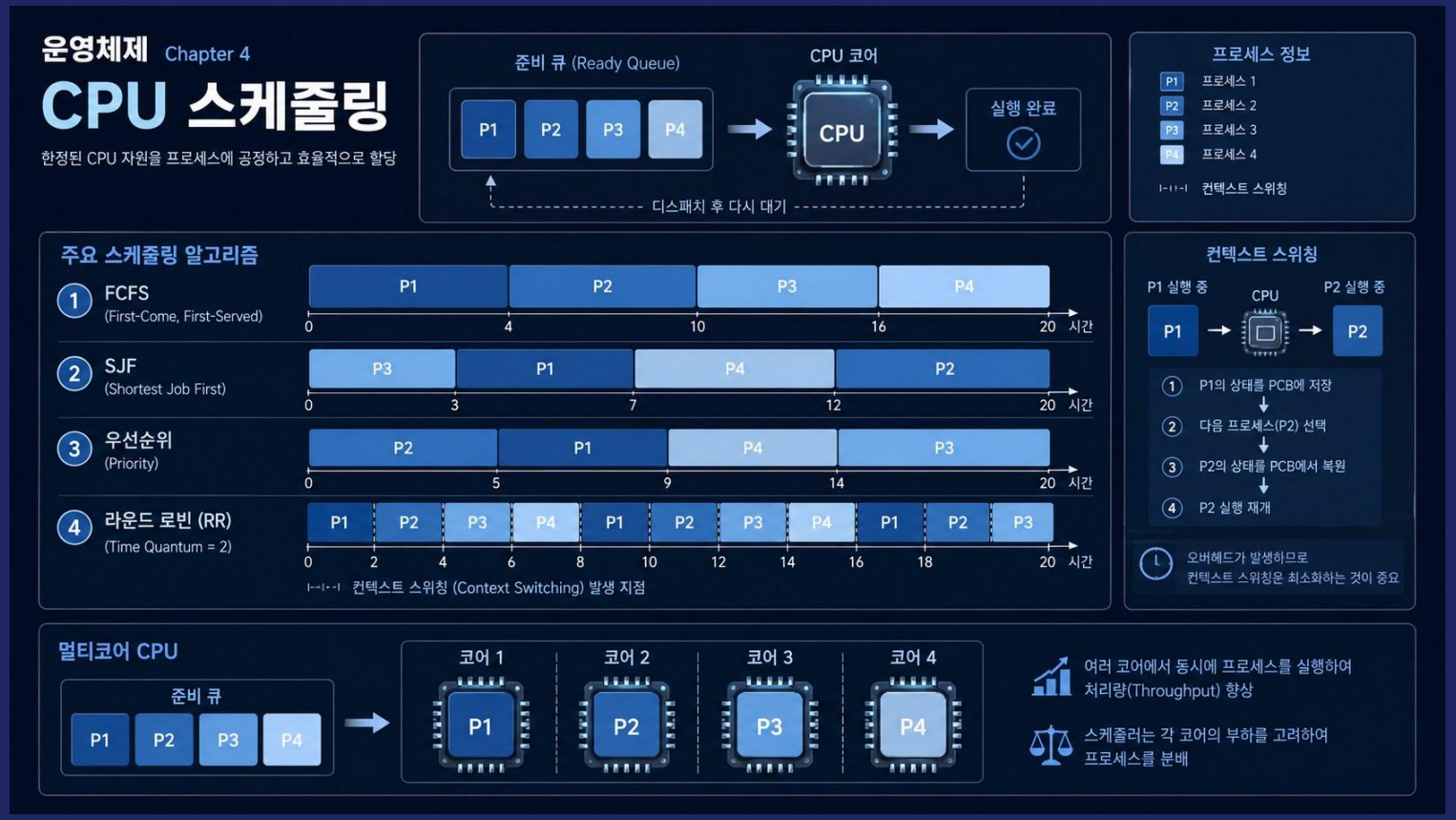
확대해서 볼 부분

2. 스케줄링 정책

FCFS·SJF·Priority·RR은 서로 다른 목표와 Trade-off를 갖습니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

CPU 스케줄링 · 복잡한 그림 세부 해설 3



확대해서 볼 부분

3. Context Switch

PCB 저장·복원과 캐시 영향 때문에 전환은 비용을 가집니다.

4. Multi-Core / Affinity

여러 코어에 부하를 나누고 필요하면 특정 코어에 프로세스를 고정합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

CPU와 스케줄링 · 동작 워크플로우

개념을 "정의"가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.

1 **Process 도착**
실행할 Process들이 도착

→

2 **Ready Queue**
CPU 앞에서 대기

→

3 **정책 선택**
FCFS/SJF/Priority/RR 정책 적용

→

4 **Dispatch**
선택된 Process를 CPU에 올림

→

5 **Burst / Quantum 실행**
Burst 또는 Quantum만큼 실행

→

6 **Switch / Complete**
종료하거나 Context Switch 후 다시 Queue

이 워크플로우에서 반드시 관찰할 3가지

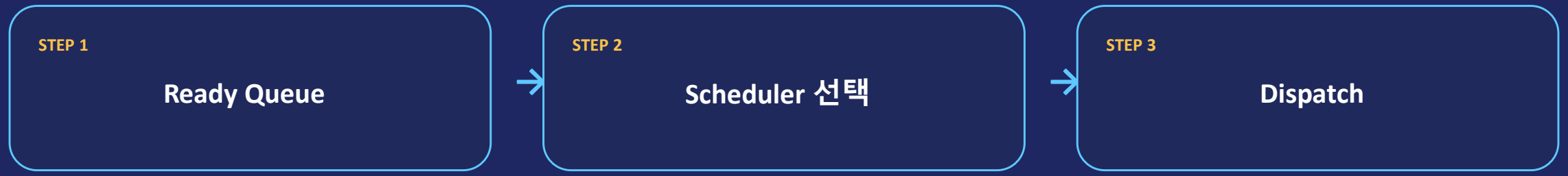
- 알고리즘이 같아도 입력 조건에 따라 결과가 달라짐
- Quantum이 작을수록 반응성은 좋아질 수 있지만 Switch가 증가
- 멀티코어는 병렬성과 부하 균형이 함께 필요

강사 멈춤 질문

어떤 정책이 "공정하다"고 말할 수 있을까?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

CPU 스케줄링 · 워크플로우 전반부 단계 해설



1	**Ready Queue**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**Scheduler 선택**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**Dispatch**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

CPU 스케줄링 · 워크플로우 후반부 단계 해설



4	CPU Burst	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	Context Switch	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	Complete/재대기	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

문제 상황: 반응성을 높이려고 Quantum을 계속 줄이면?

SCENARIO

반응성을 높이려고 Quantum을 계속 줄이면?

증상만 보고 바로 조치하지 말고 OS 상태 변화를 먼저 추적한다.

1 **STEP 1**

짧은 Quantum은 프로세스가 CPU를 자주 교대해 인터랙티브 반응성을 높인다.

2 **STEP 2**

하지만 PCB 저장/복원과 캐시 교란이 자주 발생한다.

3 **STEP 3**

실제 처리에 쓰는 시간보다 전환 비용이 커지면 전체 처리량이 떨어진다.

4 **STEP 4**

멀티코어에서도 부하 균형과 Affinity 정책이 중요하다.

▶ ANIMATION 준비 구간

CPU와 스케줄링 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
알고리즘 비교	FCFS→SJF→Priority→RR	Gantt 순서와 실행 Process가 달라짐	평균 대기시간, 완료 순서	어떤 정책이 "공정하다"고 말할 수 있을까?
Quantum 실험	RR Quantum 1→5	작은 Quantum에서 Process 전환이 자주 발생	Context Switch 횟수	Quantum을 1로 만들면 왜 항상 좋지 않은가?
멀티코어	Core 수 / Affinity 변경	Process가 여러 Core로 분산되거나 한 Core에 몰림	Core별 사용률, Queue	Affinity 고정이 도움이 되는 경우와 해가 되는 경우는?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 04 · CPU 스케줄링

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=4`

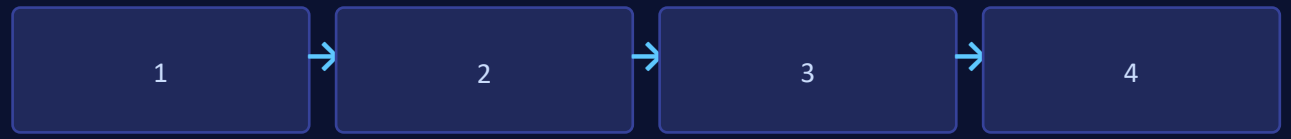
수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

▶ 지금 ANIMATION 수행

인터랙티브 시뮬레이션 · CPU Scheduler + Multi-Core Lab

HTML SIMULATOR

CPU Scheduler + Multi-Core Lab



1. 동일 P1~P4 조건에서 FCFS/SJF/Priority/RR 비교
2. RR에서 Quantum 1→5로 변경하며 Context Switch 횟수 기록
3. Core 수 1→4로 바꾸며 Queue 변화를 관찰
4. Affinity 고정 ON/OFF로 특정 Core 과부하 비교

시연 전에 반드시 질문

"버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
."

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 04 · CPU 스케줄링

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. FCFS/SJF/RR 실행 순서 비교
2. Quantum 변경으로 Switch 수 비교
3. Core 수와 Affinity 변화 관찰

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 04 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. SJF 실행 순서 직접 계산
2. RR Quantum 변화에 따른 장단점 서술
3. CPU 100% 상황에서 어떤 프로세스를 먼저 확인할지 진단 순서 작성

체크 질문

- SJF가 평균 대기시간에 유리한 이유는?
- Priority Scheduling의 Starvation은 어떻게 완화할 수 있을까?
- Context Switch는 왜 공짜가 아닌가?

파일: `os_24h_digital_worksheet.html`

DAY 2

메모리·파일·장치

데이터가 “어디에 있고, 어떻게 이동하는가”를 OS 관점으로 추적합니다.

C05 메모리 관리
C06 파일시스템
C07 장치와 I/O
C08 시스템 콜과 프로그램 실행



Chapter 05. 메모리 관리

Stack/Heap · Virtual Memory · Paging · Page Fault · Swap · OOM

학습 목표

1. Stack과 Heap의 용도를 구분한다.
2. Virtual Page가 Page Table을 통해 Physical Frame으로 매핑되는 과정을 설명한다.
3. Page Fault·Swap·OOM을 시스템 증상과 연결한다.

시작 질문

Page Fault가 항상 프로그램 오류는 아닌 이유는?



메모리 관리 · 그림 전체 보기



그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) Stack
2) Heap
3) Virtual Memory
4) Paging

메모리 관리 · 단계별 그림 해설 ①



1단계 해설

① **Stack**

함수 호출 프레임·지역 변수 중심, 일반적으로 호출/복귀에 따라 자동 관리

② **Heap**

동적 할당 영역, malloc/new와 free/delete 관리 필요

설명 포인트

- Stack가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 Heap가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

메모리 관리 · 단계별 그림 해설 ②



2단계 해설

③ Virtual Memory

프로세스마다 독립 주소 공간을 제공하고 물리 메모리와 분리

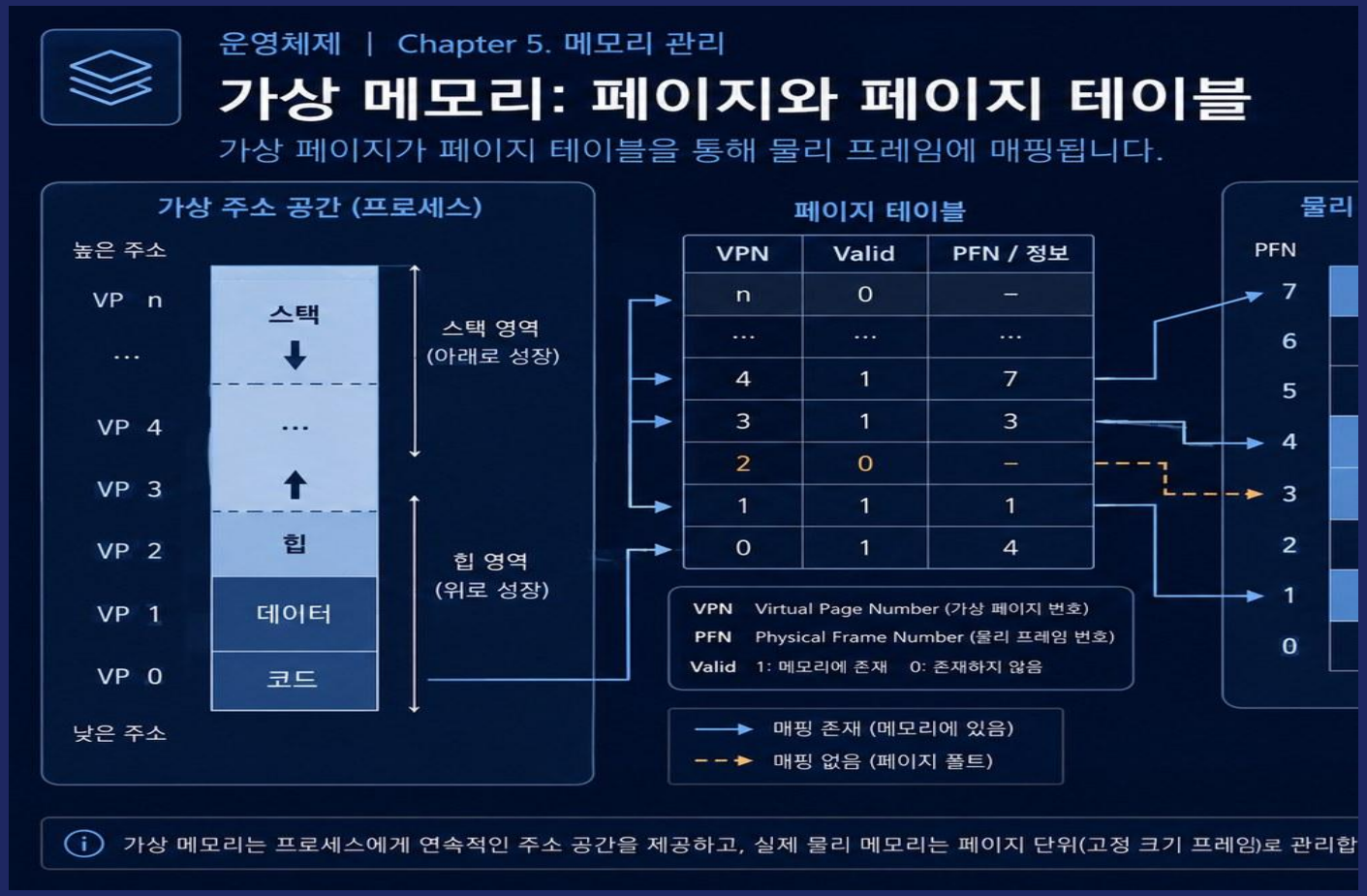
④ Paging

고정 크기 Page/Frame 매핑으로 물리 메모리의 불연속 배치를 허용

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- Virtual Memory와 Paging는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

메모리 관리 · 복잡한 그림 세부 해설 1





확대해서 볼 부분

1. 가상 주소 공간

프로세스는 연속된 Virtual Address를 보지만 실제 RAM은 분산 Frame일 수 있습니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

메모리 관리 · 복잡한 그림 세부 해설 2





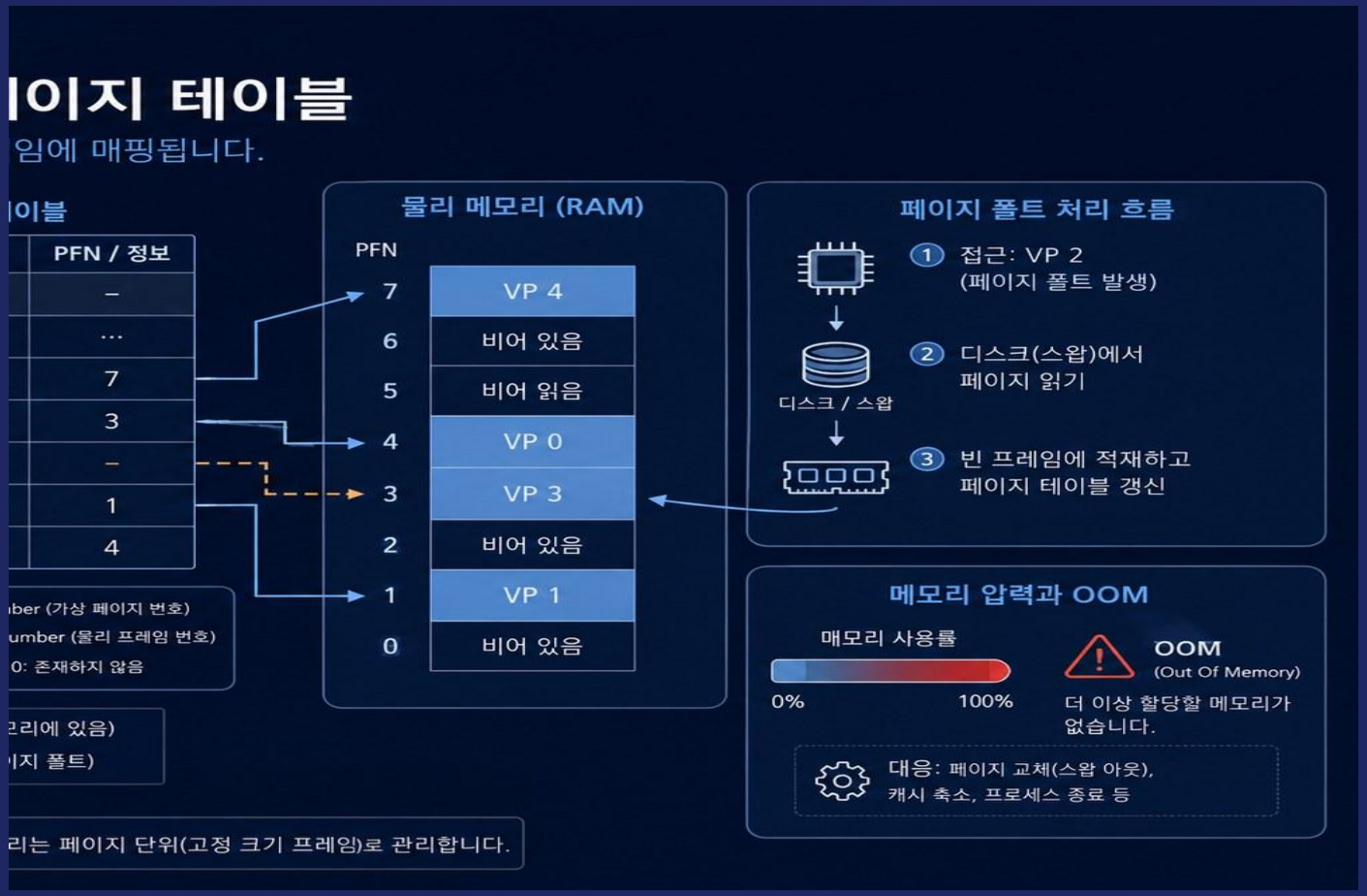
확대해서 볼 부분

2. Page Table

Virtual Page Number를 Physical Frame Number로 매핑합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

메모리 관리 · 복잡한 그림 세부 해설 3





확대해서 볼 부분

3. Page Fault

RAM에 없는 Page 접근 시 OS가 저장장치에서 Page In을 수행합니다.

4. Swap / OOM

메모리 압력이 높아지면 Swap과 교체가 늘고 최종적으로 OOM 종료가 발생할 수 있습니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

메모리 관리 · 동작 워크플로우

개념을 "정의"가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.

1. **가상 주소 접근** — Process는 Virtual Address를 사용
2. **Page Table 조회** — VPN을 Page Table에서 조회
3. **Hit / Page Fault** — RAM에 있으면 Hit, 없으면 Fault
4. **Frame 적재·교체** — Disk/Swap에서 Page를 Frame에 적재
5. **명령 재실행** — Page Table 갱신 후 중단된 명령 재실행
6. **Swap / OOM 대응** — 메모리 압박 시 Swap 또는 OOM 처리

이 워크플로우에서 반드시 관찰할 3가지

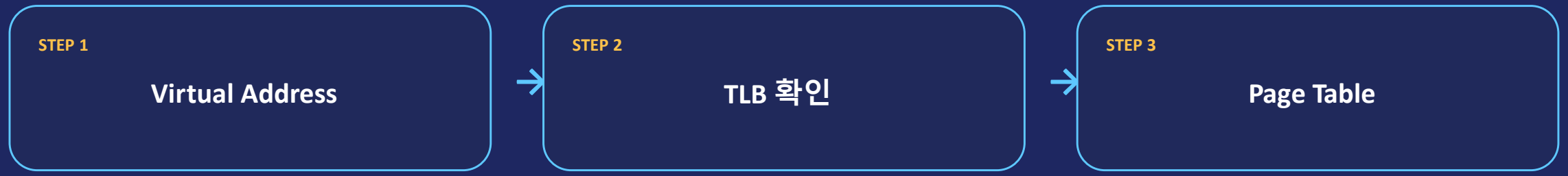
- Page Fault는 정상 메커니즘일 수 있음
- Fault 빈도가 높으면 성능 문제가 됨
- Stack/Heap 성장과 Physical RAM 압박은 서로 연결됨

강사 멈춤 질문

Page Fault가 오류가 아니라는 뜻은?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

메모리 관리 · 워크플로우 전반부 단계 해설



1	**Virtual Address**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**TLB 확인**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**Page Table**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

메모리 관리 · 워크플로우 후반부 단계 해설



4	RAM Frame	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	Page Fault	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	Disk/Swap	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

Page Fault는 오류인가, 정상 동작인가?

SCENARIO

Page Fault는 오류인가, 정상 동작인가?

증상만 보고 바로 조치하지 말고
OS 상태 변화를 먼저 추적한다.

1 STEP 1

Virtual Page에 접근했는데 RAM에 없으면 Page Fault가 발생한다.

2 STEP 2

OS는 Disk/Swap에서 필요한 Page를 읽어 빈 Frame에 적재한다.

3 STEP 3

Frame이 없으면 교체 정책으로 기존 Page를 내보낼 수 있다.

4 STEP 4

Fault가 과도하면 Thrashing과 성능 저하, 메모리 고갈 시 OOM이 발생한다.

▶ ANIMATION 준비 구간

메모리 관리 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
Page Hit/Fault	여러 Virtual Page 선택	Frame이 강조되거나 Disk→RAM 이동이 발생	Hit/Fault, Frame 매핑	Page Fault가 오류가 아니라는 뜻은?
Stack / Heap	함수 호출·malloc 반복	메모리 블록이 양쪽에서 증가	RAM 사용량, Stack/Heap 크기	Memory Leak은 어느 영역과 더 관련이 깊은가?
압박과 OOM	할당 반복 후 임계치 초과	Swap 게이지가 올라가고 마지막에 OOM 상태 표시	RAM, Swap, OOM	RAM이 가득 찼다고 즉시 OOM이 나는 것은 왜 아닌가?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 05 · 메모리 관리

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=5`

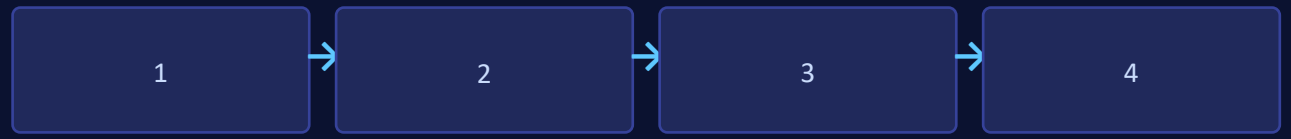
수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

▶ 지금 ANIMATION 수행

인터랙티브 시뮬레이션 · Paging + Stack/Heap/Swap/OOM 시뮬레이터

HTML SIMULATOR

Paging + Stack/Heap/Swap/OOM 시뮬레이터



1. Virtual Page를 순서대로 눌러 Hit와 Fault를 구분
2. Frame이 가득 찬 뒤 새 Page 접근으로 Replacement 확인
3. 함수 호출/ malloc을 반복하며 Stack/Heap 성장 관찰
4. RAM 사용량이 임계치를 넘을 때 Swap과 OOM 상태 확인

시연 전에 반드시 질문

**“버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
.”**

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 05 · 메모리 관리

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. Page Hit/Fault Latency 비교
2. Replacement 정책 확인
3. Memory Pressure에서 Swap/OOM 관찰

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 05 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. Page Fault 처리 순서를 카드로 배열
2. RAM 98% + Swap 2GB 사례 원인 분석
3. Virtual Memory의 장점을 격리·효율 관점으로 설명

체크 질문

- Page Fault가 항상 프로그램 오류는 아닌 이유는?
- Memory Leak과 OOM의 관계는?
- Stack Overflow와 Heap Exhaustion은 어떻게 다른가?

파일: `os_24h_digital_worksheet.html`

Chapter 06. 파일시스템

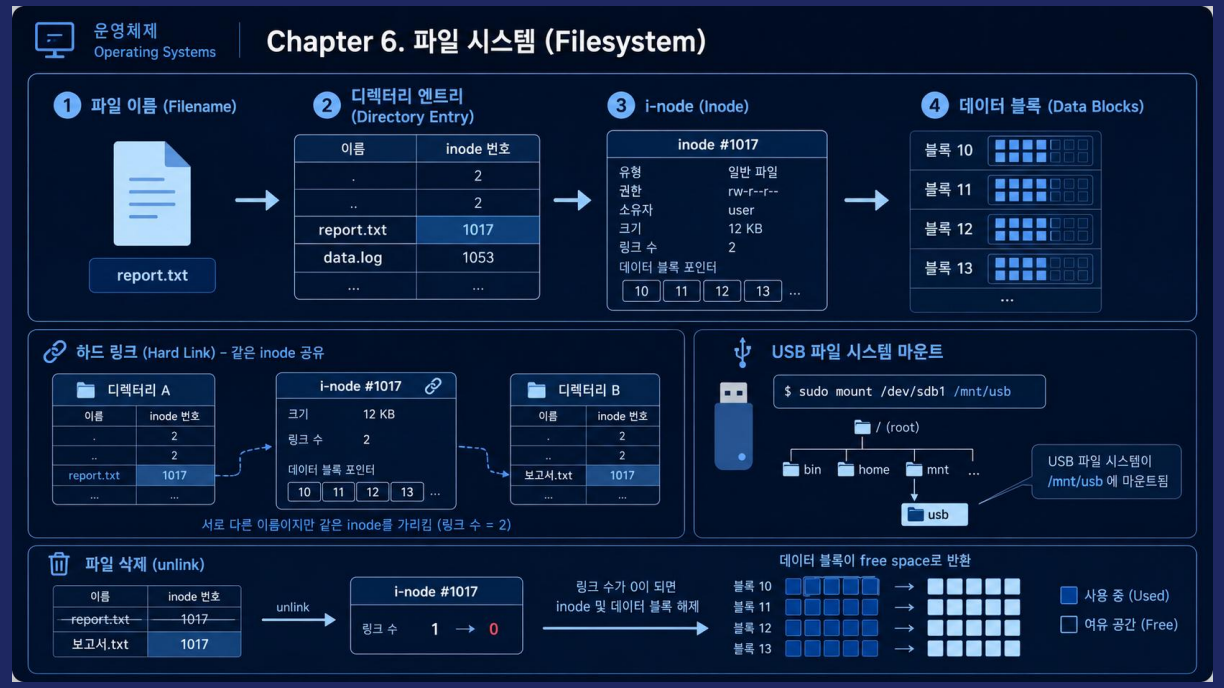
Directory Entry · inode · Data Block · Link · Mount · Delete

학습 목표

1. 파일명과 inode, Data Block의 관계를 설명한다.
2. Hard Link와 Mount의 의미를 설명한다.
3. 파일 삭제 시 실제 데이터가 언제 반환되는지 이해한다.

시작 질문

Hard Link가 같은 inode를 공유한다는 것은 무엇을 의미하는가?



파일시스템 · 그림 전체 보기

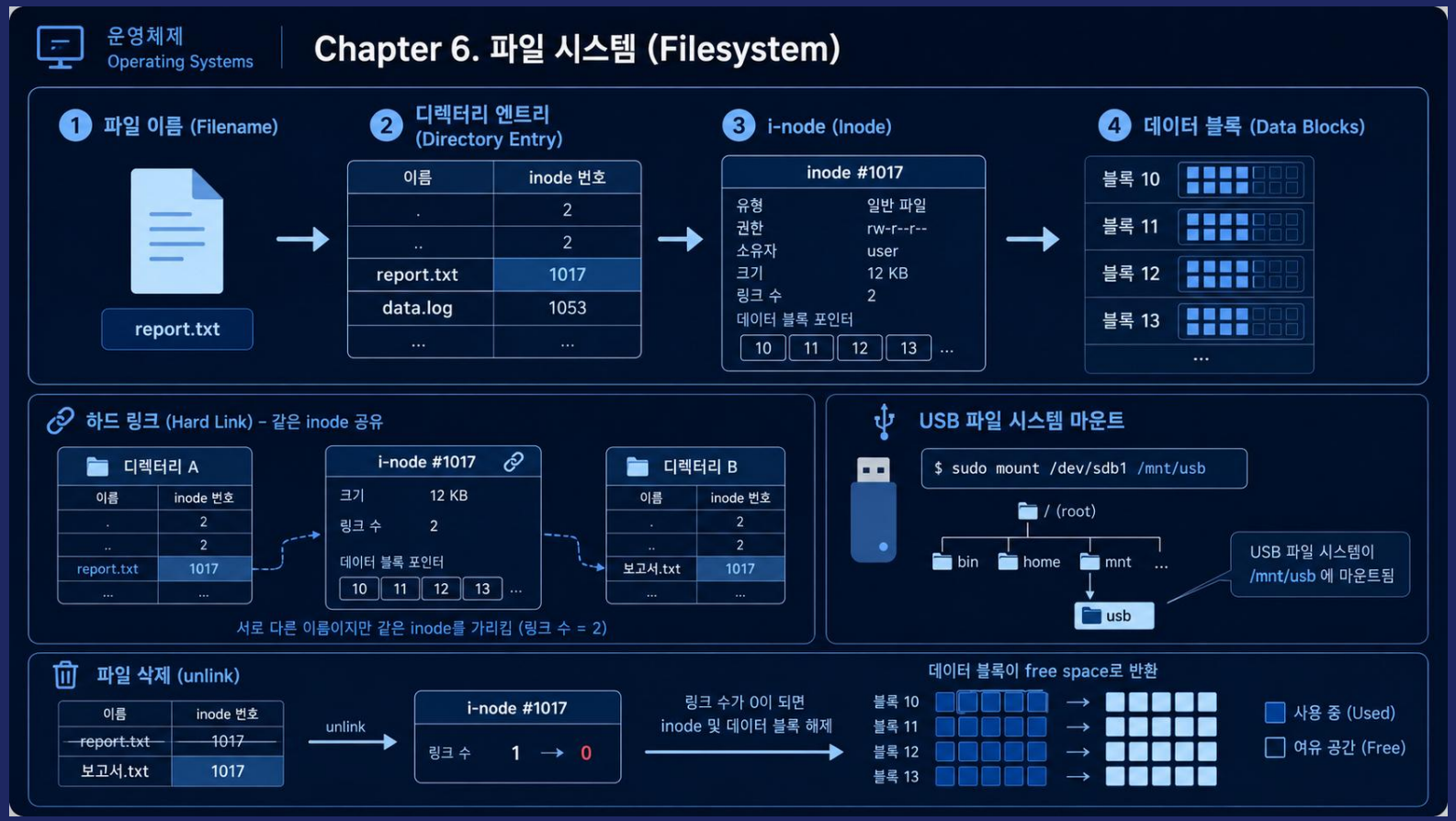


그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

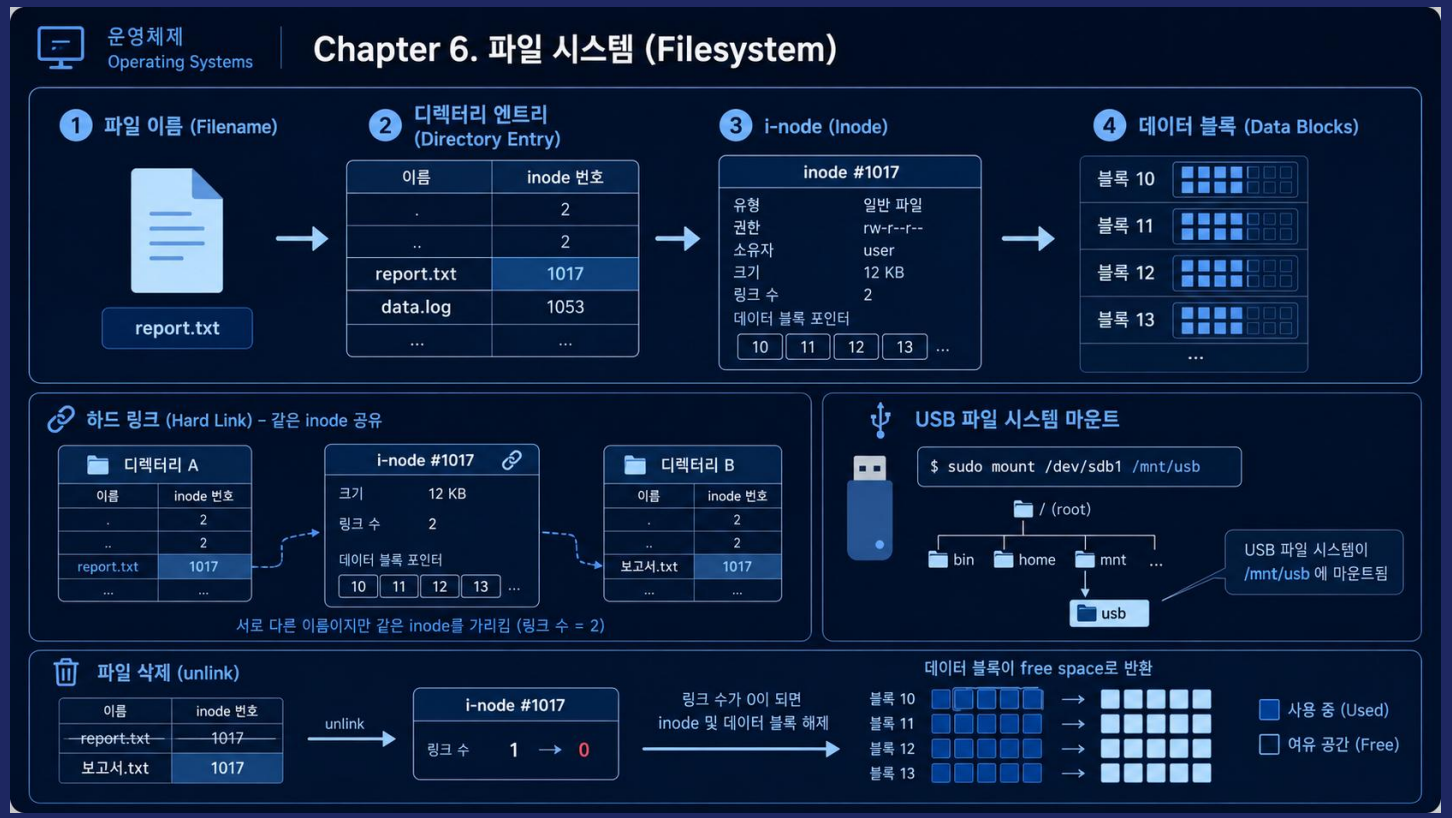
설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) Directory Entry
2) inode
3) Data Block
4) Mount

파일시스템 · 단계별 그림 해설 ①



1단계 해설

① Directory Entry

파일명과 inode 번호의 매핑

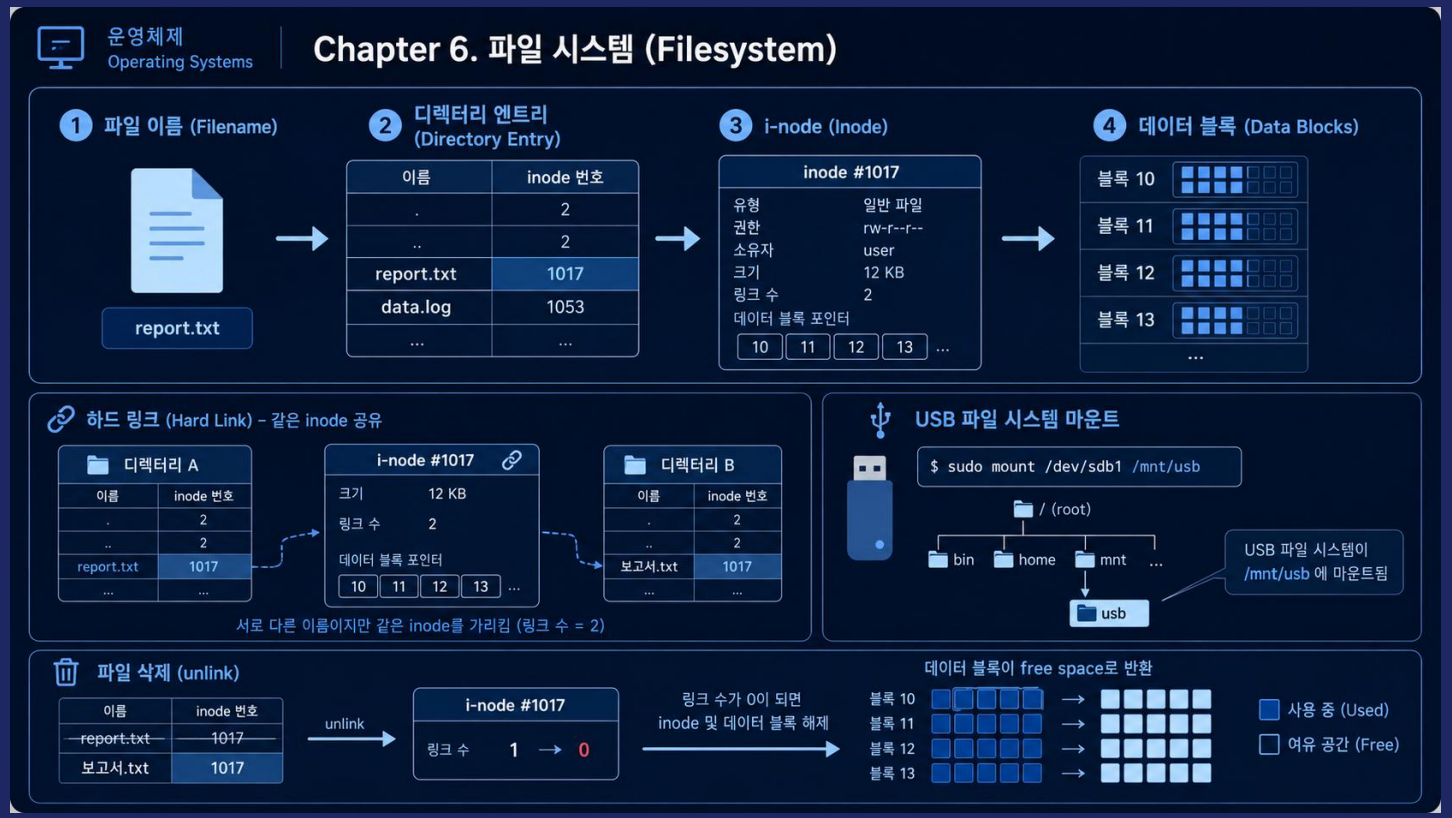
② inode

권한·소유자·크기·블록 포인터 등 메타데이터

설명 포인트

- Directory Entry가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 inode가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

파일시스템 · 단계별 그림 해설 ②



2단계 해설

③ Data Block

파일의 실제 내용 저장

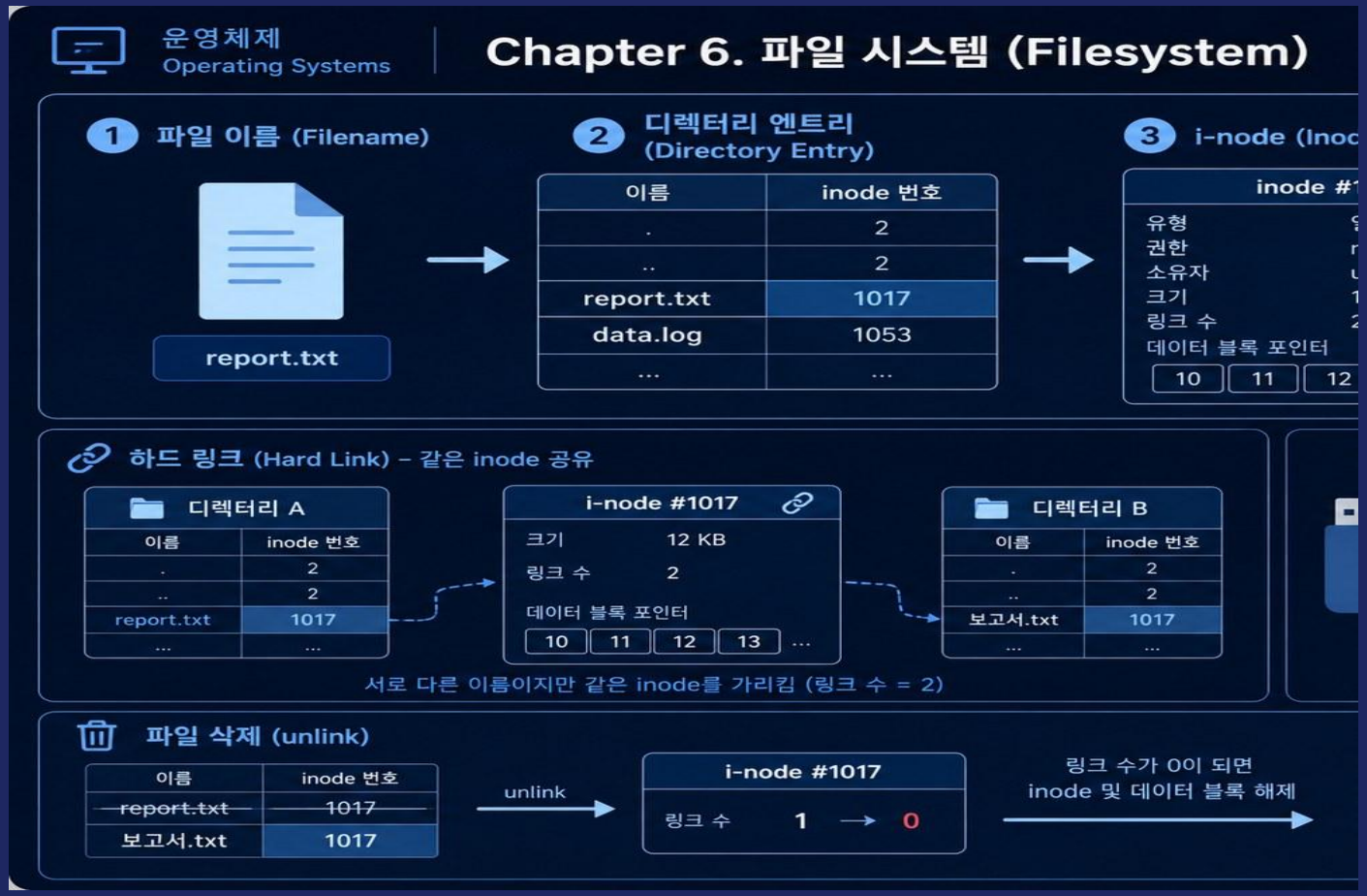
④ Mount

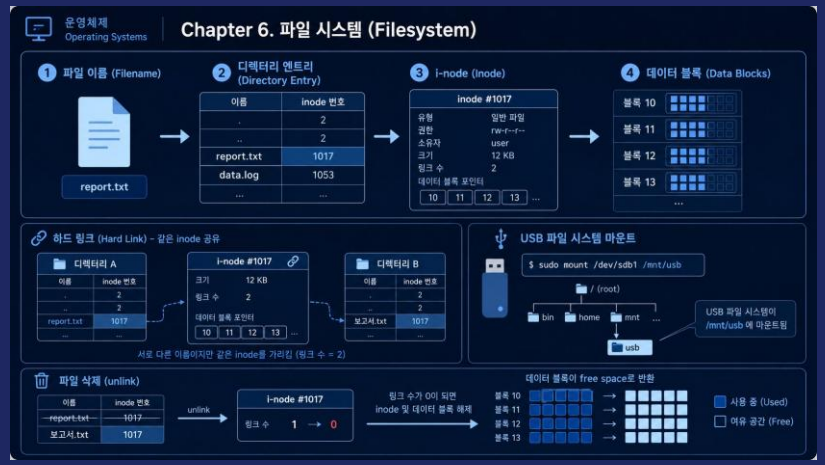
별도 파일시스템을 하나의 디렉터리 트리에 연결

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- Data Block와 Mount는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

파일시스템 · 복잡한 그림 세부 해설 1



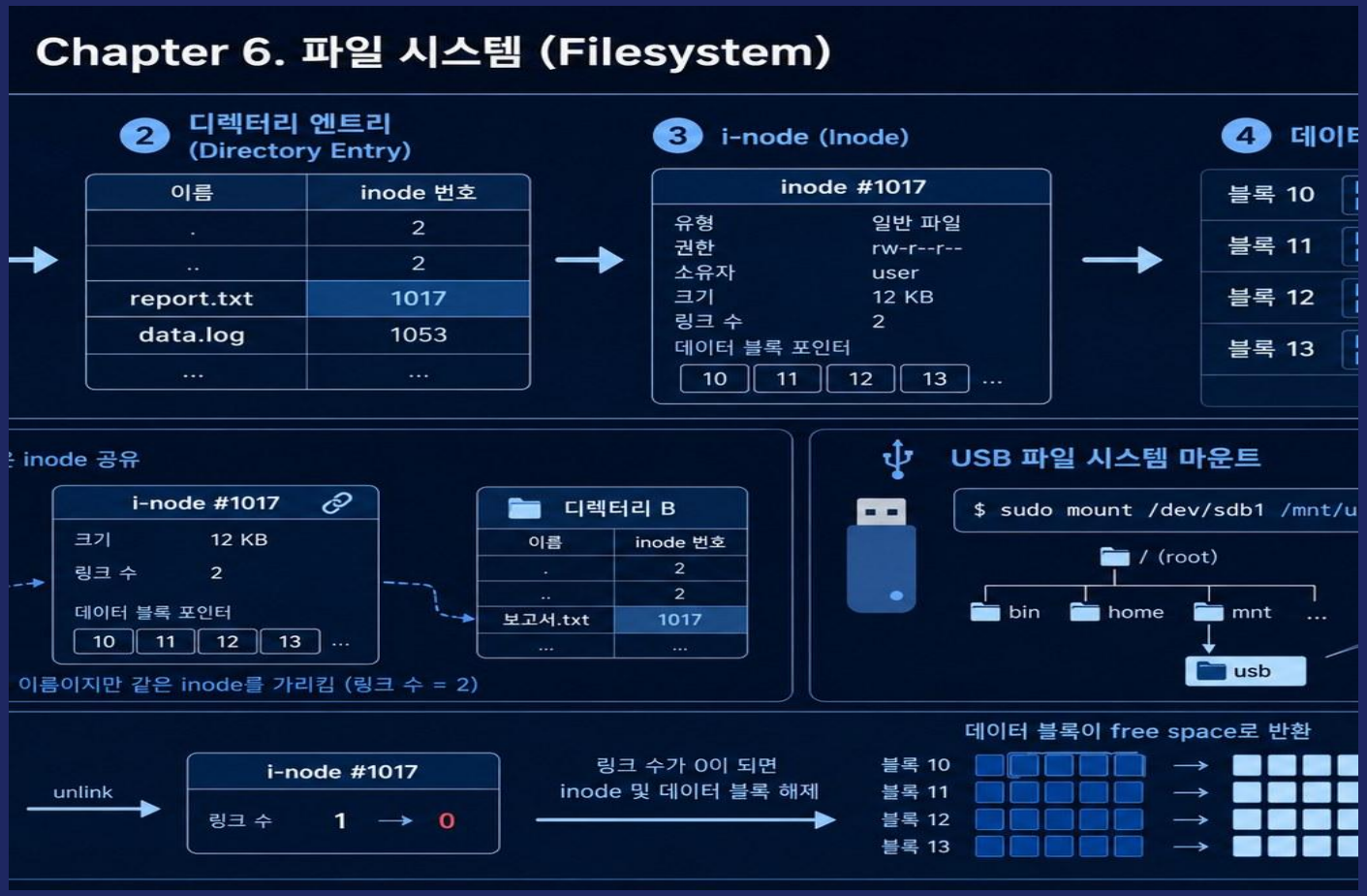


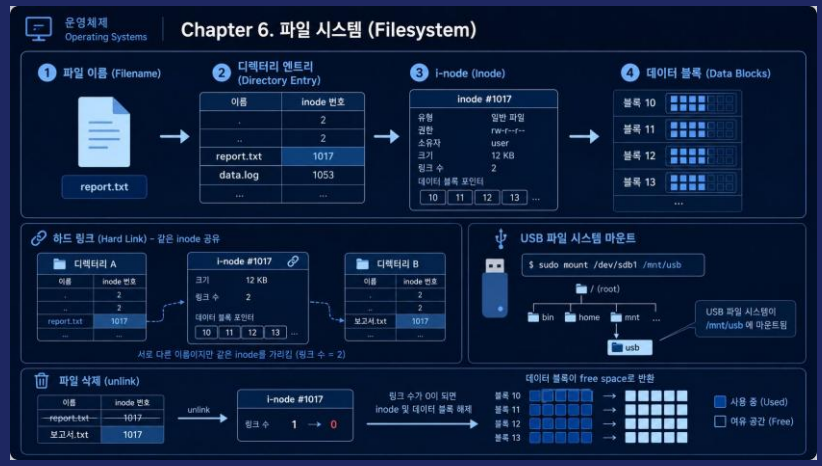
확대해서 볼 부분

1. Directory Entry
파일명과 inode 번호를 연결하는 이름 공간입니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

파일시스템 · 복잡한 그림 세부 해설 2



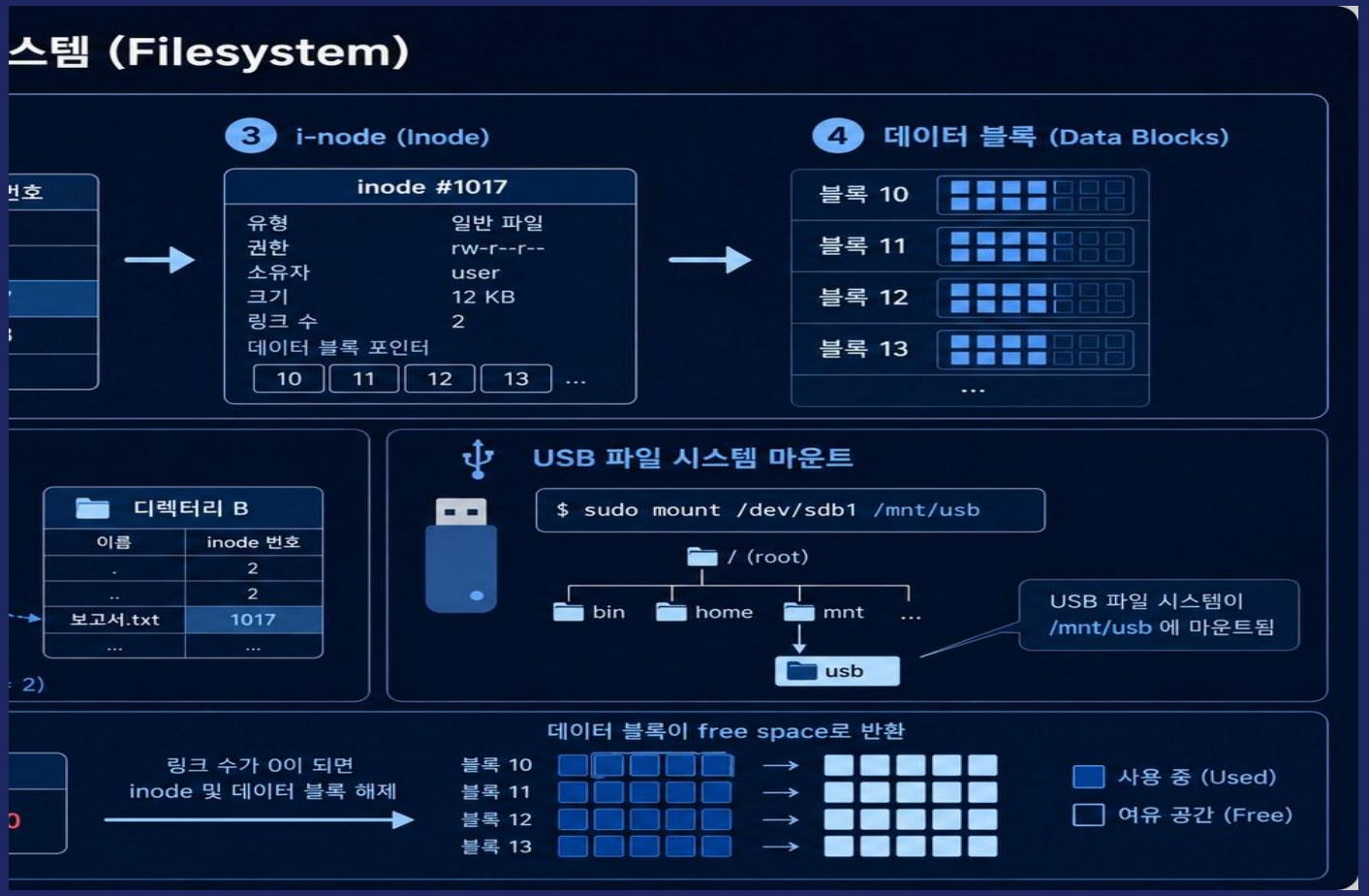


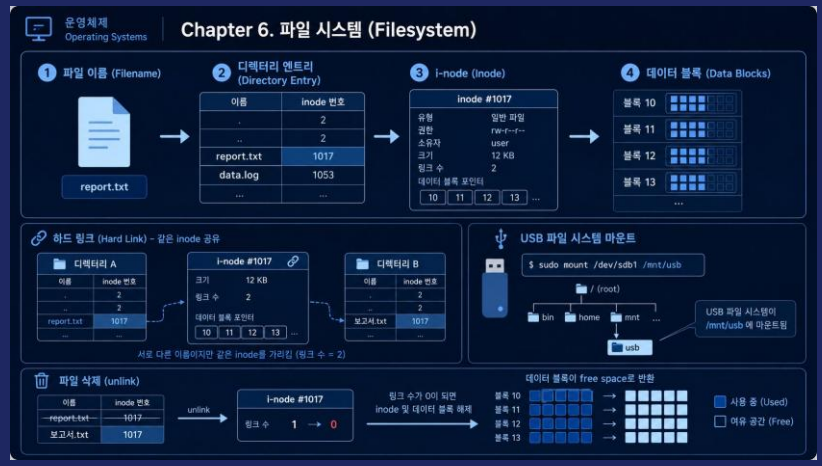
확대해서 볼 부분

2. inode
권한·소유자·크기·Block 위치 같은 메타데이터를 가집니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

파일시스템 · 복잡한 그림 세부 해설 3





확대해서 볼 부분

3. Data Block
파일의 실제 내용이 저장되는 영역입니다.

4. Link / Mount / Delete
Hard Link와 open FD 때문에 unlink 후에도 실제 Block 반환이 지연될 수 있습니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

파일시스템 · 동작 워크플로우

개념을 "정의"가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.



이 워크플로우에서 반드시 관찰할 3가지

- 파일명과 실제 데이터는 분리되어 있음
- Hard Link는 이름이 달라도 같은 inode를 공유
- unlink는 이름을 지우는 것이며 블록 반환 시점과 다를 수 있음

강사 멈춤 질문

파일명만 바꿔도 데이터가 그대로일 수 있는 이유는?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

파일시스템 · 워크플로우 전반부 단계 해설



1	**Filename**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**Directory Entry**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**inode Lookup**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

파일시스템 · 워크플로우 후반부 단계 해설



4	**Data Block**	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	**Open FD**	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	**Read/Write/Unlink**	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

파일을 지웠는데 왜 디스크 공간이 바로 안 늘 수도 있을까?

SCENARIO

파일을 지웠는데 왜 디스크 공간이 바로 안 늘 수도 있을까?

증상만 보고 바로 조치하지 말고 OS 상태 변화를 먼저 추적한다.

1 STEP 1

unlink는 우선 파일명(Directory Entry)과 inode 연결을 제거한다.

2 STEP 2

Hard Link가 남아 있으면 inode의 link count가 0이 아니다.

3 STEP 3

프로세스가 파일을 열어둔 경우도 실제 블록 반환이 지연될 수 있다.

4 STEP 4

최종 참조가 사라질 때 inode와 Data Block이 Free Space로 돌아간다.

▶ ANIMATION 준비 구간

파일시스템 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
읽기 경로	파일 읽기 실행	Filename→inode→Block 경로가 순차 강조	inode 번호, Block 수	파일명만 바꿔도 데이터가 그대로일 수 있는 이유는?
Hard Link	Hard Link 추가	link count가 증가하고 두 이름이 같은 inode를 가리킴	link count	한 이름을 지워도 데이터가 남을 수 있는 이유는?
Mount / Delete	USB Mount 후 파일 삭제	파일 트리에 USB가 붙고 삭제 시 Block 상태 변화	Mount point, free block	Mount는 복사와 어떻게 다른가?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 06 · 파일시스템

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=6`

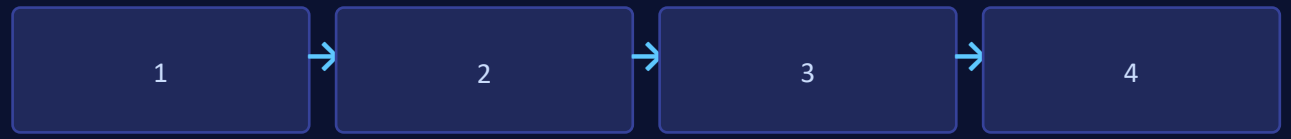
수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

▶ 지금 ANIMATION 수행

인터랙티브 시뮬레이션 · inode·Block + Mount/Delete 시뮬레이터

HTML SIMULATOR

inode·Block + Mount/Delete 시뮬레이터



1. 파일 크기를 늘려 Data Block 개수 변화 확인
2. Hard Link를 추가해 link count 증가 확인
3. USB Mount로 /mnt/usb에 파일이 나타나는 과정 확인
4. 파일 삭제/복구 시뮬레이션으로 Block 상태 변화 관찰

시연 전에 반드시 질문

"버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
."

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 06 · 파일시스템

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. Hard Link 후 link count 관찰
2. unlink 후 Block 유지 조건 확인
3. USB Mount 경로 연결

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 06 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. 파일 읽기 경로를 Filename→Directory→inode→Block 순으로 작성
2. df와 du의 차이를 사례로 구분
3. “공간은 남았는데 파일 생성 실패”에서 inode 사용량을 확인하는 이유 설명

체크 질문

- Hard Link가 같은 inode를 공유한다는 것은 무엇을 의미하는가?
- Mount는 파일 복사와 어떻게 다른가?
- 삭제와 실제 블록 반환은 왜 같은 순간이 아닐 수 있는가?

파일: `os_24h_digital_worksheet.html`

Chapter 07. 장치와 I/O

Device Driver · Polling · Interrupt · DMA · Buffer

학습 목표

1. 응용프로그램부터 장치까지 I/O 경로를 설명한다.
2. Polling·Interrupt·DMA의 CPU 부담 차이를 비교한다.
3. 입력속도와 처리속도의 불균형이 Buffer Overflow와 Drop으로 이어짐을 설명한다.

시작 질문

DMA는 CPU를 완전히 사용하지 않는다는 뜻인가?



장치와 I/O · 그림 전체 보기



그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) Device Driver
2) Polling
3) Interrupt
4) DMA

장치와 I/O · 단계별 그림 해설 ①



1단계 해설

① Device Driver

장치별 제어 방식을 OS의 표준 인터페이스로 연결

② Polling

CPU가 반복적으로 상태를 확인: 단순하지만 낭비 가능

설명 포인트

- Device Driver가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 Polling가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

장치와 I/O · 단계별 그림 해설 ②



2단계 해설

③ Interrupt

이벤트 발생 시 장치가 CPU에 알림: 불필요한 확인 감소

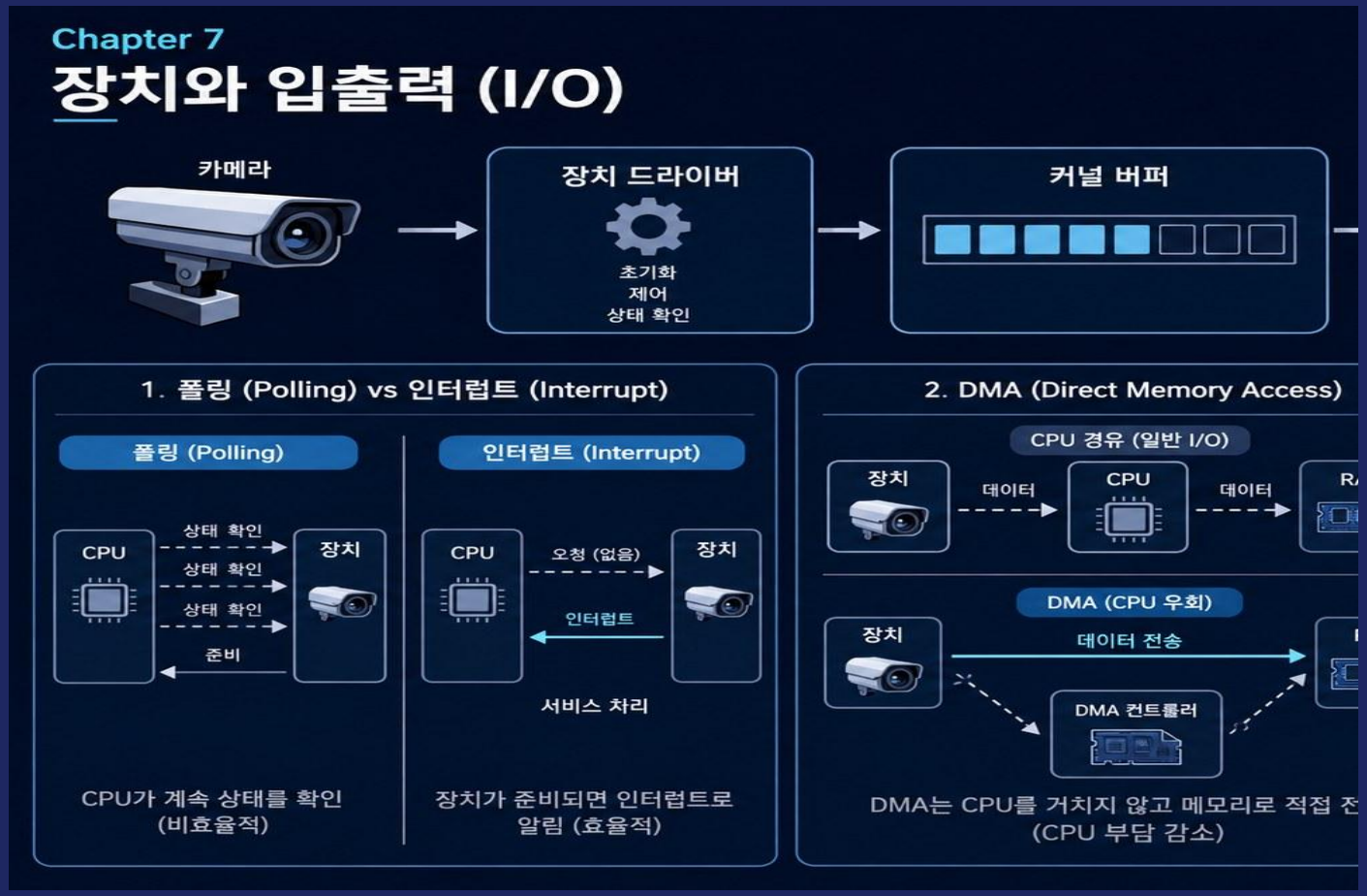
④ DMA

장치↔메모리 전송을 CPU 대신 DMA Controller가 수행해 부하 감소

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- Interrupt와 DMA는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

장치와 I/O · 복잡한 그림 세부 해설 1





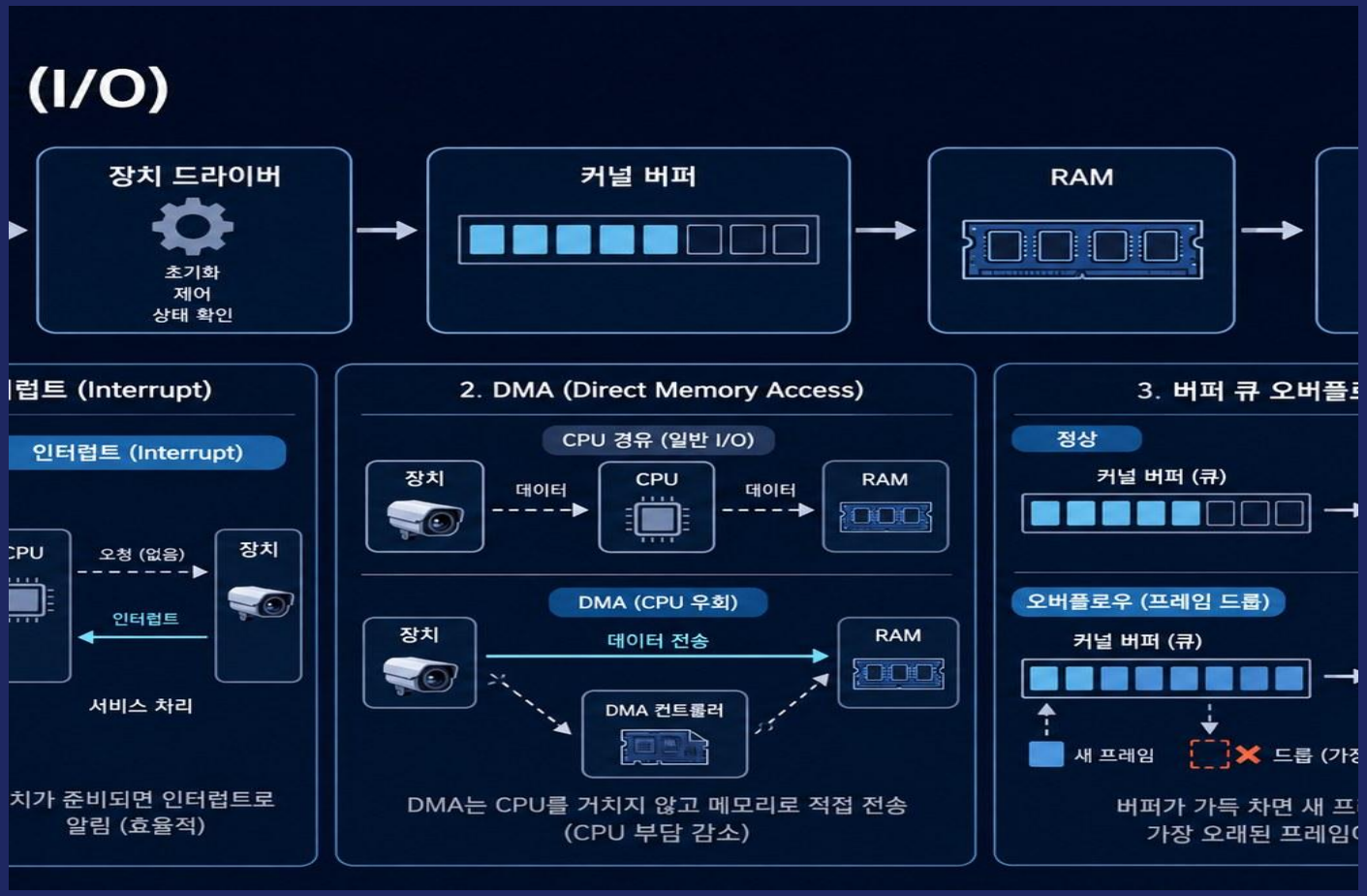
확대해서 볼 부분

1. Device Driver

장치별 제어 방식을 Kernel의 표준 I/O 인터페이스에 연결합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

장치와 I/O · 복잡한 그림 세부 해설 2





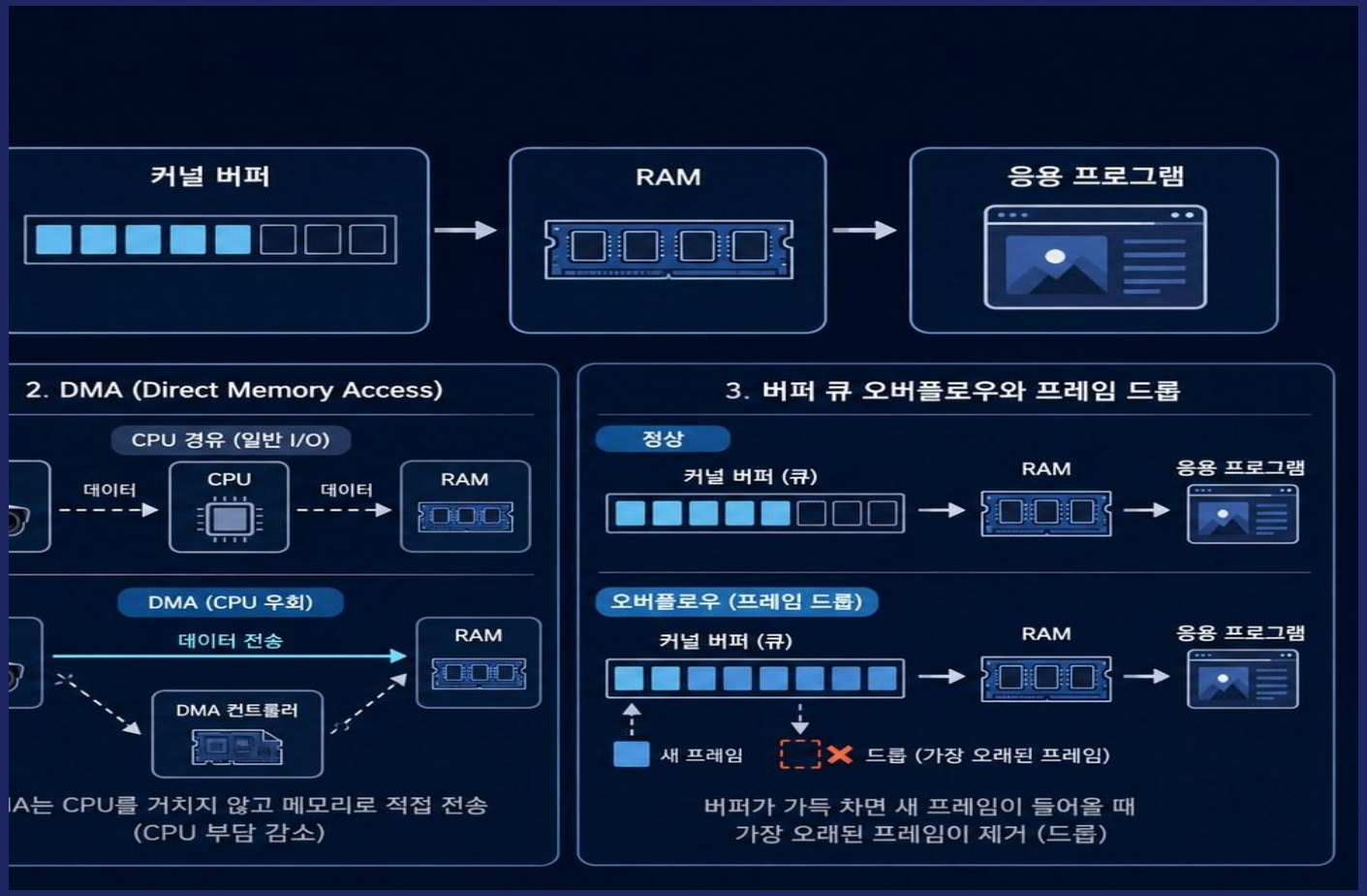
확대해서 볼 부분

2. Polling / Interrupt

CPU가 직접 반복 확인하는 방식과 장치가 이벤트를 알리는 방식을 비교합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

장치와 I/O · 복잡한 그림 세부 해설 3





확대해서 볼 부분

3. DMA

대량 데이터 전송에서 CPU 대신 DMA가 RAM 전송을 담당해 부하를 줄입니다.

4. Buffer / Drop

생산 속도가 소비 속도보다 높으면 Queue가 쌓이고 Buffer가 차면 Frame Drop이 발생합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

장치와 I/O · 동작 워크플로우

개념을 "정의"가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.

1. **Device Event** — Camera 등 장치에서 데이터 발생
2. **Driver** — Driver가 장치별 제어를 추상화
3. **Interrupt / DMA** — Interrupt로 알리거나 DMA로 전송
4. **Kernel Buffer** — Kernel Buffer에 데이터 축적
5. **User Buffer** — User Space로 복사/매핑
6. **Application 소비** — Application이 처리 속도에 맞춰 소비

이 워크플로우에서 반드시 관찰할 3가지

- 생산 속도 > 소비 속도면 Queue가 누적
- Polling/Interrupt/DMA는 CPU 관여 방식이 다름
- Buffer 크기는 Drop과 Latency 사이의 절충

강사 멈춤 질문

입력과 처리 속도가 같으면 Queue는 어떻게 되는가?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

장치와 I/O · 워크플로우 전반부 단계 해설



1	**Camera 생성**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**Driver 처리**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**IRQ/DMA**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

장치와 I/O · 워크플로우 후반부 단계 해설



4	**Ring Buffer**	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	**RAM 전달**	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	**App 소비**	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

카메라 60 FPS 입력, 애플리케이션 20 FPS 처리 — 무슨 일이 생길까?

SCENARIO

카메라 60 FPS 입력, 애플리케이션 20 FPS 처리 — 무슨 일이 생길까?

증상만 보고 바로 조치하지 말고
OS 상태 변화를 먼저 추적한다.

1 STEP 1

초당 60개의 Frame이 Buffer에 들어오는데 20개만 소비된다.

2 STEP 2

초기에는 Buffer가 차면서 지연이 증가한다.

3 STEP 3

Buffer가 가득 차면 정책에 따라 새 Frame 또는 오래된 Frame이 Drop된다.

4 STEP 4

실시간 AI에서는 “정확도”뿐 아니라 Queue 길이·Latency·Drop 정책도 중요하다.

▶ ANIMATION 준비 구간

장치와 I/O · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
Camera Pipeline	Input FPS 30, Buffer 5	Frame 카드가 Driver→Buffer→App으로 이동	Queue 길이, Drop	입력과 처리 속도가 같으면 Queue는 어떻게 되는가?
Overflow	Input FPS 60, Buffer 축소	Buffer가 차고 초과 Frame이 Drop 표시	Drop 수, 지연	Buffer를 크게 하면 무엇이 좋아지고 무엇이 나빠지는가?
I/O 방식 비교	Polling→Interrupt→DMA	CPU/Speed/Interrupt 게이지가 서로 다르게 변화	CPU %, 전송속도	DMA가 CPU를 완전히 배제한다는 말은 맞는가?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 07 · 장치와 I/O

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=7`

수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

인터랙티브 시뮬레이션 · Camera I/O Pipeline + Polling/Interrupt/DMA 비교

▶ 지금 ANIMATION 수행

HTML SIMULATOR

Camera I/O Pipeline + Polling/Interrupt/DMA 비교



1. Input FPS와 Buffer 크기를 바꾸어 Drop 수 관찰
2. Polling 모드에서 CPU 사용률 확인
3. Interrupt로 전환해 CPU 사용률 감소 비교
4. DMA 선택 후 Transfer Speed와 CPU 점유율 변화 비교

시연 전에 반드시 질문

“버튼을 누르기 전에 무엇이 어떻게 변할지 예측해보세요.”

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 07 · 장치와 I/O

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. Polling/Interrupt/DMA CPU 사용률 비교
2. Input FPS와 Buffer 크기 조정
3. Queue→Latency→Drop 순서 확인

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 07 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. Camera→Driver→Kernel Buffer→RAM→App 흐름 작성
2. Polling과 Interrupt를 “누가 언제 확인하는가” 관점으로 설명
3. 버퍼 크기를 무한히 키우는 것이 해결책이 아닌 이유 서술

체크 질문

- DMA는 CPU를 완전히 사용하지 않는다는 뜻인가?
- Buffer가 크면 Drop은 줄지만 무엇이 늘어날 수 있는가?
- Interrupt가 너무 자주 발생하면 어떤 문제가 생길까?

파일: `os_24h_digital_worksheet.html`

Chapter 08. 시스템 콜과 프로그램 실행

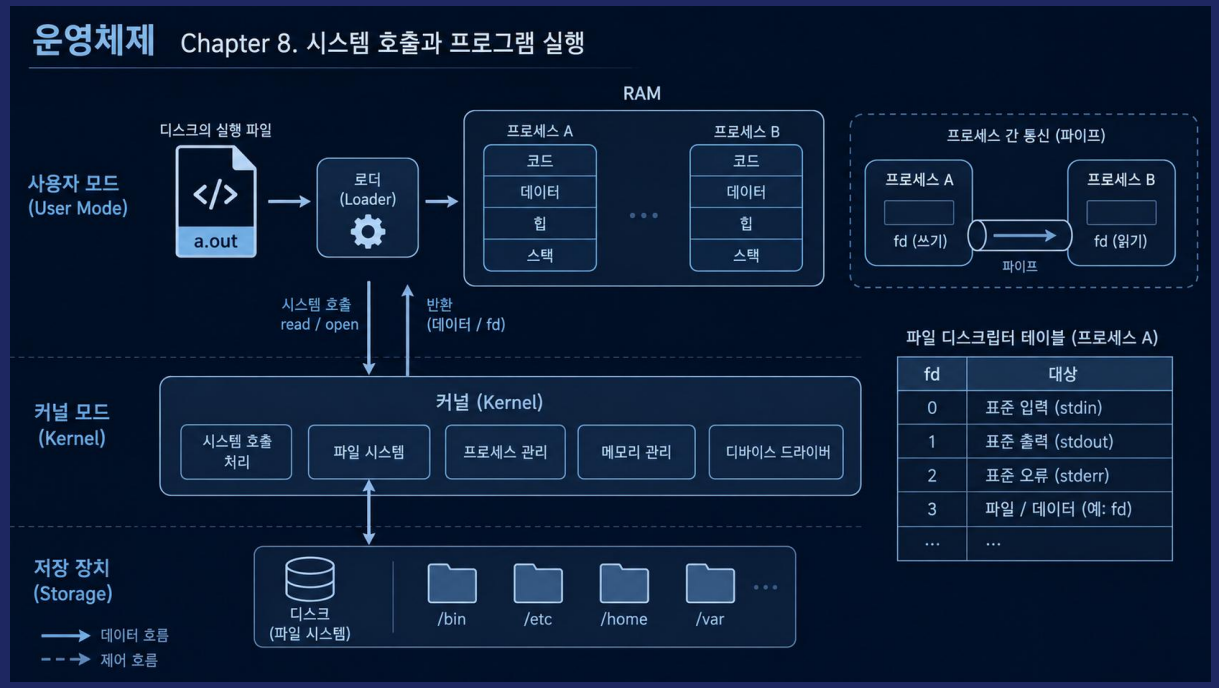
Loader · System Call · File Descriptor · Pipe · strace

학습 목표

1. 실행 파일이 메모리에 로드되어 실행되는 흐름을 설명한다.
2. System Call이 User Mode와 Kernel Mode를 연결하는 이유를 설명한다.
3. File Descriptor와 Pipe를 이용한 I/O 추상화를 이해한다.

시작 질문

왜 모든 함수 호출이 System Call은 아닌가?



시스템 콜과 프로그램 실행 · 그림 전체 보기

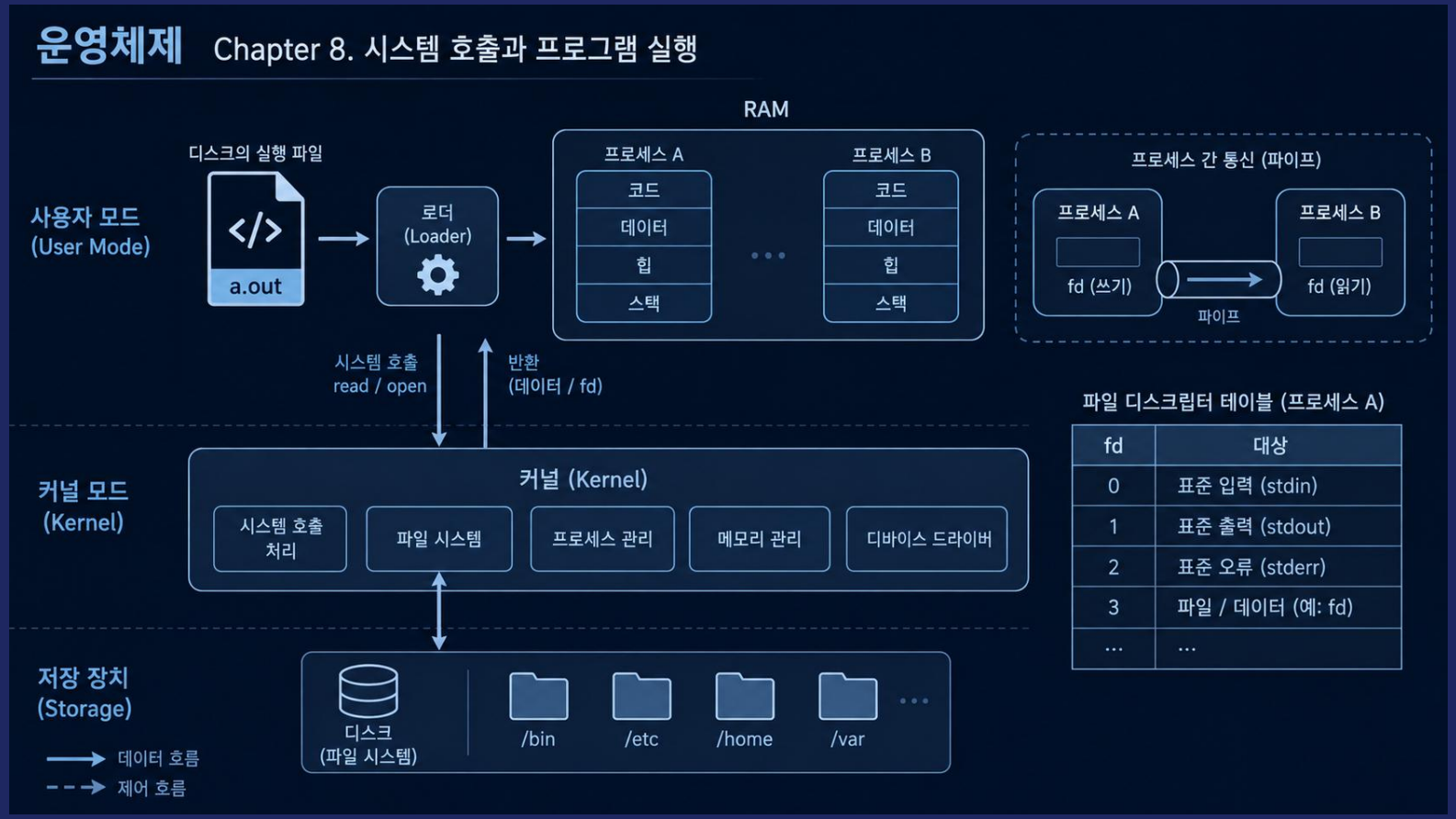


그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

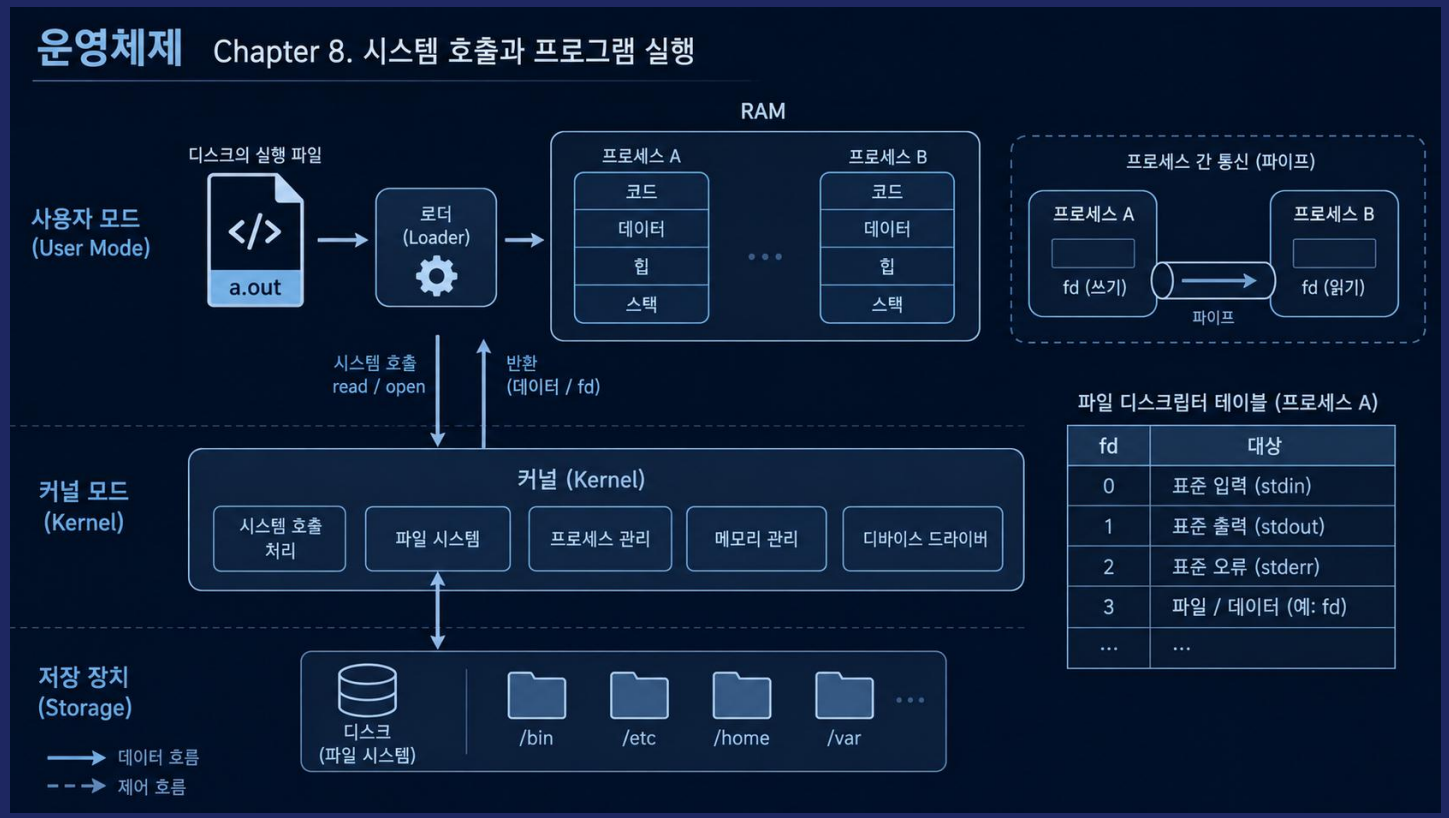
설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) Loader
2) System Call
3) File Descriptor
4) Pipe

시스템 콜과 프로그램 실행 · 단계별 그림 해설 ①



1단계 해설

① Loader

실행 파일과 필요한 라이브러리를 주소 공간에 배치

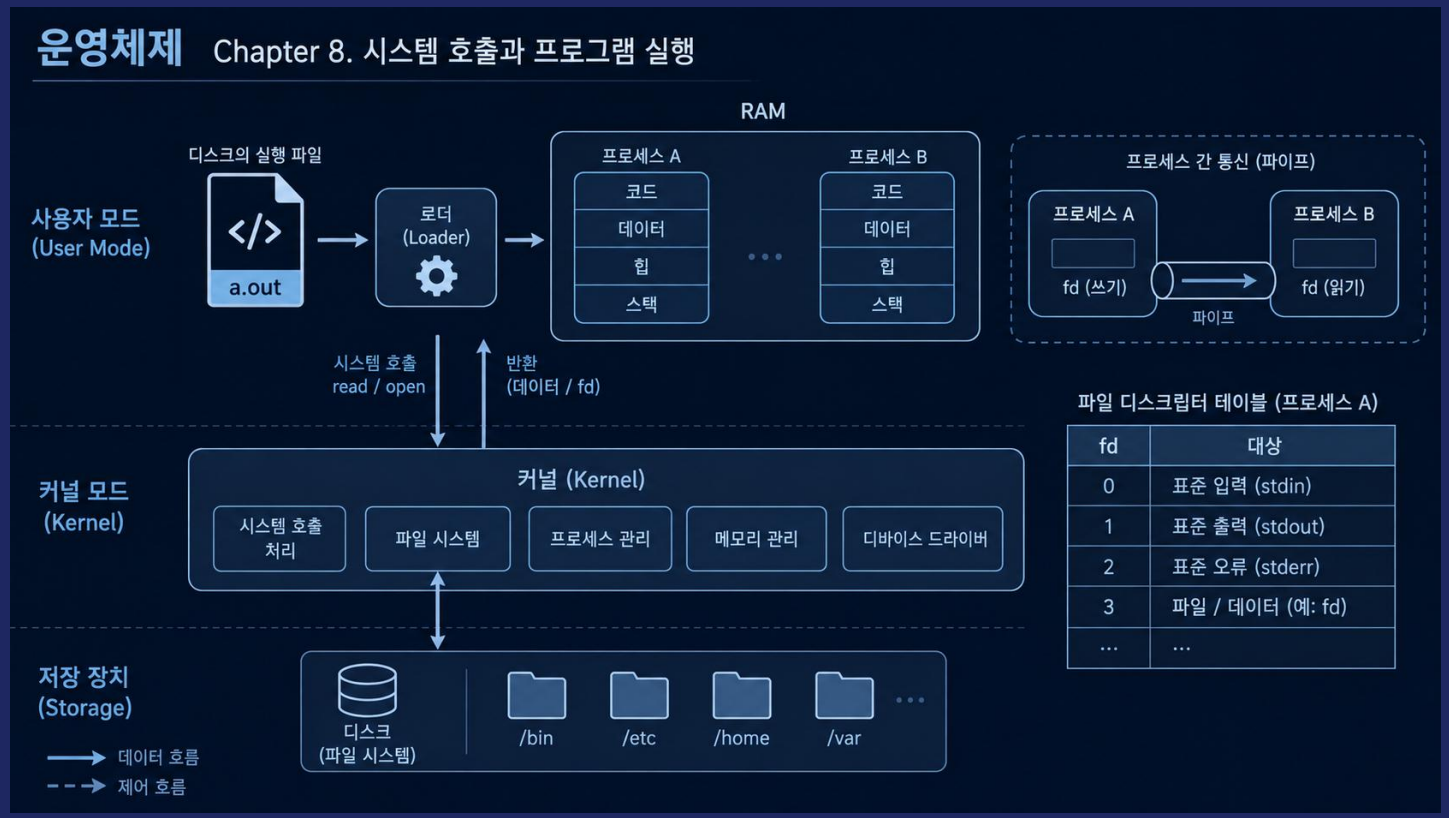
② System Call

User 프로그램이 Kernel 기능을 요청하는 공식 진입점

설명 포인트

- Loader가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 System Call가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

시스템 콜과 프로그램 실행 · 단계별 그림 해설 ②



파일 디스크립터 테이블 (프로세스 A)

fd	대상
0	표준 입력 (stdin)
1	표준 출력 (stdout)
2	표준 오류 (stderr)
3	파일 / 데이터 (예: fd)
…	…

2단계 해설

③ File Descriptor

파일·소켓·파이프 등 I/O 객체를 정수 번호로 추상화

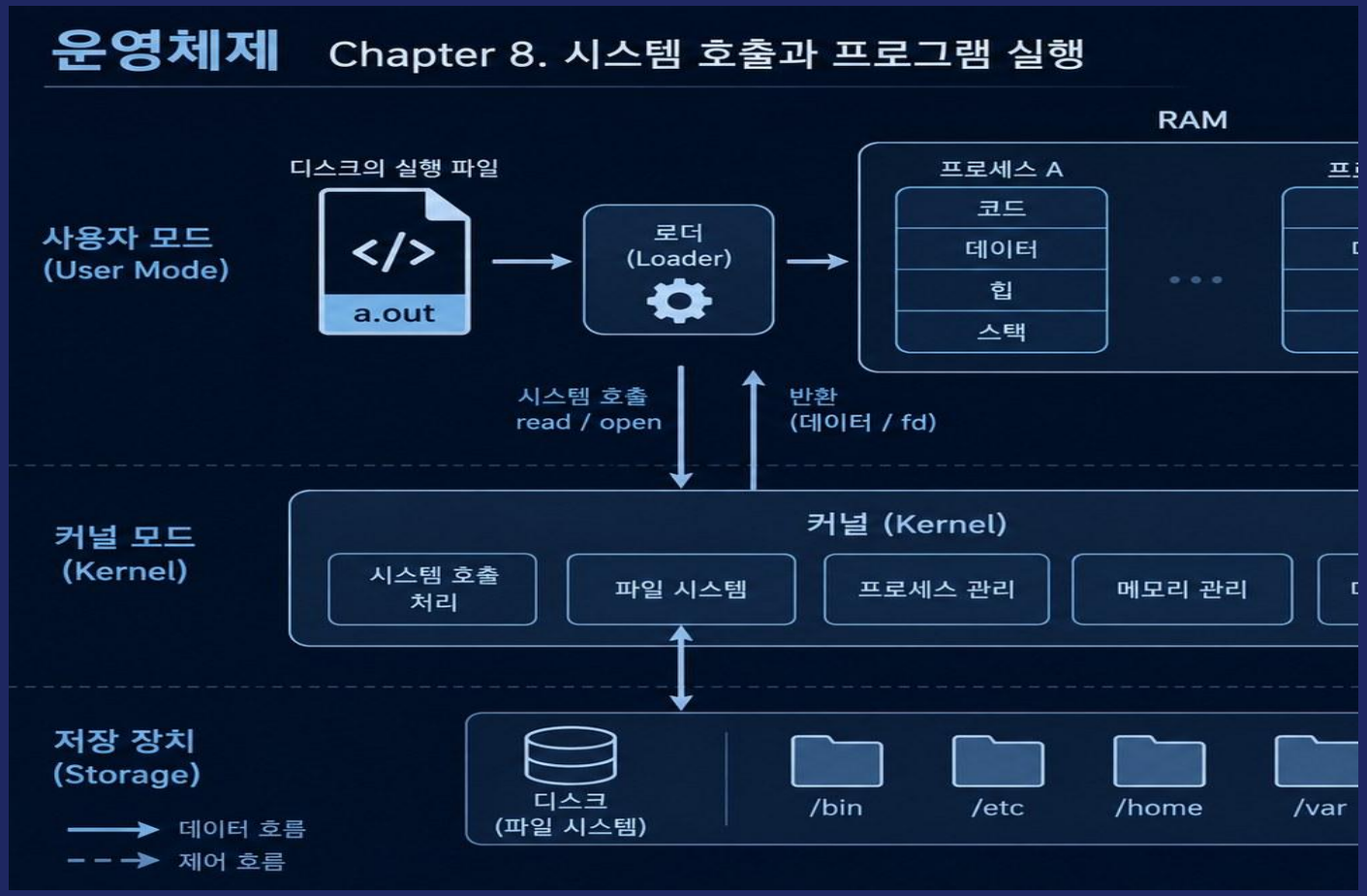
④ Pipe

한 프로세스의 출력 스트림을 다른 프로세스 입력으로 연결

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- File Descriptor와 Pipe는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

시스템 콜과 프로그램 실행 · 복잡한 그림 세부 해설 1





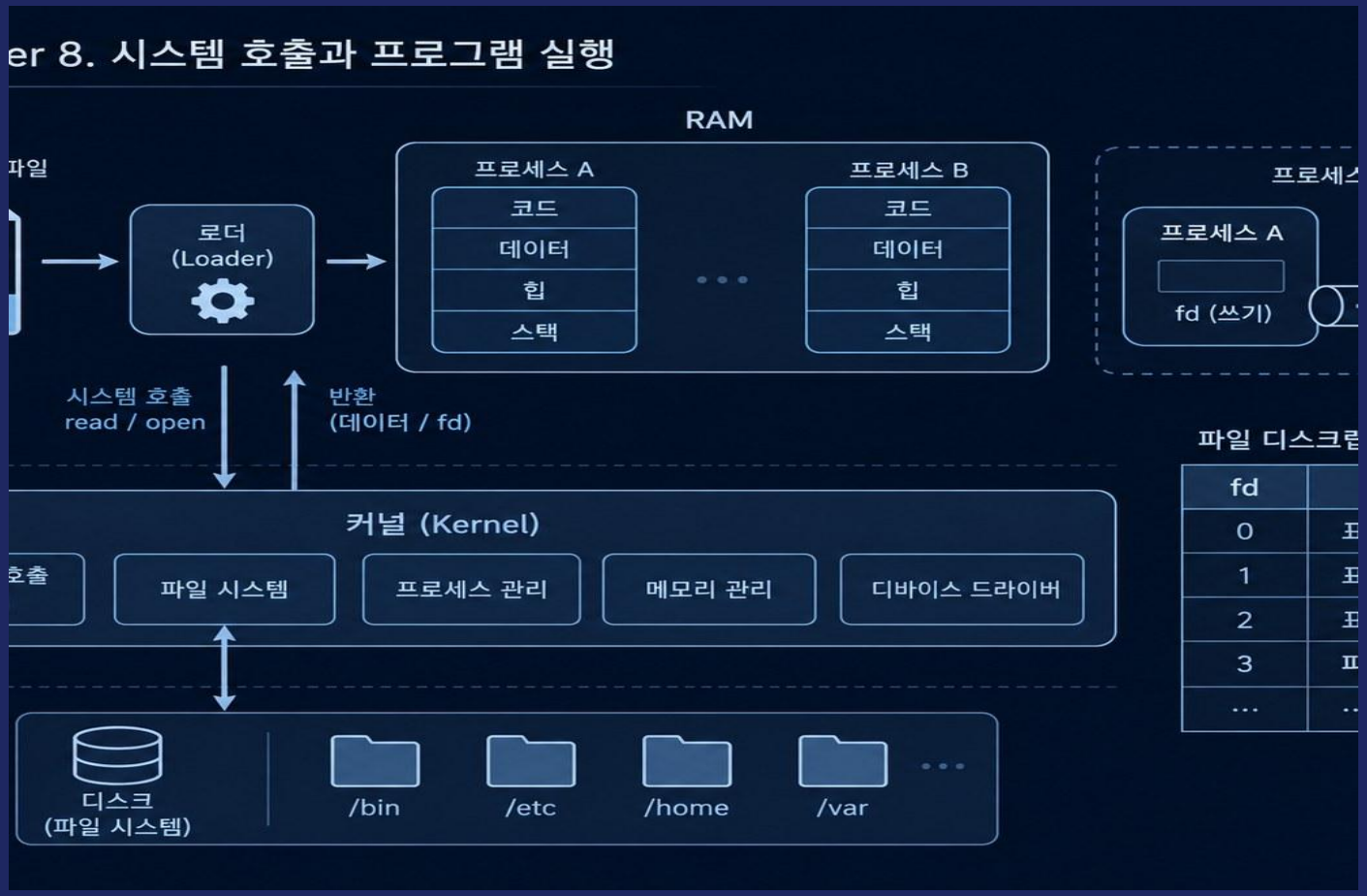
확대해서 볼 부분

1. Loader

실행 파일과 라이브러리를 프로세스 주소 공간에 매핑합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

시스템 콜과 프로그램 실행 · 복잡한 그림 세부 해설 2





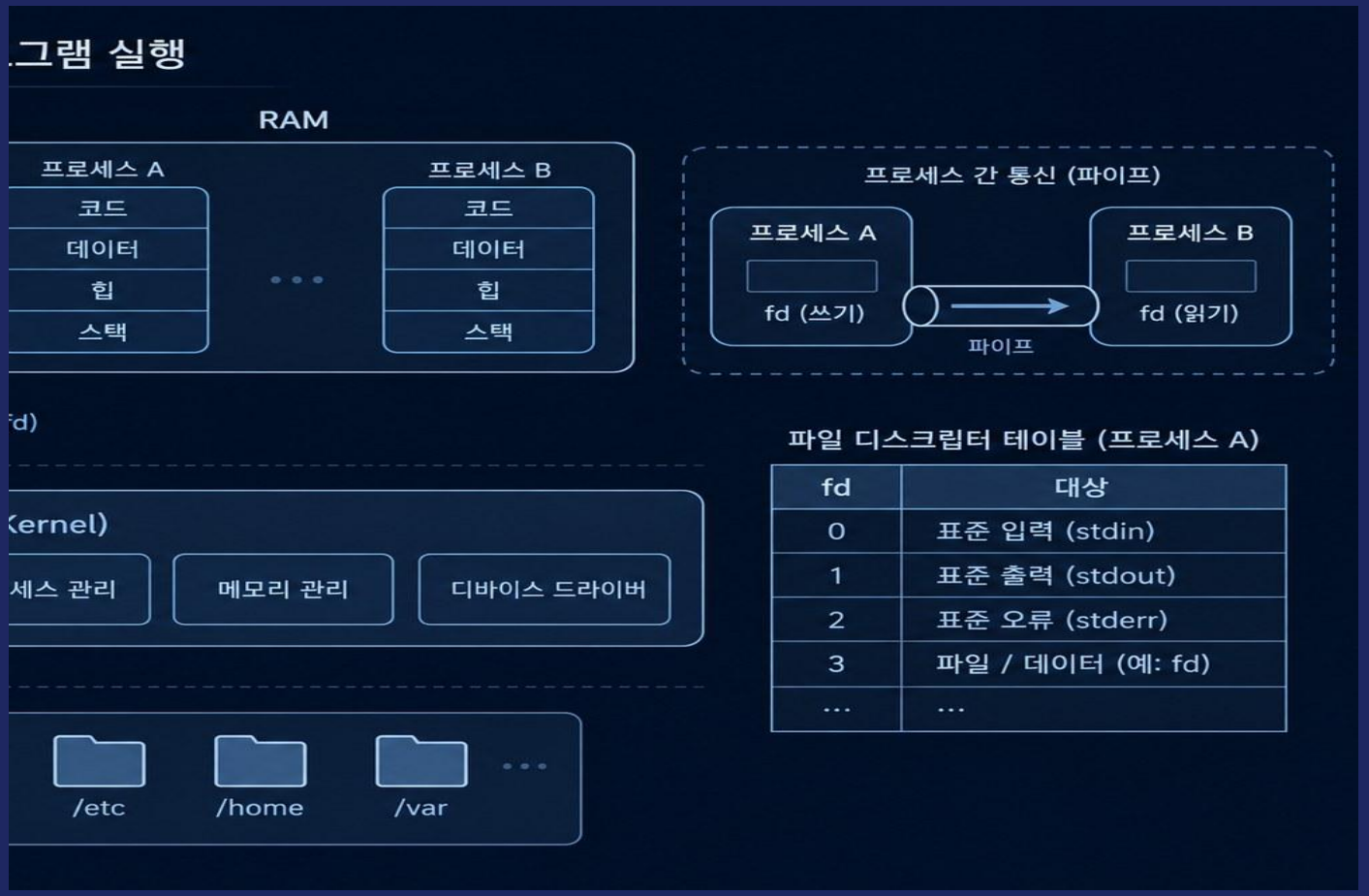
확대해서 볼 부분

2. System Call

User 프로그램이 Kernel 기능을 호출하는 공식 진입점입니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

시스템 콜과 프로그램 실행 · 복잡한 그림 세부 해설 3





확대해서 볼 부분

3. File Descriptor

파일·Socket·Pipe를 정수 번호로 동일하게 다루는 I/O 추상화입니다.

4. Pipe / strace

프로세스 간 스트림을 연결하고 strace로 실제 System Call 순서를 관찰합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

시스템 콜과 프로그램 실행 · 동작 워크플로우

개념을 “정의”가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.



이 워크플로우에서 반드시 관찰할 3가지

- Library Call과 System Call은 같은 것이 아님
- fd는 파일뿐 아니라 Socket/Pipe에도 사용
- strace는 보이지 않는 Kernel 진입을 관찰하게 해줌

강사 멈춤 질문

프로그램 “실행”은 언제 시작되었다고 볼 수 있을까?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

시스템 콜과 프로그램 실행 · 워크플로우 전반부 단계 해설



1	**Binary 선택**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**Loader/execve**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**User Mode**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

시스템 콜과 프로그램 실행 · 워크플로우 후반부 단계 해설



4	System Call	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	Kernel I/O	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	Return/User Buffer	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

read("data.bin") 한 줄 아래에서 실제로 벌어지는 일

SCENARIO

read("data.bin") 한 줄 아래에서 실제로 벌어지는 일

증상만 보고 바로 조치하지 말고
OS 상태 변화를 먼저 추적한다.

1 STEP 1

Library 함수가 System Call을 준비한다.

2 STEP 2

CPU가 Kernel Mode로 전환되고 fd가 유효한지 확인한다.

3 STEP 3

Filesystem/Driver가 Storage에서 데이터를 읽는다.

4 STEP 4

Kernel Buffer에서 User Buffer로 데이터가 전달되고 User Mode로 복귀한다.

▶ ANIMATION 준비 구간

시스템 콜과 프로그램 실행 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
Loader 흐름	실행 + read()	Binary→Loader→RAM→CPU→Kernel로 강조 이동	execve/openat/read 로그	프로그램 "실행"은 언제 시작되었다고 볼 수 있을까?
FD Table	open()/close() 반복	fd 3,4,5가 생성·반환	fd 번호, 객체 수	왜 stdin/stdout/stderr가 0/1/2인가?
Pipe	pipe write	P1→Pipe→P2로 데이터가 이동	Pipe 양 끝, read/write	Pipe와 파일이 같은 fd 인터페이스를 쓰는 장점은?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 08 · 시스템 콜과 프로그램 실행

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=8`

수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

인터랙티브 시뮬레이션 · Program Loader + System Call Trace + FD/Pipe Lab

▶ 지금 ANIMATION 수행

HTML SIMULATOR

Program Loader + System Call Trace + FD/Pipe Lab



1. 실행+read() 버튼으로 Binary→Loader→RAM→CPU→Kernel→Storage 흐름 관찰
2. strace 로그에서 execve/openat/read/close 순서 확인
3. open()을 반복해 fd 3,4,5… 할당 확인
4. Pipe write로 P1→Pipe→P2 데이터 흐름 확인

시연 전에 반드시 질문

"버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
."

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 08 · 시스템 콜과 프로그램 실행

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. execve→open→read 순서 관찰
2. fd 0/1/2와 새 fd 할당 확인
3. pipe() 데이터 이동과 syscall trace 비교

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 08 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. stdin/stdout/stderr fd 0/1/2 확인
2. Library Call과 System Call 차이 서술
3. strace 결과에서 파일 접근과 오류를 찾는 연습

체크 질문

- 왜 모든 함수 호출이 System Call은 아닌가?
- fd 3이 항상 특정 파일을 의미하지 않는 이유는?
- Pipe와 일반 파일의 공통점은 무엇인가?

파일: `os_24h_digital_worksheet.html`

DAY 3

운영·네트워크·온디바이스 AI

실제 서비스 운영과 장애 진단까지 OS 지식을 통합합니다.

C09 부팅과 서비스
C10 네트워크
C11 온디바이스 AI 자원 관리
C12 종합 장애 분석



Chapter 09. 서비스와 부팅

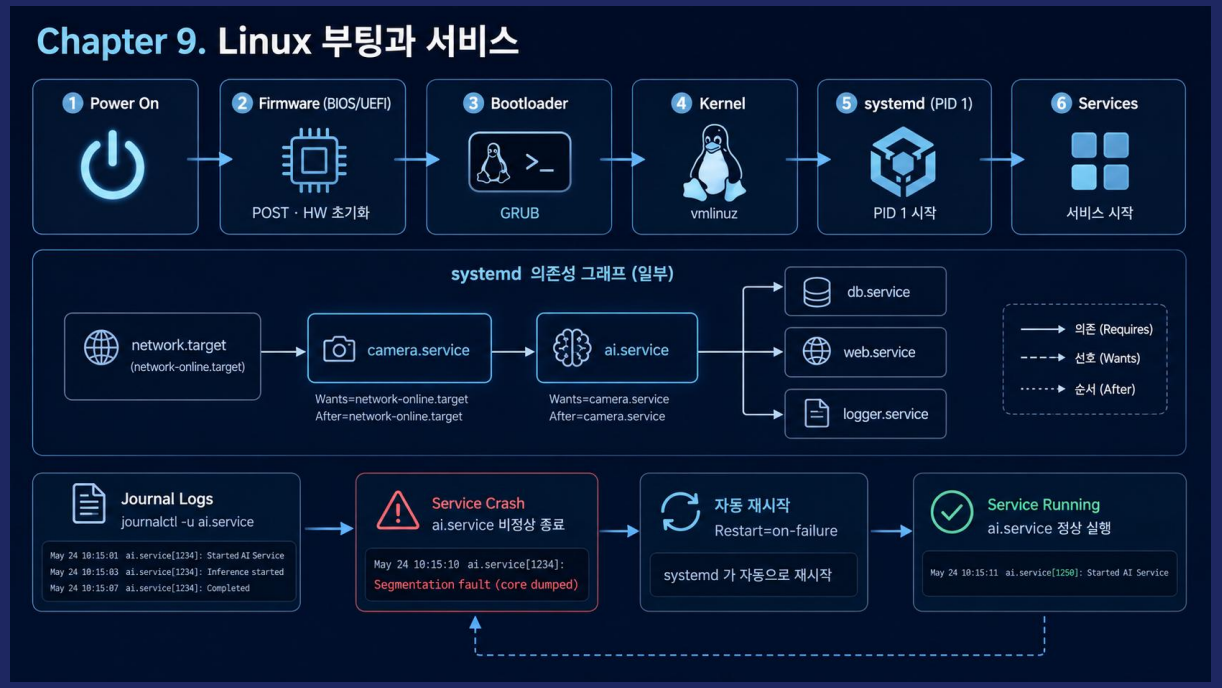
Firmware · Bootloader · Kernel · systemd · Dependency · journalctl

학습 목표

1. Linux 부팅 순서를 설명한다.
2. systemd와 서비스 의존성을 이해한다.
3. 서비스 Crash와 Auto Restart, journal 로그를 이용해 장애를 추적한다.

시작 질문

systemd가 PID 1이라는 사실이 왜 중요한가?



서비스와 부팅 · 그림 전체 보기



그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

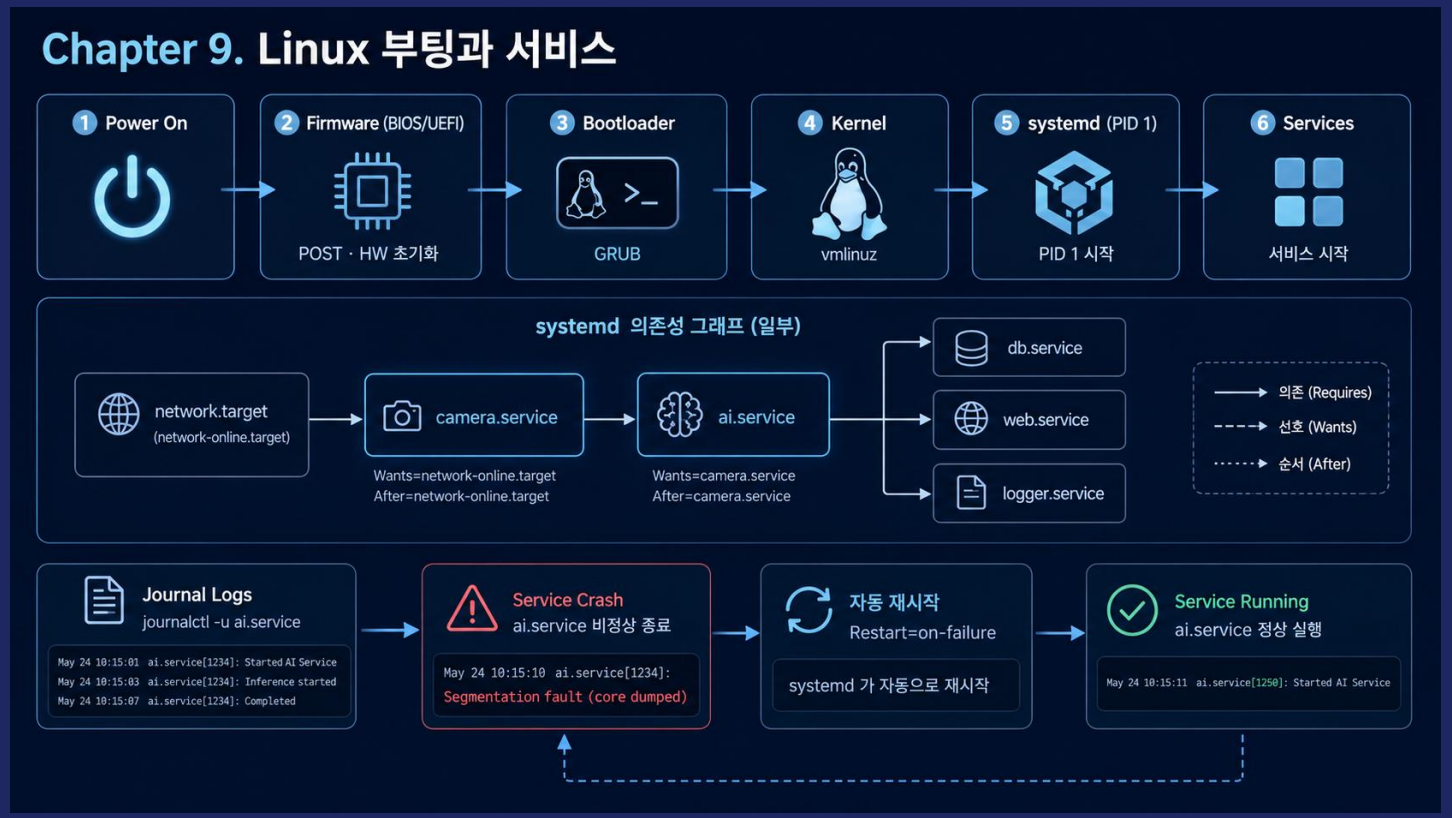
설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) Boot Sequence
2) systemd
3) Dependency
4) journal

서비스와 부팅 · 단계별 그림 해설 ①



1단계 해설

① Boot Sequence

Firmware→Bootloader→Kernel→systemd(PID 1)→Services

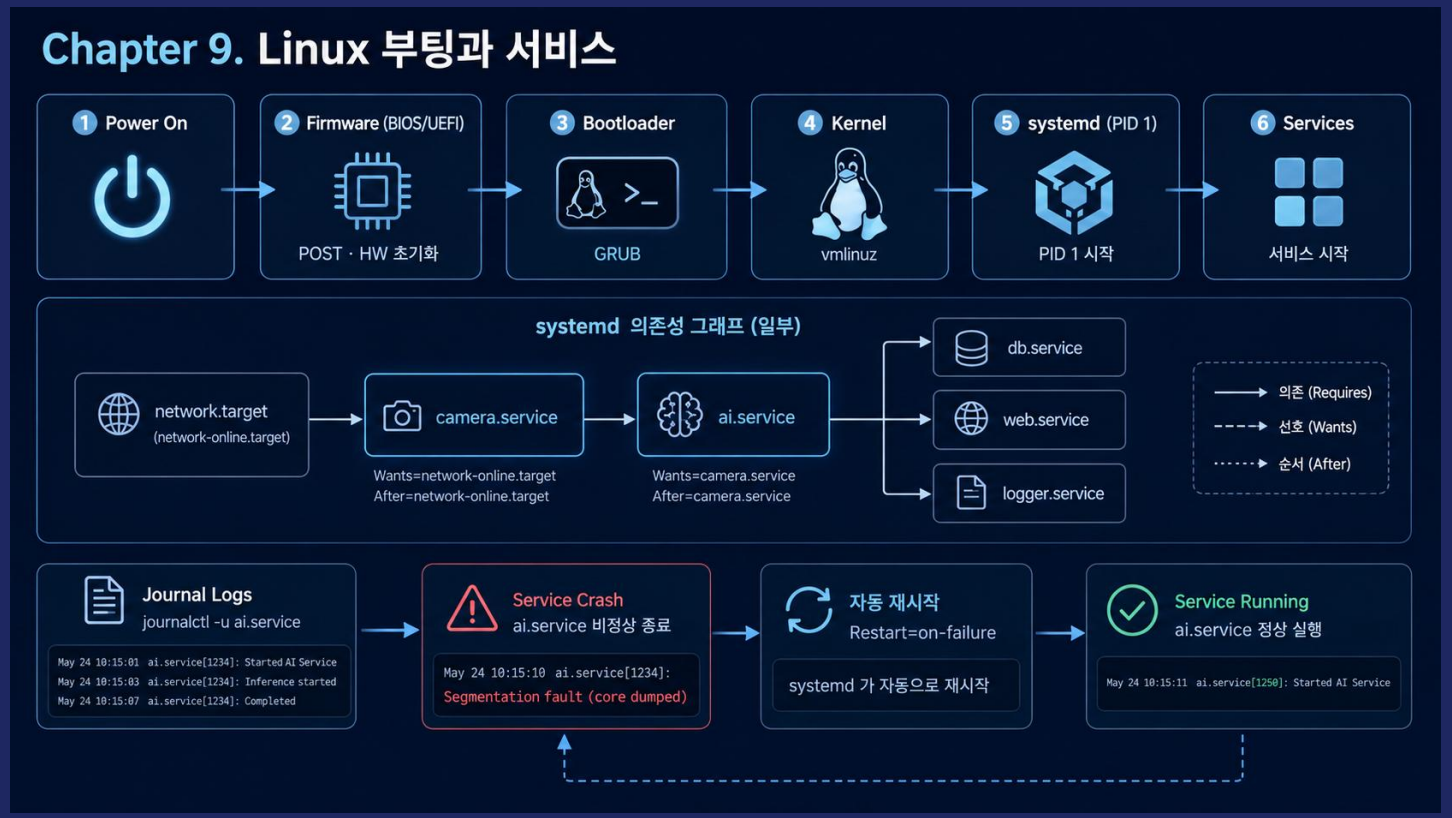
② systemd

서비스 시작/중지, 의존성, Restart 정책, 로그 연계

설명 포인트

- Boot Sequence가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 systemd가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

서비스와 부팅 · 단계별 그림 해설 ②



2단계 해설

③ Dependency

선행 자원/서비스가 준비돼야 후속 서비스가 정상 시작

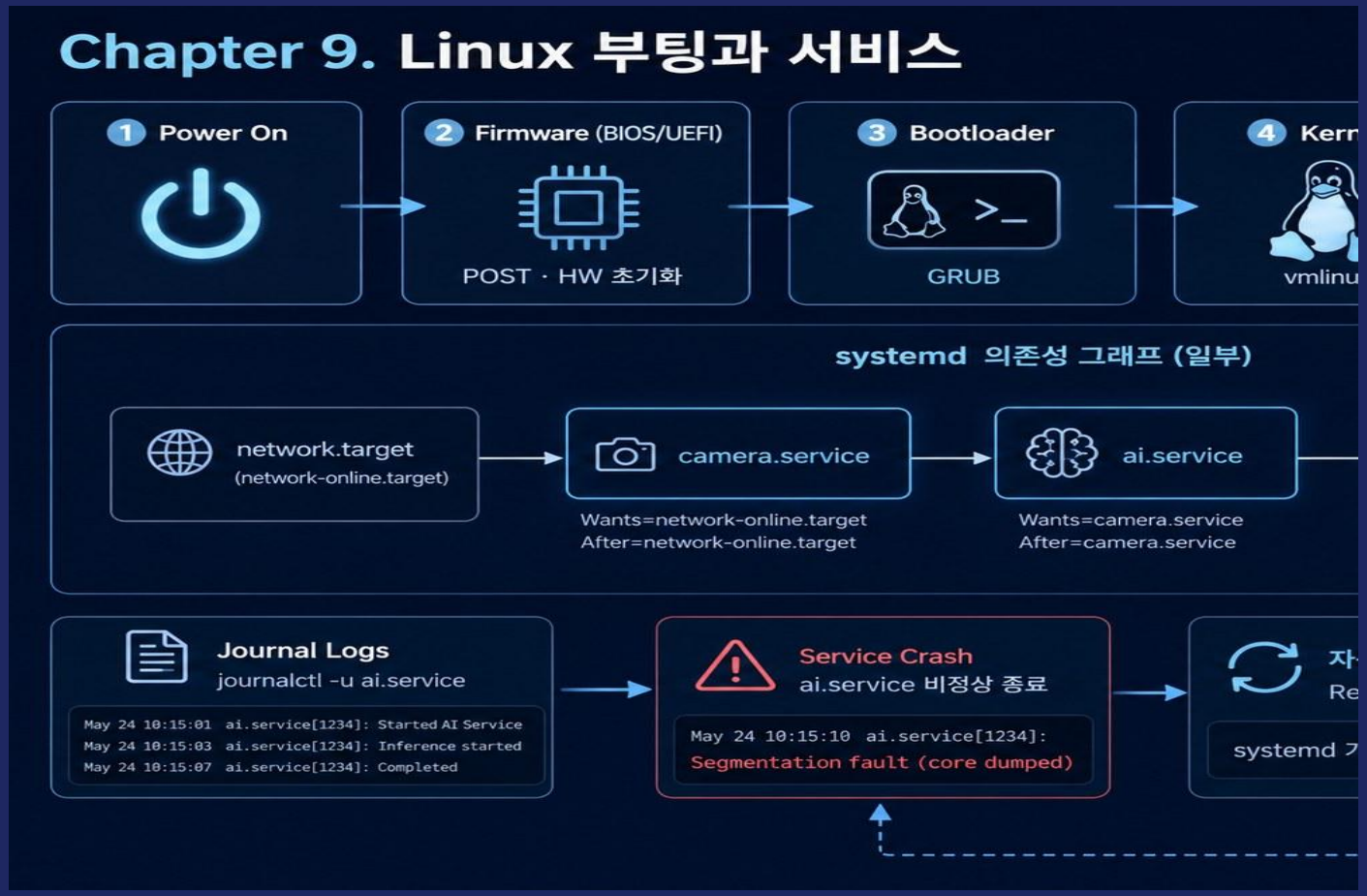
④ journal

서비스 상태 변화와 오류를 시간순으로 추적하는 핵심 증거

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- Dependency와 journal는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

부팅과 서비스 · 복잡한 그림 세부 해설 1





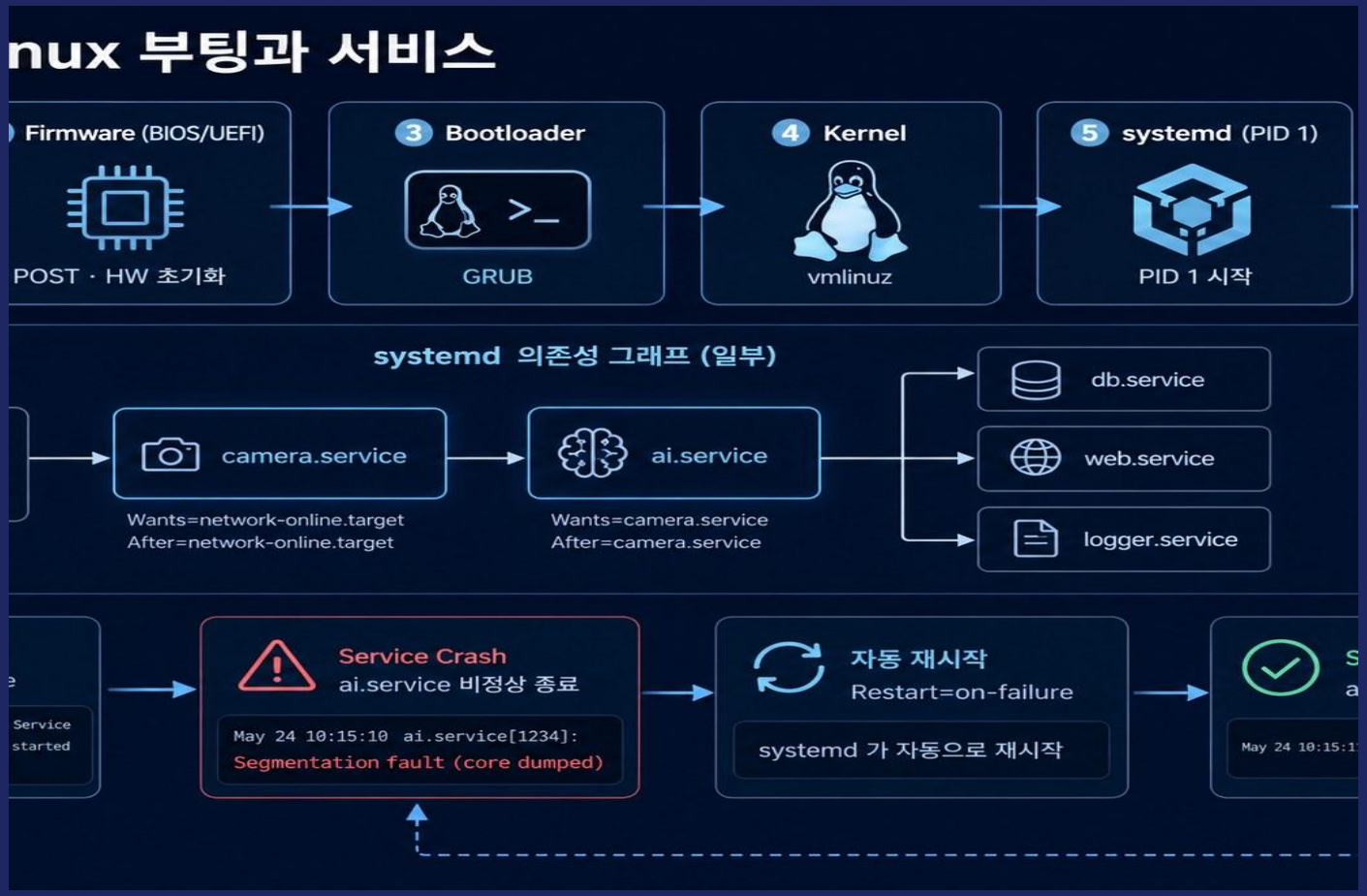
확대해서 볼 부분

1. Boot Sequence

Firmware→Bootloader→Kernel→systemd→Services 순으로 시스템이 준비됩니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

부팅과 서비스 · 복잡한 그림 세부 해설 2



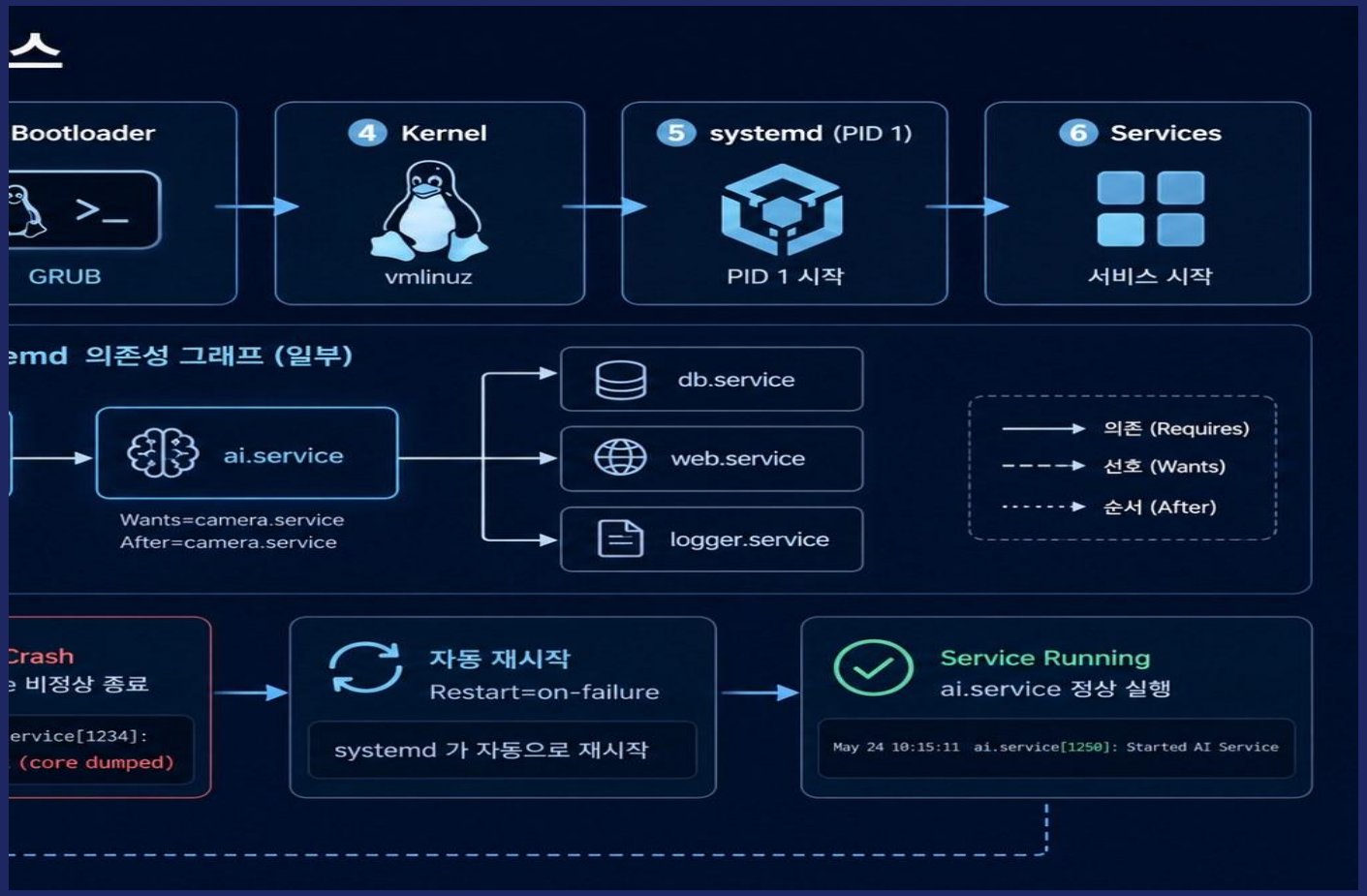


확대해서 볼 부분

2. systemd
PID 1에서 서비스 상태·의존성·Restart 정책을 관리합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

부팅과 서비스 · 복잡한 그림 세부 해설 3





확대해서 볼 부분

3. Dependency
앞선 자원이나 서비스가 준비되지 않으면 후속 서비스가 실패할 수 있습니다.

4. journal
부팅·서비스 상태 변화와 오류를 시간순으로 남기는 핵심 증거입니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

서비스와 부팅 · 동작 워크플로우

개념을 "정의"가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.



이 워크플로우에서 반드시 관찰할 3가지

- 부팅 성공과 서비스 준비 완료는 다른 시점
- Dependency 실패가 연쇄 실패를 만들 수 있음
- Crash 원인과 Restart 정책은 로그와 함께 봐야 함

강사 멈춤 질문

Kernel이 올라온 뒤에도 왜 기다려야 하는가?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

부팅과 서비스 · 워크플로우 전반부 단계 해설



1	**Firmware**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**Bootloader**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**Kernel**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

부팅과 서비스 · 워크플로우 후반부 단계 해설



4	systemd	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	Dependency	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	Service Active	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

camera.service가 부팅 때마다 실패하지만 수동 재시작하면 성공한다

SCENARIO

camera.service가 부팅 때마다 실패하지만 수동 재시작하면 성공한다

증상만 보고 바로 조치하지 말고 OS 상태 변화를 먼저 추적한다.

1 STEP 1

장치 초기화 또는 network 등 선행 자원이 늦게 준비될 수 있다.

2 STEP 2

서비스의 After/Requires/Wants 관계를 확인한다.

3 STEP 3

journalctl로 부팅 시점의 실제 오류를 확인한다.

4 STEP 4

단순 sleep보다 올바른 Dependency와 Restart 정책을 설계하는 것이 바람직하다.

▶ ANIMATION 준비 구간

서비스와 부팅 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
정상 부팅	network 정상 + Power ON	단계별 노드가 순차 활성화	서비스 상태	Kernel이 올라온 뒤에도 왜 기다려야 하는가?
Dependency 실패	network OFF 후 부팅	후속 camera/ai 서비스가 실패 표시	dependency chain	sleep을 넣는 것보다 의존성 정의가 나은 이유는?
Crash/Restart	서비스 Crash	failed→restart→running 또는 failed 유지	Restart count, journal	자동 재시작이 근본 해결이 아닐 수 있는 이유는?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 09 · 부팅과 서비스

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=9`

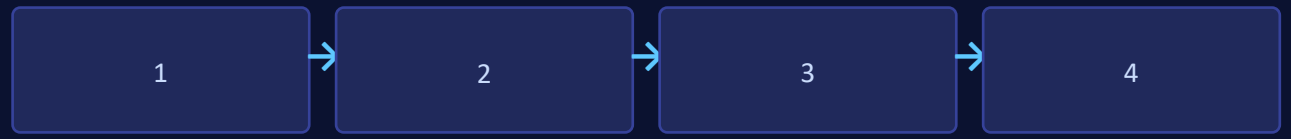
수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

인터랙티브 시뮬레이션 · Linux Boot Dependency + Service Crash/journal Lab

▶ 지금 ANIMATION 수행

HTML SIMULATOR

Linux Boot Dependency + Service Crash/journal Lab



1. network 정상 ON 상태로 Power ON → 전체 부팅 성공 확인
2. network를 끄고 다시 부팅해 후속 서비스 실패 확인
3. camera.service Start 후 Crash 실행
4. Auto Restart ON/OFF에 따른 상태와 로그 차이 확인

시연 전에 반드시 질문

"버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
."

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 09 · 부팅과 서비스

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. 정상 Boot와 Dependency Fail 비교
2. Service Crash/Auto Restart 관찰
3. journal 로그로 실패 원인 설명

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 09 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. 부팅 순서 카드 배열
2. 서비스 반복 Restart에서 먼저 확인할 로그/정책 선택
3. Dependency를 “먼저 준비되어야 하는 것” 관점으로 한 문장 설명

체크 질문

- systemd가 PID 1이라는 사실이 왜 중요한가?
- Restart=always가 모든 장애의 해결책이 아닌 이유는?
- 서비스가 실패했을 때 로그와 상태 중 무엇을 먼저 볼까?

파일: `os_24h_digital_worksheet.html`

Chapter 10. 네트워크와 OS

Socket · IP/Port · TCP · UDP · Packet Loss · Bind

학습 목표

1. Socket이 프로세스와 네트워크를 연결하는 Endpoint임을 설명한다.
2. TCP Handshake와 재전송 흐름을 이해한다.
3. Port Bind 충돌과 연결 장애를 OS 관점에서 진단한다.

시작 질문

IP가 같아도 여러 서비스가 동시에 가능한 이유는?



네트워크와 OS · 그림 전체 보기



그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) Socket
2) IP + Port
3) TCP
4) UDP

네트워크와 OS · 단계별 그림 해설 ①



1단계 해설

① **Socket**

프로세스가 네트워크를 사용하는 OS 객체

② **IP + Port**

호스트 위치와 호스트 내부 서비스 종점을 식별

설명 포인트

- Socket가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 IP + Port가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

네트워크와 OS · 단계별 그림 해설 ②



2단계 해설

③ TCP

연결 설정·순서 보장·ACK·재전송 등 신뢰성 제공

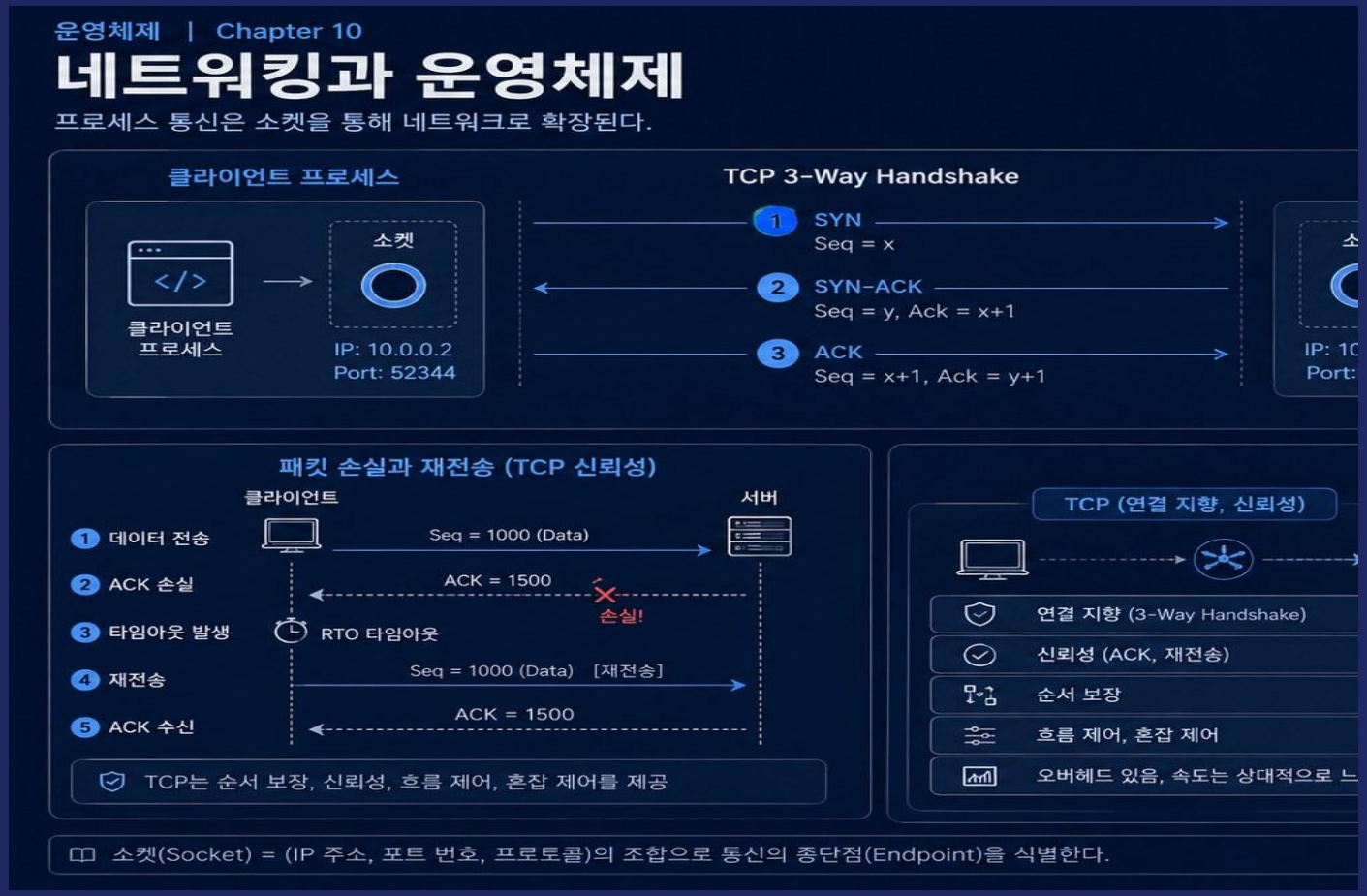
④ UDP

연결 상태를 최소화하여 단순하고 빠르지만 전달 보장 없음

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- TCP와 UDP는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

네트워크와 OS · 복잡한 그림 세부 해설 1





확대해서 볼 부분

1. Socket
프로세스와 네트워크를 연결하는 OS 객체입니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

네트워크와 OS · 복잡한 그림 세부 해설 2





확대해서 볼 부분

2. Bind / Listen

IP:Port를 Socket에 연결하고 서버가 연결을 받을 준비를 합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

네트워크와 OS · 복잡한 그림 세부 해설 3





확대해서 볼 부분

3. TCP 상태

SYN→SYN/ACK→ACK 후 Data/ACK와 Close 상태가 이어집니다.

4. Loss / Retransmit

손실이 생기면 TCP는 재전송으로 신뢰성을 유지하지만 지연과 처리량 비용이 증가합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

네트워크와 OS · 동작 워크플로우

개념을 "정의"가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.

1. **socket()** — 프로세스가 Socket 객체 생성
2. **bind / listen** — 서버가 IP:Port에 Endpoint 등록
3. **SYN** — 클라이언트가 연결 요청
4. **SYN-ACK / ACK** — 3-Way Handshake로 상태 생성
5. **Data + ACK / Retransmit** — 데이터 전송·ACK·손실 시 재전송
6. **FIN / Close** — 연결 종료 후 Socket 상태 정리

이 워크플로우에서 반드시 관찰할 3가지

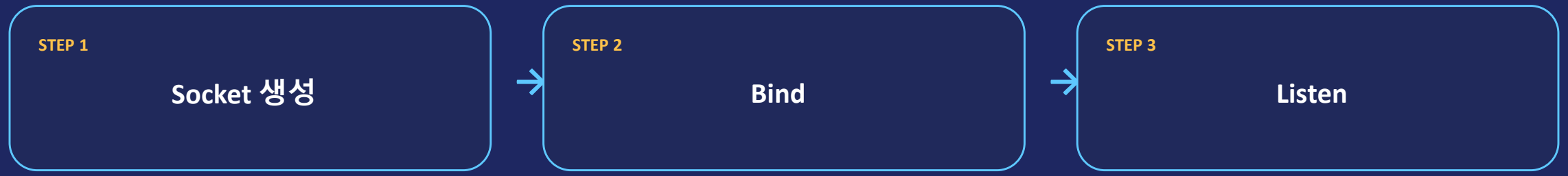
- Process가 살아 있어도 LISTEN이 아닐 수 있음
- Packet Loss는 TCP 재전송과 Latency 증가로 이어짐
- 같은 Port Bind 충돌은 OS가 차단

강사 멈춤 질문

3-Way Handshake가 필요한 이유는?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

네트워크와 OS · 워크플로우 전반부 단계 해설



1	**Socket 생성**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**Bind**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**Listen**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

네트워크와 OS · 워크플로우 후반부 단계 해설



4	**Handshake**	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	**Data/ACK**	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	**Close**	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

서버 프로세스는 살아 있는데 클라이언트가 접속하지 못한다

SCENARIO

서버 프로세스는 살아 있는데 클라이언트가 접속하지 못한다

증상만 보고 바로 조치하지 말고 OS 상태 변화를 먼저 추적한다.

1 STEP 1

프로세스가 살아 있는 것과 Port가 LISTEN 상태인 것은 다르다.

2 STEP 2

ss로 실제 Bind 주소와 Port를 확인한다.

3 STEP 3

동일 IP:Port를 다른 프로세스가 이미 사용하면 EADDRINUSE가 발생한다.

4 STEP 4

패킷이 오가지만 손실이 크면 TCP 재전송과 지연이 증가한다.

▶ ANIMATION 준비 구간

네트워크와 OS · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
정상 TCP	Loss 0% 세션 실행	SYN→SYN/ACK→ACK→Data→FIN 흐름	연결 상태, ACK	3-Way Handshake가 필요한 이유는?
Packet Loss	Loss 30~50%	일부 패킷이 실패 표시되고 재전송	loss, retransmit, latency	신뢰성이 공짜가 아닌 이유는?
Port 충돌	P1 8080 bind 후 P2 bind	P1은 LISTEN, P2는 address in use 오류	Port owner	프로세스가 살아 있다는 것과 서비스가 열려 있다는 것은 어떻게 다른가

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 10 · 네트워크와 OS

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=10`

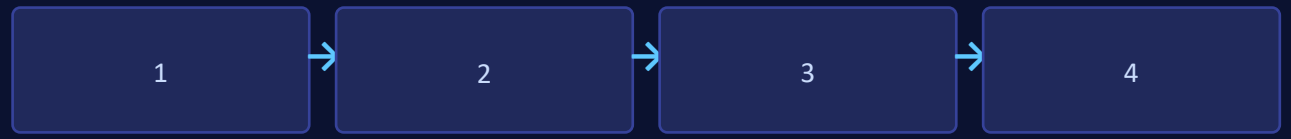
수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

▶ 지금 ANIMATION 수행

인터랙티브 시뮬레이션 · TCP Session + Socket/Port Conflict Lab

HTML SIMULATOR

TCP Session + Socket/Port Conflict Lab



1. Loss 0%에서 SYN→SYN/ACK→ACK→Data→FIN 흐름 확인
2. Loss를 30% 이상으로 올리고 재전송 개수 비교
3. P1이 8080 Bind 후 P2가 같은 Port Bind 시도
4. Port 번호를 바꿔 정상 LISTEN 상태로 복구

시연 전에 반드시 질문

"버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
."

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 10 · 네트워크와 OS

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. 정상 TCP와 Packet Loss 비교
2. Port Conflict 재현
3. UDP와 TCP의 상태 관리 차이 설명

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 10 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. TCP 3-Way Handshake 순서 작성
2. TCP vs UDP를 신뢰성/오버헤드 관점으로 비교
3. Address already in use 발생 원인 설명

체크 질문

- IP가 같아도 여러 서비스가 동시에 가능한 이유는?
- TCP 재전송은 신뢰성을 주지만 어떤 비용을 만든다?
- 0.0.0.0:8080과 127.0.0.1:8080은 의미가 어떻게 다른가?

파일: `os_24h_digital_worksheet.html`

Chapter 11. 온디바이스 AI 자원 관리

Camera · CPU/GPU/NPU · RAM · FPS · Latency · Thermal · Bottleneck

학습 목표

1. 온디바이스 AI 파이프라인의 자원 흐름을 설명한다.
2. FPS·Latency·사용률·Queue를 함께 해석해 Bottleneck을 찾는다.
3. Thermal Throttling과 자원 정책이 성능에 미치는 영향을 설명한다.

시작 질문

GPU가 100%라는 사실만으로 병목이라고 단정할 수 있을까?



온디바이스 AI 자원 관리 · 그림 전체 보기



그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) Pipeline
2) Throughput vs Latency
3) Bottleneck
4) Thermal

온디바이스 AI 자원 관리 · 단계별 그림 해설 ①



1단계 해설

① Pipeline

Camera→RAM Buffer→CPU Preprocess→GPU/NPU Inference→Postprocess/Result

② Throughput vs Latency

FPS는 초당 처리량, Latency는 한 입력 처리 시간

설명 포인트

- Pipeline가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 Throughput vs Latency가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

온디바이스 AI 자원 관리 · 단계별 그림 해설 ②



2단계 해설

③ Bottleneck

가장 느린 단계 앞에 Queue가 누적되고 뒤 단계 Utilization이 낮아질 수 있음

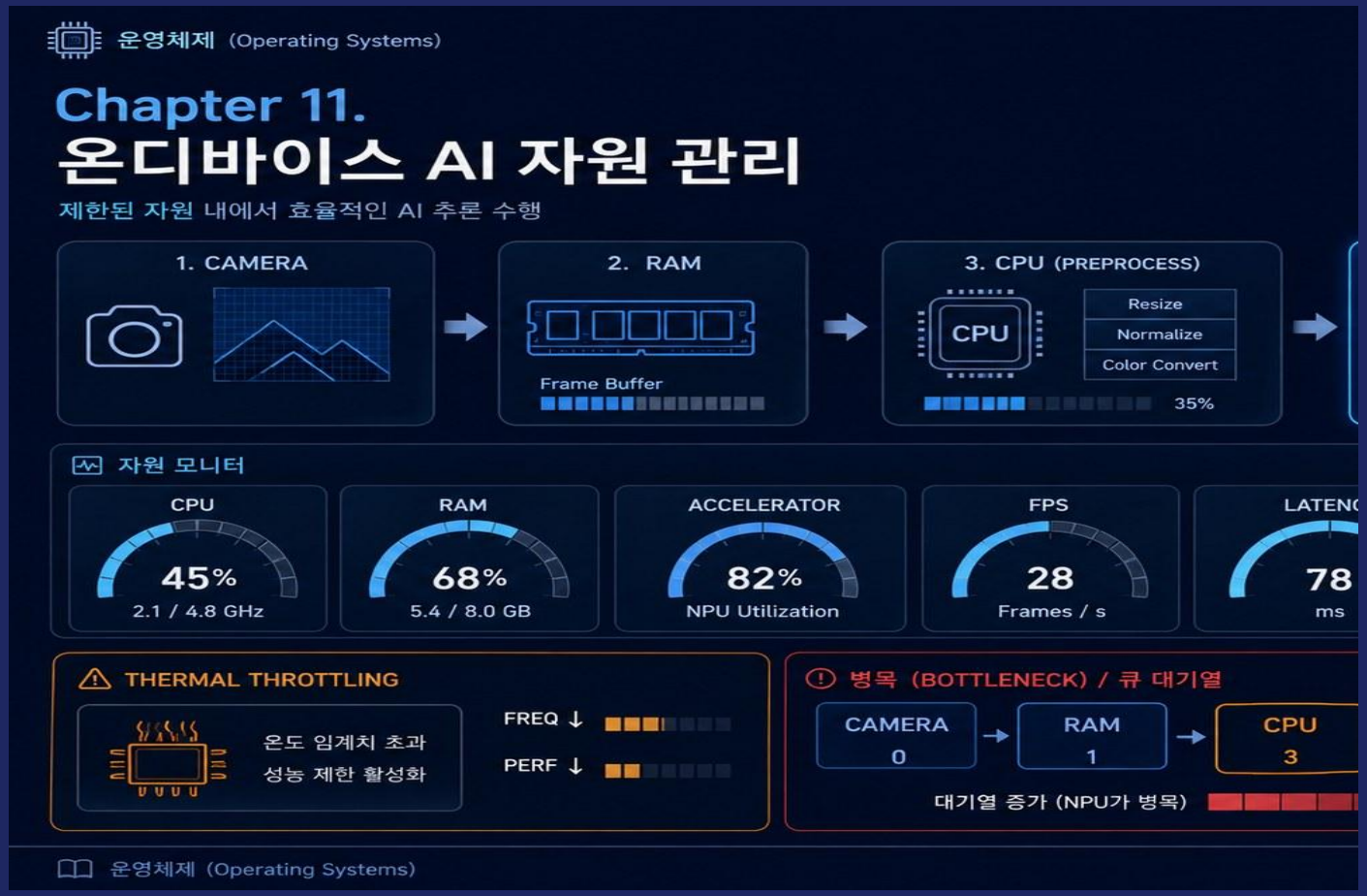
④ Thermal

온도 임계치 초과 시 Clock을 낮춰 성능과 전력을 제한

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- Bottleneck와 Thermal는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

온디바이스 AI 자원 관리 · 복잡한 그림 세부 해설 1





확대해서 볼 부분

1. AI Pipeline

Camera→Buffer→CPU 전처리→GPU/NPU 추론→후처리로 Frame이 이동합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

온디바이스 AI 자원 관리 · 복잡한 그림 세부 해설 2





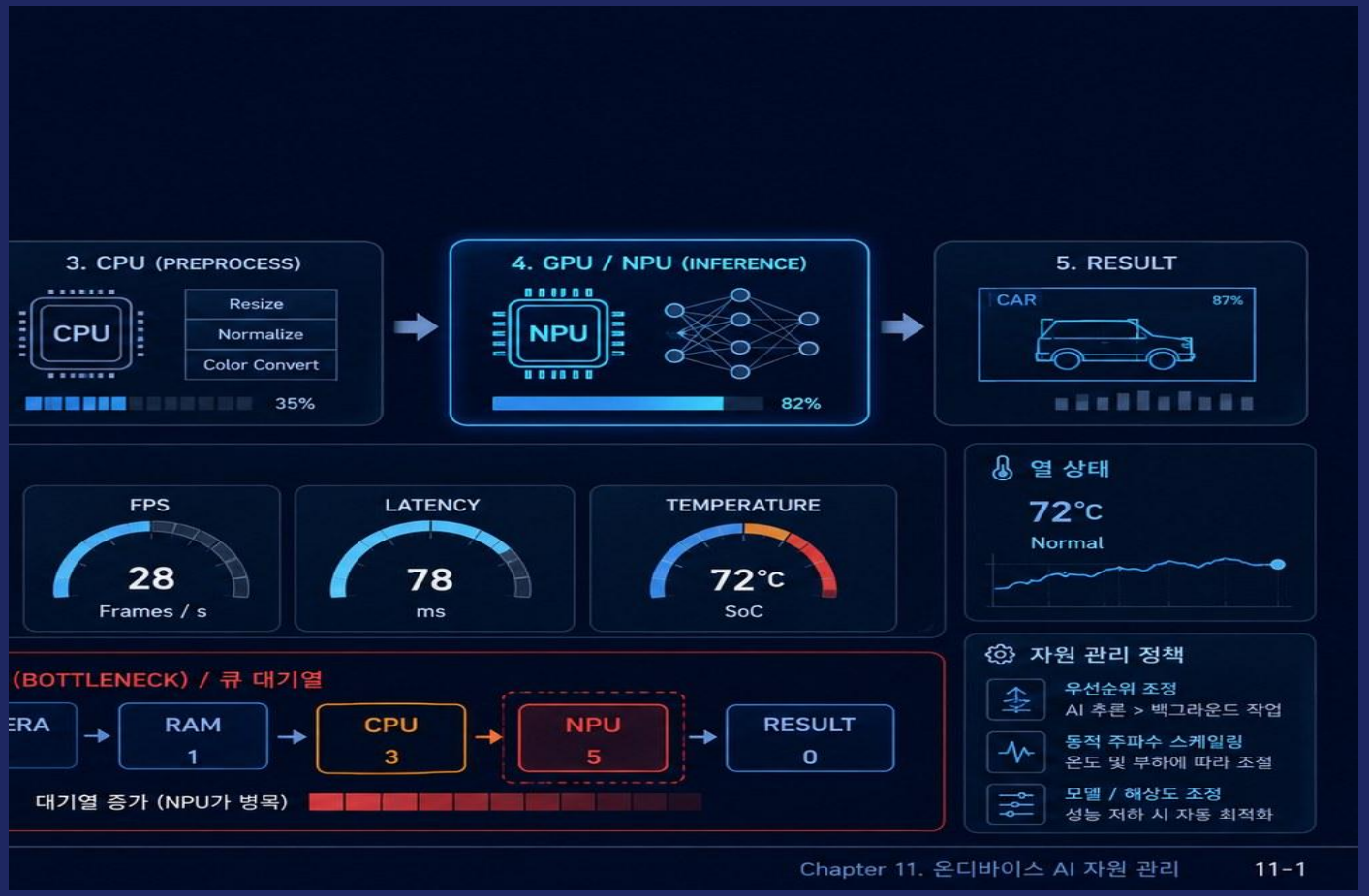
확대해서 볼 부분

2. Throughput / Latency

FPS와 한 Frame 처리시간을 동시에 봐야 시스템 성능을 해석할 수 있습니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

온디바이스 AI 자원 관리 · 복잡한 그림 세부 해설 3





확대해서 볼 부분

3. Bottleneck

가장 느린 단계 앞에 Queue가 쌓이고 뒤 단계 자원 사용률은 오히려 낮을 수 있습니다.

4. Thermal / DVFS

온도가 임계치를 넘으면 Clock을 낮춰 전력·열을 제어하므로 FPS가 감소할 수 있습니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

온디바이스 AI 자원 관리 · 동작 워크플로우

개념을 “정의”가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.



이 워크플로우에서 반드시 관찰할 3가지

- Bottleneck 앞에 Queue가 쌓이고 뒤 자원 사용률이 낮아질 수 있음
- FPS와 Latency는 서로 다른 지표
- 온도 상승은 Clock과 FPS를 떨어뜨릴 수 있음

강사 멈춤 질문

가속기 사용률만 높으면 최적화가 끝난 것인가?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

온디바이스 AI 자원 관리 · 워크플로우 전반부 단계 해설



1	**Camera**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**Frame Buffer**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**CPU Preprocess**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

온디바이스 AI 자원 관리 · 워크플로우 후반부 단계 해설



4	GPU/NPU	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	Postprocess	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	Result	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

GPU 사용률 30%, CPU 95%, 입력 Queue 증가 — GPU를 더 빠르게 하면 해결될까?

SCENARIO

GPU 사용률 30%, CPU 95%, 입력 Queue 증가 — GPU를 더 빠르게 하면 해결될까?

증상만 보고 바로 조치하지 말고 OS 상태 변화를 먼저 추적한다.

1 STEP 1

GPU가 낮은 사용률이라는 것은 입력이 충분히 공급되지 않을 수 있다는 뜻이다.

2 STEP 2

CPU 전처리가 95%라면 GPU가 기다리는 Pipeline Starvation 가능성이 높다.

3 STEP 3

입력 FPS·CPU 처리속도·Queue 길이·Latency를 함께 봐야 한다.

4 STEP 4

최적화는 더 빠른 GPU보다 CPU 전처리 병렬화/경량화가 먼저일 수 있다.

▶ ANIMATION 준비 구간

온디바이스 AI 자원 관리 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
가속기 비교	CPU/GPU/NPU 선택	CPU·Latency·FPS 게이지가 다르게 변함	CPU, RAM, FPS, Latency	가속기 사용률만 높으면 최적화가 끝난 것인가?
Thermal	Load 100%	온도 상승 후 Clock/FPS가 임계치에서 감소	Temp, Clock, FPS	처음 1분과 20분 뒤 성능이 다를 수 있는 이유는?
Bottleneck	CPU 또는 GPU 병목 선택	선택 단계 앞 Queue가 누적	Queue 위치, Utilization	GPU 사용률 30%인데 CPU 95%라면 어디부터 최적화할까?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 11 · 온디바이스 AI 자원 관리

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=11`

수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

인터랙티브 시뮬레이션 · On-device AI Resource + Thermal/Bottleneck Lab

▶ 지금 ANIMATION 수행

HTML SIMULATOR

On-device AI Resource + Thermal/Bottleneck Lab



1. 가속기 CPU/GPU/NPU를 바꾸며 Latency/FPS/CPU 변화 비교
2. 모델 크기 1→5로 올려 RAM과 Latency 변화 관찰
3. Load를 올려 Thermal Throttling 발생 확인
4. Camera/CPU/GPU/Memory Bottleneck을 선택해 Queue 누적 위치 비교

시연 전에 반드시 질문

"버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
."

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 11 · 온디바이스 AI 자원 관리

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1 Balanced/CPU/NPU Bottleneck 비교

2 모델 크기·Load 변화 관찰

3 Thermal Throttle에서 Temp→Clock→FPS 연결

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 11 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. CPU/GPU/NPU 사용률만 보고 결론 내리지 않는 이유 작성
2. FPS와 Latency 차이를 예시로 설명
3. Bottleneck 진단에 필요한 최소 4개 지표 선택

체크 질문

- GPU가 100%라는 사실만으로 병목이라고 단정할 수 있을까?
- Input FPS보다 처리 FPS가 낮으면 Queue는 어떻게 되는가?
- 온도와 성능을 함께 관리해야 하는 이유는?

파일: `os_24h_digital_worksheet.html`

Chapter 12. 종합 장애 분석 실습

증상 → 지표 → 로그 → 가설 → 검증

학습 목표

1. 운영체제 장애를 체계적인 순서로 좁혀 간다.
2. CPU·Memory·Disk·Device·Service 증거를 조합해 근본 원인을 찾는다.
3. 수정 후 반드시 재현 테스트와 지표 검증을 수행한다.

시작 질문

"재부팅해서 해결됨"은 원인 규명인가?



종합 장애 분석 실습 · 그림 전체 보기



그림 읽는 순서

1. 그림 전체에서 무엇이 흐르는지 본다
2. 어디가 OS 경계인지 찾는다
3. 입력·처리·결과의 순서를 잡는다
4. 마지막에 세부 개념을 붙인다

이번 슬라이드의 목적

설명을 그림 위에 겹치지 않고, 먼저 전체 구조를 보이게 합니다.

세부 설명은 다음 2장의 “단계별 해설”에서 나눠 진행합니다.

핵심 개념 4가지

1) 1. 증상
2) 2. 신속 점검
3) 3. 심층 증거
4) 4. 가설·검증

종합 장애 분석 실습 · 단계별 그림 해설 ①



1단계 해설

① 1. 증상

사용자/시스템에서 관찰되는 현상을 수치와 조건으로 명확히 정의

② 2. 신속 점검

top/free/df/ss 등으로 범위를 빠르게 축소

설명 포인트

- 1. 증상가 그림에서 어디에 해당하는지 먼저 찾습니다.
- 이어서 2. 신속 점검가 왜 필요한지 연결합니다.
- 수강생이 그림을 보며 “무엇이 어디를 지나가는가”를 말하게 합니다.

종합 장애 분석 실습 · 단계별 그림 해설 ②



2단계 해설

③ 3. 심층 증거

journal, /proc, /sys, /dev, 서비스 상태로 원인 후보 확인

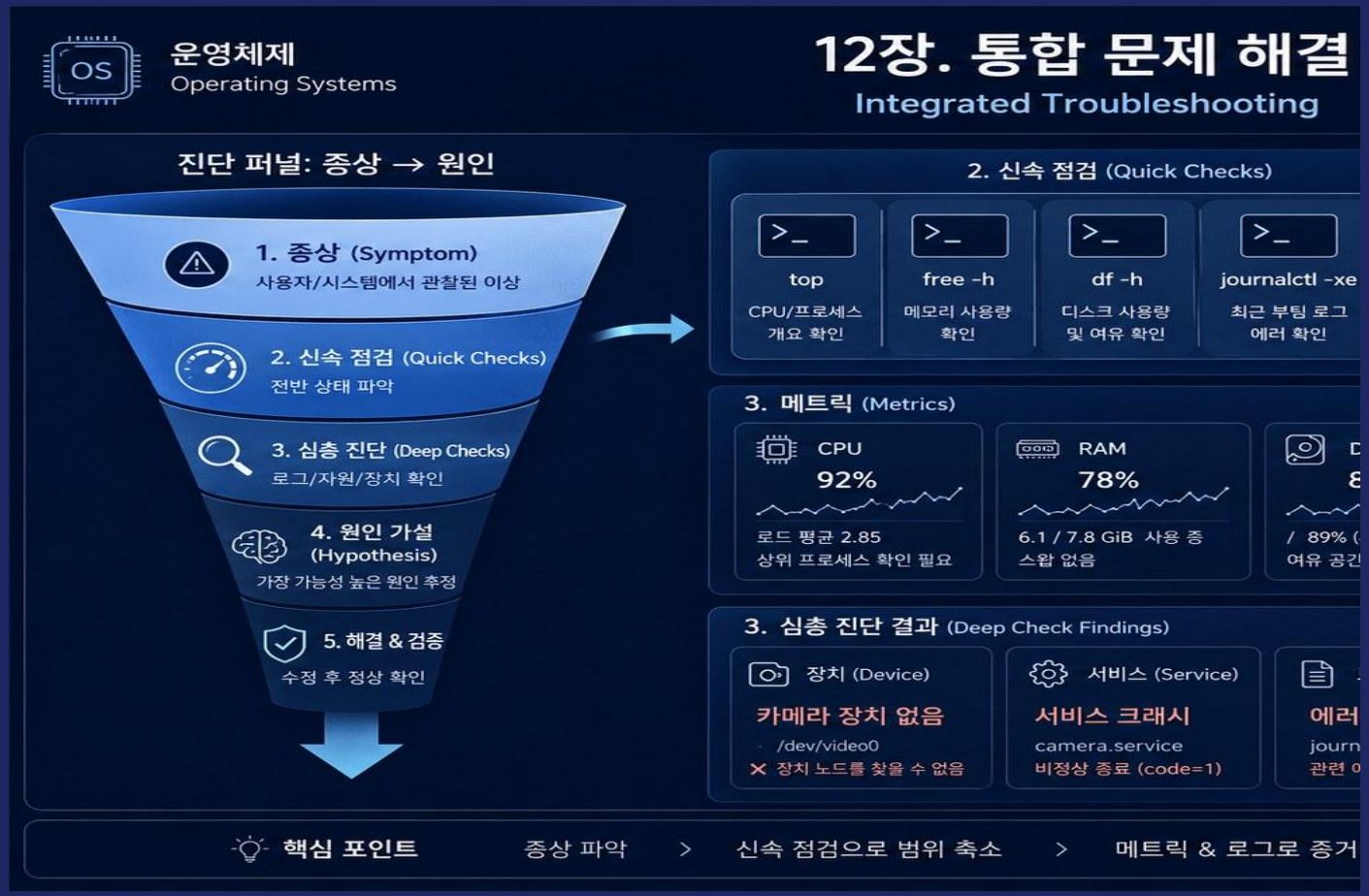
④ 4. 가설·검증

한 번에 하나의 원인 가설을 세우고 조치 전/후를 비교

연결 질문

- 앞 단계와 비교해 어디서 병목이나 충돌이 생길 수 있나요?
- 3. 심층 증거와 4. 가설·검증는 실제 문제 상황에서 어떻게 드러날까요?
- 이제 그림 전체를 한 문장으로 요약하게 합니다.

종합 장애 분석 · 복잡한 그림 세부 해설 1





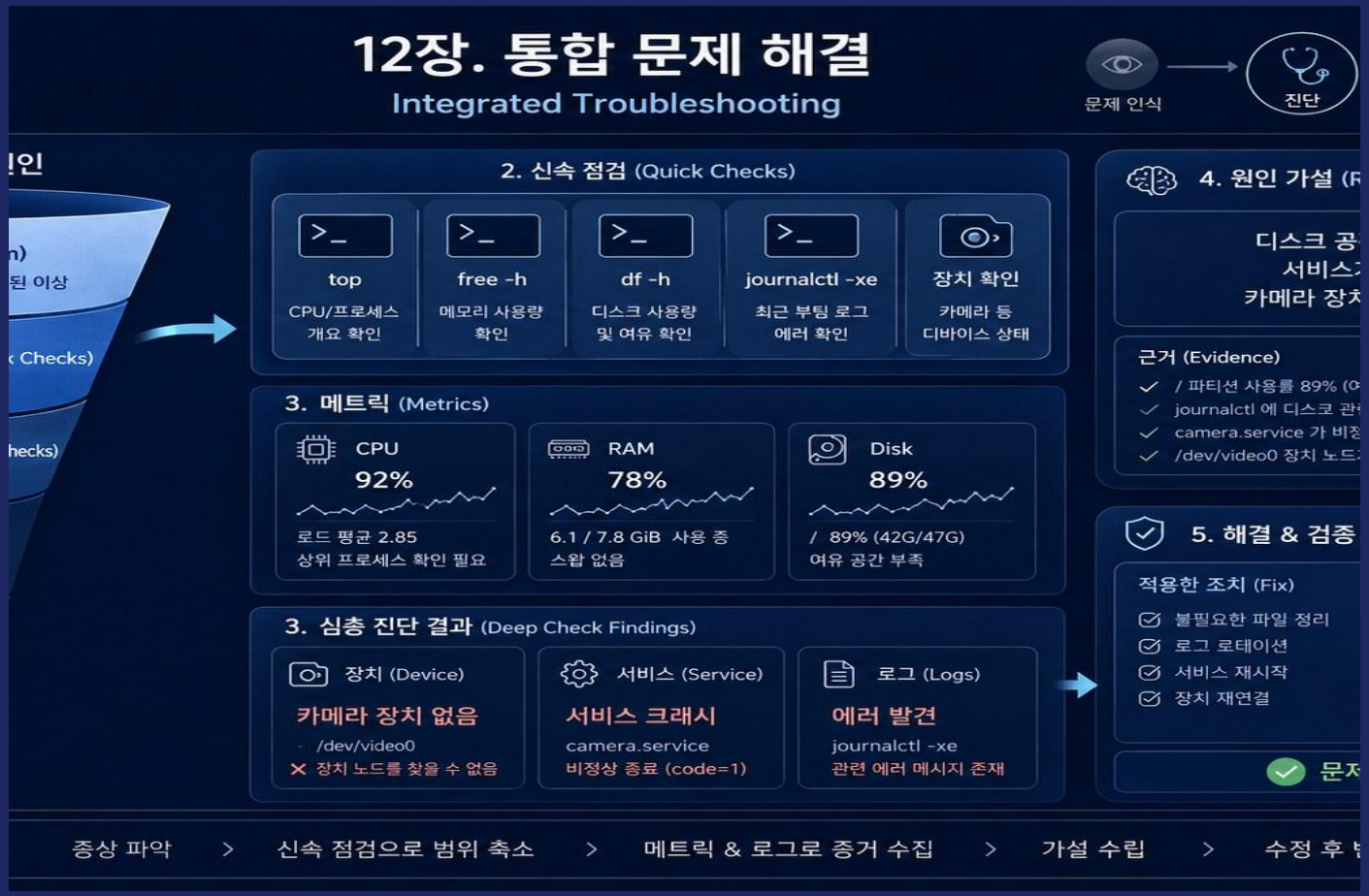
확대해서 볼 부분

1. 증상 정의

"느리다"를 조건·시간·수치가 포함된 재현 가능한 증상으로 바꿉니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 "무엇이 입력이고 무엇이 상태/자원인가?"를 수강생이 직접 가리키게 합니다.

종합 장애 분석 · 복잡한 그림 세부 해설 2





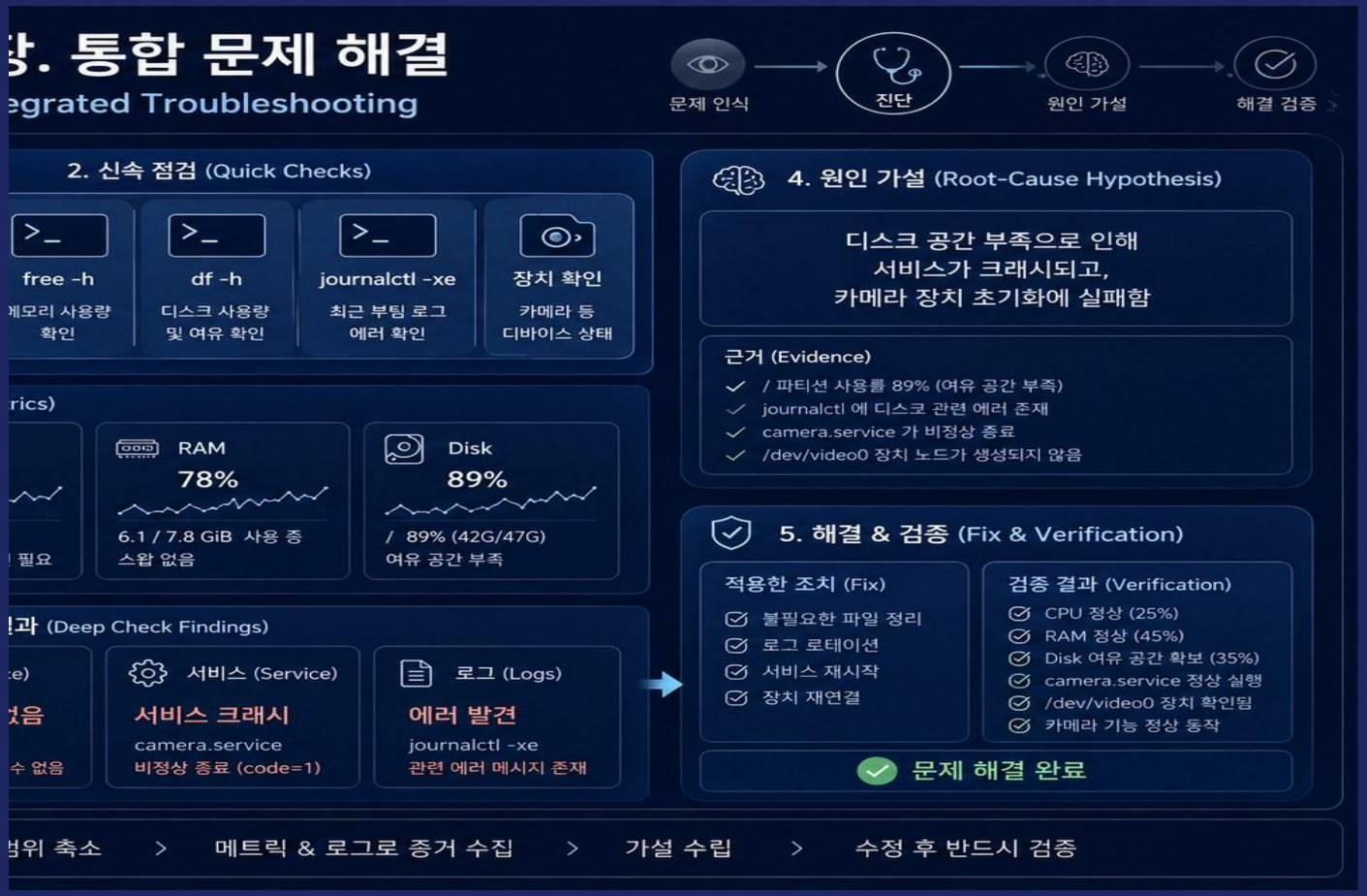
확대해서 볼 부분

2. 신속 점검

top/free/df/ss로 CPU·Memory·Disk·Network 범위를 빠르게 좁힙니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

종합 장애 분석 · 복잡한 그림 세부 해설 3





확대해서 볼 부분

3. 심층 증거

journal·/proc·/sys·/dev·service 상태에서 직접 증거를 찾습니다
.

4. 가설·수정·검증

한 번에 하나의 Root Cause 가설을 검증하고 수정 후 동일 조건으로 재시험합니다.

강사 포인트 그림을 다시 축소해서 전체를 보여주기 전에, 확대 영역에서 “무엇이 입력이고 무엇이 상태/자원인가?”를 수강생이 직접 가리키게 합니다.

종합 장애 분석 실습 · 동작 워크플로우

개념을 "정의"가 아니라 실제 시스템에서 발생하는 순서로 연결합니다.

1. **증상 정의** — 관찰 가능한 증상을 수치화
2. **Quick Check** — CPU/RAM/Disk/Service 범위 축소
3. **Metrics / Logs** — 로그·장치·지표 증거 수집
4. **원인 가설** — 가장 설명력이 높은 원인 후보 선정
5. **Fix** — 한 번에 하나의 조치 적용
6. **동일 조건 재검증** — 같은 입력·부하에서 수정 전후 비교

이 워크플로우에서 반드시 관찰할 3가지

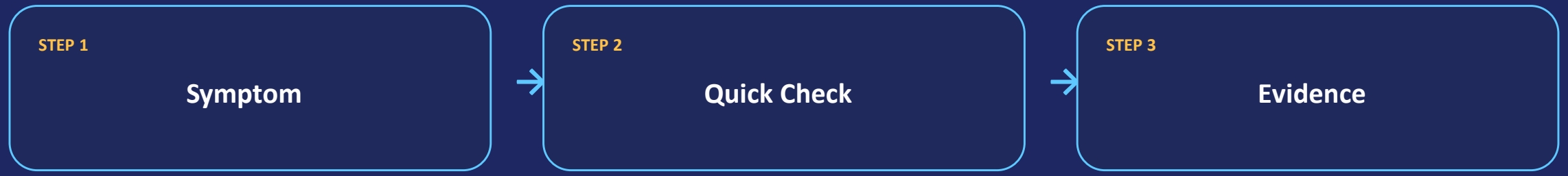
- 증상과 근본 원인을 구분
- 한 번에 하나의 가설만 검증
- 문제가 사라지는 것보다 재현 가능한 검증이 중요

강사 멈춤 질문

첫 번째 이상 지표가 항상 원인일까?

→ 수강생이 흐름의 다음 상태를 먼저 말한 뒤 다음 단계로 이동

종합 장애 분석 · 워크플로우 전반부 단계 해설



1	**Symptom**	무엇이 바뀌나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
2	**Quick Check**	무엇을 관찰하나? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.
3	**Evidence**	다음 단계 조건은? — 상태/자원/Queue의 변화를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

종합 장애 분석 · 워크플로우 후반부 단계 해설



4	**Hypothesis**	무엇이 바뀌나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
5	**Fix**	무엇을 관찰하나? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.
6	**Verify**	다음 단계 조건은? — 결과/오류/복구 조건를 확인하고 로그 또는 지표와 연결합니다.

수강생 역할: 화살표를 따라 다음 상태를 먼저 예측한 뒤, 애니메이션에서 같은 순서가 나타나는지 검산합니다.

통합 사례: AI 카메라 서비스가 느리고 가끔 종료된다

SCENARIO

AI 카메라 서비스가 느리고 가끔 종료된다

증상만 보고 바로 조치하지 말고 OS 상태 변화를 먼저 추적한다.

1 STEP 1

CPU·RAM·Disk 지표를 먼저 수집한다.

2 STEP 2

/dev/video0 존재와 camera.service 상태를 확인한다.

3 STEP 3

journal에서 allocation failure, disk full, timeout 등 직접 증거를 찾는다.

4 STEP 4

가설을 하나 선택해 조치하고 동일 부하에서 재현 테스트한다.

▶ ANIMATION 준비 구간

종합 장애 분석 실습 · 애니메이션 시연 가이드

버튼을 "보여주는 것"이 아니라 조작 → 움직임 → 지표 → 판단의 순서로 시연합니다.

시나리오	사용자 조작	화면에서 움직이는 것	관찰 지표	강사 질문
랜덤 장애	Fault 발생 버튼	CPU/RAM/Disk/Device/Service 중 하나가 비정상 표시	비정상 지표, 로그	첫 번째 이상 지표가 항상 원인일까?
도구 선택	top→free→df→journalctl→/dev	도구를 선택할수록 단서가 누적	증거의 일관성	어떤 단서가 가설을 강화하고 어떤 단서가 반증하는가?
원인 제출·검증	Root Cause 선택 후 복구	결과가 정상으로 돌아오고 재검증	수정 전후 지표	"재부팅 후 됨"과 "원인 해결"의 차이는?

시연 공식 | ① 먼저 예측 → ② 조작 → ③ 움직임을 말로 설명 → ④ 숫자/로그 확인 → ⑤ 원인 문장으로 정리

▶ ANIMATION 수행 구간

Chapter 12 · 종합 장애 분석

지금 PPT 설명을 멈추고 브라우저에서 직접 조작합니다.

수행 순서

① 시나리오 선택 후 결과를 먼저 예측
② Step으로 상태를 한 단계씩 이동
③ Auto Play로 전체 흐름 확인
④ Queue·지표·Event Log를 함께 읽기
⑤ 결과를 한 문장으로 설명 후 Worksheet 이동



파일 `os_24h_interactive_animations_PRO.html?ch=12`

수행 완료 후 PPT로 복귀 → 다음 슬라이드의 DEMO 질문에 답하기

▶ 지금 ANIMATION 수행

인터랙티브 시뮬레이션 · OS 장애 시나리오 + 탐정 최종 시뮬레이터

HTML SIMULATOR

OS 장애 시나리오 + 탐정 최종 시뮬레이터



1. 랜덤 장애 발생 후 CPU/RAM/Disk/로그 관찰
2. 도구 선택 순서 top→free→df→journalctl→/dev로 단서 수집
3. 증상과 원인을 구분해 Root Cause 선택
4. 복구 후 동일 조건으로 FPS/서비스/장치 정상 여부 재검증

시연 전에 반드시 질문

**“버튼을 누르기 전에
무엇이 어떻게 변할지 예측해보세요
.”**

배포 파일

os_24h_interactive_animations_reduced_advanced.html

PPT와 같은 폴더에 두고 브라우저에서 실행

✍ WORKSHEET 수행 구간

Chapter 12 · 종합 장애 분석

애니메이션에서 관찰한 결과를 자신의 판단과 근거로 기록합니다.

반드시 이 순서로 수행

① 예측/판단 문제를 수정하지 않은 상태로 작성
② 「채점하기」를 눌러 답 잠금
③ 오답의 정답 표시를 확인
④ 애니메이션으로 오답 원인 재검산
⑤ 마지막 서술형을 자신의 말로 작성
⑥ 완료 화면을 만들고 점수 확인

완료 조건: 채점 점수 + 서술형 + 완료 화면

이번 Chapter 검증 포인트

1. 랜덤 장애에서 최소 3개 증거 수집
2. 가설을 증거와 연결
3. 수정 후 동일 부하에서 Verify 완료

파일 `os_24h_digital_worksheet.html`

✍ 지금 WORKSHEET 수행

디지털 워크시트 · Chapter 12 실력 검증

진행 규칙

1. 예측/판단 문제를 먼저 작성
2. 「채점하기」로 예측 칸 잠금
3. 틀린 칸의 정답을 보고 원인 확인
4. HTML 시뮬레이터에서 다시 검산
5. 마지막 한 줄 설명을 자신의 언어로 작성

틀린 예측 = 감점이 아니라 학습 증거

오늘의 검증 과제

1. 최종 워크시트에서 메모리 부족·장치 오류·서비스 설정 사례 진단
2. 5단계 진단 순서를 개인 언어로 작성
3. 짝과 진단 근거를 설명하고 반대 가설을 하나 검토

체크 질문

- "재부팅해서 해결됨"은 원인 규명인가?
- 증상과 원인을 혼동하면 어떤 위험이 있는가?
- 조치 후 검증은 어떤 조건에서 해야 하는가?

파일: `os_24h_digital_worksheet.html`

COURSE INTEGRATION

최종 통합: 한 번에 보는 OS 진단 루프

증상 정의 → 신속 점검 → 자원 지표 → 로그/장치 → **원인 가설** → 수정 → 재검증

1 증상

언제, 어떤 조건에서, 얼마나 느리거나 실패하는가?

2 신속 점검

CPU/RAM/Disk/Network/Service 중 어느 영역이 비정상인가?

3 증거

top, free, df, journalctl, /proc, /sys, /dev에서 직접 직접 증거 수집

4 가설

한 번에 하나의 근본 원인 후보를 세우고 반증 가능한 조건 정의

5 검증

동일 부하·동일 입력 조건으로 수정 전/후를 비교

평가 기준: “정답”보다 “근거 있는 판단”

평가 요소	관찰 포인트	권장 비중
개념 이해	핵심 용어를 자신의 말로 설명	25%
상태 예측	스케줄링·Paging·I/O·TCP 흐름의 다음 상태 예측	20%
관찰 능력	지표/로그/장치 상태에서 필요한 증거 선택	20%
진단 논리	증상→가설→증거→검증 순서로 원인 좁히기	25%
설명/협업	근거를 짝에게 설명하고 반대 가설 검토	10%

권장 최종 실습

“온디바이스 AI 실행 환경에서 OS 문제 5가지를 진단하고, 각 문제마다 증거·원인·조치·재검증 결과를 기록 기록한다.”

다음 과정으로 연결하기

⌨️ Linux & Shell

오늘 배운 OS 상태를 더 빠르게 탐색하고 자동화하는 도구

🔌 Embedded I/O

Driver·Interrupt·DMA·Buffer 개념을 실제 장치 제어에 적용

⚡ ONNX · TensorRT

CPU/GPU/NPU·RAM·Latency·Thermal 관점으로 추론 최적화

🤖 ROS2

Process/Thread/Network/Service 개념을 분산 로봇 시스템에 확장

운영체제는 “한 과목”이 아니라 이후 모든 임베디드·AI 실습의 공통 디버깅 언어입니다.